base44/entities

Analysis.jsonc

```
{
  "name": "Analysis",
  "type": "object",
  "properties": {
    "location_name": {
      "type": "string",
      "title": "Location Name",
      "description": "Human-readable name of the analyzed location"
    },
    "lat": {
      "type": "number",
      "title": "Latitude"
    },
    "lng": {
      "type": "number",
      "title": "Longitude"
    },
    "area_hectares": {
      "type": "number",
      "title": "Area (hectares)"
    },
    "boundary": {
      "type": "array",
      "title": "Boundary Polygon",
      "description": "Array of [lat, lng] vertices defining the analyzed
area",
      "items": {
        "type": "array",
        "items": {
          "type": "number"
        },
        "minItems": 2,
        "maxItems": 2
      }
    },
    "vegetation_pct": {
      "type": "number",
      "title": "Vegetation %"
    },
    "buildings_pct": {
      "type": "number",
      "title": "Buildings %"
    },
```

```
  "roads_pct": {
    "type": "number",
    "title": "Roads %"
  },
  "open_land_pct": {
    "type": "number",
    "title": "Open Land %"
  },
  "water_pct": {
    "type": "number",
    "title": "Water %"
  },
  "carbon_impact_score": {
    "type": "number",
    "title": "Carbon Impact Score (0-100)"
  },
  "carbon_absorption_tons_year": {
    "type": "number",
    "title": "Carbon Absorption (tons CO2/yr)"
  },
  "tree_canopy_estimate": {
    "type": "number",
    "title": "Estimated Tree Equivalents"
  },
  "sustainability_score": {
    "type": "number",
    "title": "Sustainability Score (0-100)"
  },
  "green_cover_rating": {
    "type": "string",
    "title": "Green Cover Rating",
    "enum": [
      "Excellent",
      "Good",
      "Moderate",
      "Low",
      "Poor"
    ]
  },
  "summary": {
    "type": "string",
    "title": "Summary"
  },
  "recommendations": {
    "type": "array",
```

```
      "title": "Recommendations",
      "items": {
        "type": "string"
      }
    },
    "land_use_breakdown": {
      "type": "array",
      "title": "Land Use Breakdown",
      "items": {
        "type": "object",
        "properties": {
          "category": {
            "type": "string"
          },
          "pct": {
            "type": "number"
          },
          "color": {
            "type": "string"
          }
        }
      }
    }
  },
  "required": [
    "location_name",
    "lat",
    "lng",
    "area_hectares"
  ],
  "rls": {
    "read": {
      "$or": [
        {
          "created_by_id": "{{user.id}}"
        },
        {
          "user_condition": {
            "role": "admin"
          }
        }
      ]
    },
    "update": {
      "$or": [
```

```
        {
          "created_by_id": "{{user.id}}"
        },
        {
          "user_condition": {
            "role": "admin"
          }
        }
      ]
    },
    "delete": {
      "$or": [
        {
          "created_by_id": "{{user.id}}"
        },
        {
          "user_condition": {
            "role": "admin"
          }
        }
      ]
    }
  }
}
```

User.jsonc

```
{
  "properties": {
    "role": {
      "type": "string",
      "description": "The role of the user in the app",
      "enum": [
        "admin",
        "user"
      ]
    }
  },
  "required": [
    "role"
  ],
```

```
  "name": "User",
  "type": "object"
}
```

functions /analyzeArea
entry.ts

```
import { createClientFromRequest } from 'npm:@base44/sdk@0.8.52';

export default async function (req: Request): Promise<Response> {
  try {
    const base44 = createClientFromRequest(req);
    const user = await base44.auth.me();
    if (!user) return Response.json({ error: 'Unauthorized' }, { status: 401
});

    const body = await req.json().catch(() => ({}));
    const locationName = String(body?.location_name || '').trim();
    const lat = parseFloat(body?.lat);
    const lng = parseFloat(body?.lng);
    const areaHectares = parseFloat(body?.area_hectares);

    if (!locationName || isNaN(lat) || isNaN(lng) || isNaN(areaHectares)) {
      return Response.json({ error: 'Missing or invalid parameters' }, {
status: 400 });
    }
    if (areaHectares <= 0 || areaHectares > 50000) {
      return Response.json({ error: 'Area out of supported range (0-50,000
ha)' }, { status: 400 });
    }
    if (lat < -90 || lat > 90 || lng < -180 || lng > 180) {
      return Response.json({ error: 'Invalid coordinates' }, { status: 400
});
    }

    const prompt = `You are a geospatial environmental analyst. Analyze the
land cover and environmental condition of the area near "${locationName}"
(centered at latitude ${lat}, longitude ${lng}), covering approximately
${areaHectares.toFixed(2)} hectares.

Use your knowledge of this location's geography, urbanization, climate, and
vegetation to produce an INDICATIVE land-cover distribution and environmental
assessment. These are planning-grade estimates, not precise measurements.

Return JSON with these fields:
```

```
- vegetation_pct: estimated % covered by trees, shrubs, and grass (0-100)
- buildings_pct: estimated % covered by built structures (0-100)
- roads_pct: estimated % covered by roads and paved surfaces (0-100)
- open_land_pct: estimated % of bare / open soil or barren land (0-100)
- water_pct: estimated % covered by water bodies (0-100)
- The five percentages should sum to approximately 100.
- carbon_impact_score: 0-100, higher = greater carbon footprint /
environmental degradation
- carbon_absorption_tons_year: estimated tonnes of CO2 absorbed per year by
vegetation in this area
- tree_canopy_estimate: estimated number of mature-tree equivalents in this
area
- sustainability_score: 0-100, higher = more environmentally sustainable
- green_cover_rating: one of "Excellent", "Good", "Moderate", "Low", "Poor"
- summary: 2-3 sentence plain-language summary of the area's environmental
condition
- recommendations: array of 3-5 specific, actionable sustainability
recommendations
- land_use_breakdown: array of {category, pct, color} for charting —
categories are "Vegetation", "Buildings", "Roads", "Open Land", "Water" and
color is a hex color suited to each category.

Be realistic and grounded in the actual characteristics of this location.`;

    const schema = {
      type: 'object',
      properties: {
        vegetation_pct: { type: 'number' },
        buildings_pct: { type: 'number' },
        roads_pct: { type: 'number' },
        open_land_pct: { type: 'number' },
        water_pct: { type: 'number' },
        carbon_impact_score: { type: 'number' },
        carbon_absorption_tons_year: { type: 'number' },
        tree_canopy_estimate: { type: 'number' },
        sustainability_score: { type: 'number' },
        green_cover_rating: { type: 'string' },
        summary: { type: 'string' },
        recommendations: { type: 'array', items: { type: 'string' } },
        land_use_breakdown: {
          type: 'array',
          items: {
            type: 'object',
            properties: {
              category: { type: 'string' },
```

```
              pct: { type: 'number' },
              color: { type: 'string' }
            }
          }
        }
      },
      required: ['vegetation_pct', 'buildings_pct', 'roads_pct',
'open_land_pct', 'water_pct', 'carbon_impact_score',
'carbon_absorption_tons_year', 'tree_canopy_estimate',
'sustainability_score', 'green_cover_rating', 'summary', 'recommendations',
'land_use_breakdown']
    };

    const result = await base44.asServiceRole.integrations.Core.InvokeLLM({
      prompt,
      add_context_from_internet: true,
      model: 'gemini_3_flash',
      response_json_schema: schema
    });

    return Response.json({ analysis: result });
  } catch (error) {
    return Response.json({ error: (error as Error).message }, { status: 500
});
  }
}
```

geocodeLocation

```
import { createClientFromRequest } from 'npm:@base44/sdk@0.8.52';

export default async function (req: Request): Promise<Response> {
  try {
    const base44 = createClientFromRequest(req);
    const user = await base44.auth.me();
    if (!user) return Response.json({ error: 'Unauthorized' }, { status: 401
});

    const body = await req.json().catch(() => ({}));
    const query = String(body?.query || '').trim();
    if (!query || query.length < 2) {
      return Response.json({ error: 'Query too short' }, { status: 400 });
    }

    const url =
`https://nominatim.openstreetmap.org/search?format=jsonv2&addressdetails=1&li
mit=6&q=${encodeURIComponent(query)}`;
```

```
    const res = await fetch(url, {
      headers: { 'User-Agent': 'GreenScopeAI/1.0 (base44 environmental
analysis app)' },
      signal: AbortSignal.timeout(10000)
    });
    if (!res.ok) return Response.json({ error: 'Geocoding service
unavailable' }, { status: 502 });

    const data = await res.json();
    const results = (Array.isArray(data) ? data : []).map((r: any) => ({
      display_name: r.display_name,
      lat: parseFloat(r.lat),
      lng: parseFloat(r.lon),
      type: r.type,
      class: r.class
    })).filter((r: any) => !isNaN(r.lat) && !isNaN(r.lng));

    return Response.json({ results });
  } catch (error) {
    return Response.json({ error: (error as Error).message }, { status: 500
});
  }
}
```

config.jsonc

```
{
  "name": "base44-app",
  "site": {
    "installCommand": "npm install",
    "buildCommand": "npm run build",
    "serveCommand": "npm run dev",
    "outputDirectory": "./dist"
  }
}
```

/src/api

base44Client.js

```
import { createClient } from '@base44/sdk';
import { appParams } from '@/lib/app-params';

const { appId, token, functionsVersion, appBaseUrl } = appParams;
```

```
export const base44 = createClient({
  appId,
  token,
  functionsVersion,
  serverUrl: '',
  appBaseUrl
});
```

/src/components
/ui
accordion.jsx

```
import * as React from "react"
import * as AccordionPrimitive from "@radix-ui/react-accordion"
import { ChevronDown } from "lucide-react"

import { cn } from "@/lib/utils"

const Accordion = AccordionPrimitive.Root

const AccordionItem = React.forwardRef(({ className, ...props }, ref) => (
  <AccordionPrimitive.Item ref={ref} className={cn("border-b", className)}
{...props} />
))
AccordionItem.displayName = "AccordionItem"

const AccordionTrigger = React.forwardRef(({ className, children, ...props },
ref) => (
  <AccordionPrimitive.Header className="flex">
    <AccordionPrimitive.Trigger
      ref={ref}
      className={cn(
        "flex flex-1 items-center justify-between py-4 text-sm font-medium
transition-all hover:underline text-left
[&[data-state=open]>svg]:rotate-180",
        className
      )}
      {...props}>
      {children}
      <ChevronDown
        className="h-4 w-4 shrink-0 text-muted-foreground
transition-transform duration-200" />
    </AccordionPrimitive.Trigger>
```

```
  </AccordionPrimitive.Header>
))
AccordionTrigger.displayName = AccordionPrimitive.Trigger.displayName

const AccordionContent = React.forwardRef(({ className, children, ...props },
ref) => (
  <AccordionPrimitive.Content
    ref={ref}
    className="overflow-hidden text-sm
data-[state=closed]:animate-accordion-up
data-[state=open]:animate-accordion-down"
    {...props}>
    <div className={cn("pb-4 pt-0", className)}>{children}</div>
  </AccordionPrimitive.Content>
))
AccordionContent.displayName = AccordionPrimitive.Content.displayName

export { Accordion, AccordionItem, AccordionTrigger, AccordionContent }
```

alert-dialog.jsx

```
import * as React from "react"
import * as AlertDialogPrimitive from "@radix-ui/react-alert-dialog"

import { cn } from "@/lib/utils"
import { buttonVariants } from "@/components/ui/button"

const AlertDialog = AlertDialogPrimitive.Root

const AlertDialogTrigger = AlertDialogPrimitive.Trigger

const AlertDialogPortal = AlertDialogPrimitive.Portal

const AlertDialogOverlay = React.forwardRef(({ className, ...props }, ref) =>
(
  <AlertDialogPrimitive.Overlay
    className={cn(
      "fixed inset-0 z-50 bg-black/80 data-[state=open]:animate-in
data-[state=closed]:animate-out data-[state=closed]:fade-out-0
data-[state=open]:fade-in-0",
      className
    )}
    {...props}
    ref={ref} />
))
```

```
AlertDialogOverlay.displayName = AlertDialogPrimitive.Overlay.displayName

const AlertDialogContent = React.forwardRef(({ className, ...props }, ref) =>
(
  <AlertDialogPortal>
    <AlertDialogOverlay />
    <AlertDialogPrimitive.Content
      ref={ref}
      className={cn(
        "fixed left-[50%] top-[50%] z-50 grid w-full max-w-lg
translate-x-[-50%] translate-y-[-50%] gap-4 border bg-background p-6
shadow-lg duration-200 data-[state=open]:animate-in
data-[state=closed]:animate-out data-[state=closed]:fade-out-0
data-[state=open]:fade-in-0 data-[state=closed]:zoom-out-95
data-[state=open]:zoom-in-95 data-[state=closed]:slide-out-to-left-1/2
data-[state=closed]:slide-out-to-top-[48%]
data-[state=open]:slide-in-from-left-1/2
data-[state=open]:slide-in-from-top-[48%] sm:rounded-lg",
        className
      )}
      {...props} />
  </AlertDialogPortal>
))
AlertDialogContent.displayName = AlertDialogPrimitive.Content.displayName

const AlertDialogHeader = ({
  className,
  ...props
}) => (
  <div
    className={cn("flex flex-col space-y-2 text-center sm:text-left",
className)}
    {...props} />
)
AlertDialogHeader.displayName = "AlertDialogHeader"

const AlertDialogFooter = ({
  className,
  ...props
}) => (
  <div
    className={cn("flex flex-col-reverse sm:flex-row sm:justify-end
sm:space-x-2", className)}
    {...props} />
)
```

```
AlertDialogFooter.displayName = "AlertDialogFooter"

const AlertDialogTitle = React.forwardRef(({ className, ...props }, ref) => (
  <AlertDialogPrimitive.Title ref={ref} className={cn("text-lg
font-semibold", className)} {...props} />
))
AlertDialogTitle.displayName = AlertDialogPrimitive.Title.displayName

const AlertDialogDescription = React.forwardRef(({ className, ...props },
ref) => (
  <AlertDialogPrimitive.Description
    ref={ref}
    className={cn("text-sm text-muted-foreground", className)}
    {...props} />
))
AlertDialogDescription.displayName =
  AlertDialogPrimitive.Description.displayName

const AlertDialogAction = React.forwardRef(({ className, ...props }, ref) =>
(
  <AlertDialogPrimitive.Action ref={ref} className={cn(buttonVariants(),
className)} {...props} />
))
AlertDialogAction.displayName = AlertDialogPrimitive.Action.displayName

const AlertDialogCancel = React.forwardRef(({ className, ...props }, ref) =>
(
  <AlertDialogPrimitive.Cancel
    ref={ref}
    className={cn(buttonVariants({ variant: "outline" }), "mt-2 sm:mt-0",
className)}
    {...props} />
))
AlertDialogCancel.displayName = AlertDialogPrimitive.Cancel.displayName

export {
  AlertDialog,
  AlertDialogPortal,
  AlertDialogOverlay,
  AlertDialogTrigger,
  AlertDialogContent,
  AlertDialogHeader,
  AlertDialogFooter,
  AlertDialogTitle,
  AlertDialogDescription,
```

```
  AlertDialogAction,
  AlertDialogCancel,
}
```

alert.jsx

```
import * as React from "react"
import { cva } from "class-variance-authority";

import { cn } from "@/lib/utils"

const alertVariants = cva(
  "relative w-full rounded-lg border px-4 py-3 text-sm
[&>svg+div]:translate-y-[-3px] [&>svg]:absolute [&>svg]:left-4 [&>svg]:top-4
[&>svg]:text-foreground [&>svg~*]:pl-7",
  {
    variants: {
      variant: {
        default: "bg-background text-foreground",
        destructive:
          "border-destructive/50 text-destructive dark:border-destructive
[&>svg]:text-destructive",
      },
    },
    defaultVariants: {
      variant: "default",
    },
  }
)

const Alert = React.forwardRef(({ className, variant, ...props }, ref) => (
  <div
    ref={ref}
    role="alert"
    className={cn(alertVariants({ variant }), className)}
    {...props} />
))
Alert.displayName = "Alert"

const AlertTitle = React.forwardRef(({ className, ...props }, ref) => (
  <h5
    ref={ref}
    className={cn("mb-1 font-medium leading-none tracking-tight", className)}
```

```
    {...props} />
))
AlertTitle.displayName = "AlertTitle"

const AlertDescription = React.forwardRef(({ className, ...props }, ref) => (
  <div
    ref={ref}
    className={cn("text-sm [&_p]:leading-relaxed", className)}
    {...props} />
))
AlertDescription.displayName = "AlertDescription"

export { Alert, AlertTitle, AlertDescription }
```

aspect-ratio.jsx

```
import * as AspectRatioPrimitive from "@radix-ui/react-aspect-ratio"

const AspectRatio = AspectRatioPrimitive.Root

export { AspectRatio }
```

avatar.jsx

```
"use client"

import * as React from "react"
import * as AvatarPrimitive from "@radix-ui/react-avatar"

import { cn } from "@/lib/utils"

const Avatar = React.forwardRef(({ className, ...props }, ref) => (
  <AvatarPrimitive.Root
    ref={ref}
    className={cn("relative flex h-10 w-10 shrink-0 overflow-hidden
rounded-full", className)}
    {...props} />
))
Avatar.displayName = AvatarPrimitive.Root.displayName

const AvatarImage = React.forwardRef(({ className, ...props }, ref) => (
  <AvatarPrimitive.Image
    ref={ref}
```

```
    className={cn("aspect-square h-full w-full", className)}
    {...props} />
))
AvatarImage.displayName = AvatarPrimitive.Image.displayName

const AvatarFallback = React.forwardRef(({ className, ...props }, ref) => (
  <AvatarPrimitive.Fallback
    ref={ref}
    className={cn(
      "flex h-full w-full items-center justify-center rounded-full bg-muted",
      className
    )}
    {...props} />
))
AvatarFallback.displayName = AvatarPrimitive.Fallback.displayName

export { Avatar, AvatarImage, AvatarFallback }
```

badge.jsx

```
import * as React from "react"
import { cva } from "class-variance-authority";

import { cn } from "@/lib/utils"

const badgeVariants = cva(
  "inline-flex items-center rounded-md border px-2.5 py-0.5 text-xs
font-semibold transition-colors focus:outline-none focus:ring-2
focus:ring-ring focus:ring-offset-2",
  {
    variants: {
      variant: {
        default:
          "border-transparent bg-primary text-primary-foreground shadow
hover:bg-primary/80",
        secondary:
          "border-transparent bg-secondary text-secondary-foreground
hover:bg-secondary/80",
        destructive:
          "border-transparent bg-destructive text-destructive-foreground
shadow hover:bg-destructive/80",
        outline: "text-foreground",
      },
    },
```

```
    defaultVariants: {
      variant: "default",
    },
  }
)

function Badge({
  className,
  variant,
  ...props
}) {
  return (<div className={cn(badgeVariants({ variant }), className)}
{...props} />);
}

export { Badge, badgeVariants }
```

breadcrumb.jsx

```
import * as React from "react"
import { Slot } from "@radix-ui/react-slot"
import { ChevronRight, MoreHorizontal } from "lucide-react"

import { cn } from "@/lib/utils"

const Breadcrumb = React.forwardRef(
  ({ ...props }, ref) => <nav ref={ref} aria-label="breadcrumb" {...props} />
)
Breadcrumb.displayName = "Breadcrumb"

const BreadcrumbList = React.forwardRef(({ className, ...props }, ref) => (
  <ol
    ref={ref}
    className={cn(
      "flex flex-wrap items-center gap-1.5 break-words text-sm
text-muted-foreground sm:gap-2.5",
      className
    )}
    {...props} />
))
BreadcrumbList.displayName = "BreadcrumbList"

const BreadcrumbItem = React.forwardRef(({ className, ...props }, ref) => (
  <li
    ref={ref}
    className={cn("inline-flex items-center gap-1.5", className)}
```

```
    {...props} />
))
BreadcrumbItem.displayName = "BreadcrumbItem"

const BreadcrumbLink = React.forwardRef(({ asChild, className, ...props },
ref) => {
  const Comp = asChild ? Slot : "a"

  return (
    (<Comp
      ref={ref}
      className={cn("transition-colors hover:text-foreground", className)}
      {...props} />)
  );
})
BreadcrumbLink.displayName = "BreadcrumbLink"

const BreadcrumbPage = React.forwardRef(({ className, ...props }, ref) => (
  <span
    ref={ref}
    role="link"
    aria-disabled="true"
    aria-current="page"
    className={cn("font-normal text-foreground", className)}
    {...props} />
))
BreadcrumbPage.displayName = "BreadcrumbPage"

const BreadcrumbSeparator = ({
  children,
  className,
  ...props
}) => (
  <li
    role="presentation"
    aria-hidden="true"
    className={cn("[&>svg]:w-3.5 [&>svg]:h-3.5", className)}
    {...props}>
    {children ?? <ChevronRight />}
  </li>
)
BreadcrumbSeparator.displayName = "BreadcrumbSeparator"

const BreadcrumbEllipsis = ({
  className,
```

```
  ...props
}) => (
  <span
    role="presentation"
    aria-hidden="true"
    className={cn("flex h-9 w-9 items-center justify-center", className)}
    {...props}>
    <MoreHorizontal className="h-4 w-4" />
    <span className="sr-only">More</span>
  </span>
)
BreadcrumbEllipsis.displayName = "BreadcrumbElipssis"

export {
  Breadcrumb,
  BreadcrumbList,
  BreadcrumbItem,
  BreadcrumbLink,
  BreadcrumbPage,
  BreadcrumbSeparator,
  BreadcrumbEllipsis,
}
```

button.jsx

```
import * as React from "react"
import { Slot } from "@radix-ui/react-slot"
import { cva } from "class-variance-authority";

import { cn } from "@/lib/utils"

const buttonVariants = cva(
  "inline-flex items-center justify-center gap-2 whitespace-nowrap rounded-md
text-sm font-medium transition-colors focus-visible:outline-none
focus-visible:ring-1 focus-visible:ring-ring disabled:pointer-events-none
disabled:opacity-50 [&_svg]:pointer-events-none [&_svg]:size-4
[&_svg]:shrink-0",
  {
    variants: {
      variant: {
        default:
          "bg-primary text-primary-foreground shadow hover:bg-primary/90",
        destructive:
          "bg-destructive text-destructive-foreground shadow-sm
hover:bg-destructive/90",
        outline:
```

```
          "border border-input bg-transparent shadow-sm hover:bg-accent
hover:text-accent-foreground",
        secondary:
          "bg-secondary text-secondary-foreground shadow-sm
hover:bg-secondary/80",
        ghost: "hover:bg-accent hover:text-accent-foreground",
        link: "text-primary underline-offset-4 hover:underline",
      },
      size: {
        default: "h-9 px-4 py-2",
        sm: "h-8 rounded-md px-3 text-xs",
        lg: "h-10 rounded-md px-8",
        icon: "h-9 w-9",
      },
    },
    defaultVariants: {
      variant: "default",
      size: "default",
    },
  }
)

const Button = React.forwardRef(({ className, variant, size, asChild = false,
...props }, ref) => {
  const Comp = asChild ? Slot : "button"
  return (
    (<Comp
      className={cn(buttonVariants({ variant, size, className }))}
      ref={ref}
      {...props} />)
  );
})
Button.displayName = "Button"

export { Button, buttonVariants }
```

calendar.jsx

```
import * as React from "react"
import { ChevronLeft, ChevronRight } from "lucide-react"
import { DayPicker } from "react-day-picker"

import { cn } from "@/lib/utils"
import { buttonVariants } from "@/components/ui/button"

function Calendar({
  className,
```

```
  classNames,
  showOutsideDays = true,
  ...props
}) {
  return (
    (<DayPicker
      showOutsideDays={showOutsideDays}
      className={cn("p-3", className)}
      classNames={{
        months: "flex flex-col sm:flex-row space-y-4 sm:space-x-4
sm:space-y-0",
        month: "space-y-4",
        caption: "flex justify-center pt-1 relative items-center",
        caption_label: "text-sm font-medium",
        nav: "space-x-1 flex items-center",
        nav_button: cn(
          buttonVariants({ variant: "outline" }),
          "h-7 w-7 bg-transparent p-0 opacity-50 hover:opacity-100"
        ),
        nav_button_previous: "absolute left-1",
        nav_button_next: "absolute right-1",
        table: "w-full border-collapse space-y-1",
        head_row: "flex",
        head_cell:
          "text-muted-foreground rounded-md w-8 font-normal text-[0.8rem]",
        row: "flex w-full mt-2",
        cell: cn(
          "relative p-0 text-center text-sm focus-within:relative
focus-within:z-20 [&:has([aria-selected])]:bg-accent
[&:has([aria-selected].day-outside)]:bg-accent/50
[&:has([aria-selected].day-range-end)]:rounded-r-md",
          props.mode === "range"
            ? "[&:has(>.day-range-end)]:rounded-r-md
[&:has(>.day-range-start)]:rounded-l-md
first:[&:has([aria-selected])]:rounded-l-md
last:[&:has([aria-selected])]:rounded-r-md"
            : "[&:has([aria-selected])]:rounded-md"
        ),
        day: cn(
          buttonVariants({ variant: "ghost" }),
          "h-8 w-8 p-0 font-normal aria-selected:opacity-100"
        ),
        day_range_start: "day-range-start",
        day_range_end: "day-range-end",
        day_selected:
```

```
          "bg-primary text-primary-foreground hover:bg-primary
hover:text-primary-foreground focus:bg-primary
focus:text-primary-foreground",
        day_today: "bg-accent text-accent-foreground",
        day_outside:
          "day-outside text-muted-foreground aria-selected:bg-accent/50
aria-selected:text-muted-foreground",
        day_disabled: "text-muted-foreground opacity-50",
        day_range_middle:
          "aria-selected:bg-accent aria-selected:text-accent-foreground",
        day_hidden: "invisible",
        ...classNames,
      }}
      components={{
        IconLeft: ({ className, ...props }) => (
          <ChevronLeft className={cn("h-4 w-4", className)} {...props} />
        ),
        IconRight: ({ className, ...props }) => (
          <ChevronRight className={cn("h-4 w-4", className)} {...props} />
        ),
      }}
      {...props} />)
  );
}
Calendar.displayName = "Calendar"

export { Calendar }
```

card.jsx

```
import * as React from "react"

import { cn } from "@/lib/utils"

const Card = React.forwardRef(({ className, ...props }, ref) => (
  <div
    ref={ref}
    className={cn("rounded-xl border bg-card text-card-foreground shadow",
className)}
    {...props} />
))
Card.displayName = "Card"

const CardHeader = React.forwardRef(({ className, ...props }, ref) => (
  <div
```

```
    ref={ref}
    className={cn("flex flex-col space-y-1.5 p-6", className)}
    {...props} />
))
CardHeader.displayName = "CardHeader"

const CardTitle = React.forwardRef(({ className, ...props }, ref) => (
  <div
    ref={ref}
    className={cn("font-semibold leading-none tracking-tight", className)}
    {...props} />
))
CardTitle.displayName = "CardTitle"

const CardDescription = React.forwardRef(({ className, ...props }, ref) => (
  <div
    ref={ref}
    className={cn("text-sm text-muted-foreground", className)}
    {...props} />
))
CardDescription.displayName = "CardDescription"

const CardContent = React.forwardRef(({ className, ...props }, ref) => (
  <div ref={ref} className={cn("p-6 pt-0", className)} {...props} />
))
CardContent.displayName = "CardContent"

const CardFooter = React.forwardRef(({ className, ...props }, ref) => (
  <div
    ref={ref}
    className={cn("flex items-center p-6 pt-0", className)}
    {...props} />
))
CardFooter.displayName = "CardFooter"

export { Card, CardHeader, CardFooter, CardTitle, CardDescription,
CardContent }
```

carousel.jsx

```
import * as React from "react"
import useEmblaCarousel from "embla-carousel-react";
import { ArrowLeft, ArrowRight } from "lucide-react"

import { cn } from "@/lib/utils"
```

```
import { Button } from "@/components/ui/button"

const CarouselContext = React.createContext(null)

function useCarousel() {
  const context = React.useContext(CarouselContext)

  if (!context) {
    throw new Error("useCarousel must be used within a <Carousel />")
  }

  return context
}

const Carousel = React.forwardRef((
  {
    orientation = "horizontal",
    opts,
    setApi,
    plugins,
    className,
    children,
    ...props
  },
  ref
) => {
  const [carouselRef, api] = useEmblaCarousel({
    ...opts,
    axis: orientation === "horizontal" ? "x" : "y",
  }, plugins)
  const [canScrollPrev, setCanScrollPrev] = React.useState(false)
  const [canScrollNext, setCanScrollNext] = React.useState(false)

  const onSelect = React.useCallback((api) => {
    if (!api) {
      return
    }

    setCanScrollPrev(api.canScrollPrev())
    setCanScrollNext(api.canScrollNext())
  }, [])

  const scrollPrev = React.useCallback(() => {
    api?.scrollPrev()
  }, [api])
```

```
const scrollNext = React.useCallback(() => {
  api?.scrollNext()
}, [api])

const handleKeyDown = React.useCallback((event) => {
  if (event.key === "ArrowLeft") {
    event.preventDefault()
    scrollPrev()
  } else if (event.key === "ArrowRight") {
    event.preventDefault()
    scrollNext()
  }
}, [scrollPrev, scrollNext])

React.useEffect(() => {
  if (!api || !setApi) {
    return
  }

  setApi(api)
}, [api, setApi])

React.useEffect(() => {
  if (!api) {
    return
  }

  onSelect(api)
  api.on("reInit", onSelect)
  api.on("select", onSelect)

  return () => {
    api?.off("select", onSelect)
  };
}, [api, onSelect])

return (
  (<CarouselContext.Provider
    value={{
      carouselRef,
      api: api,
      opts,
      orientation:
        orientation || (opts?.axis === "y" ? "vertical" : "horizontal"),
```

```
        scrollPrev,
        scrollNext,
        canScrollPrev,
        canScrollNext,
      }}>
      <div
        ref={ref}
        onKeyDownCapture={handleKeyDown}
        className={cn("relative", className)}
        role="region"
        aria-roledescription="carousel"
        {...props}>
        {children}
      </div>
    </CarouselContext.Provider>)
  );
})
Carousel.displayName = "Carousel"

const CarouselContent = React.forwardRef(({ className, ...props }, ref) => {
  const { carouselRef, orientation } = useCarousel()

  return (
    (<div ref={carouselRef} className="overflow-hidden">
      <div
        ref={ref}
        className={cn(
          "flex",
          orientation === "horizontal" ? "-ml-4" : "-mt-4 flex-col",
          className
        )}
        {...props} />
    </div>)
  );
})
CarouselContent.displayName = "CarouselContent"

const CarouselItem = React.forwardRef(({ className, ...props }, ref) => {
  const { orientation } = useCarousel()

  return (
    (<div
      ref={ref}
      role="group"
      aria-roledescription="slide"
```

```
      className={cn(
        "min-w-0 shrink-0 grow-0 basis-full",
        orientation === "horizontal" ? "pl-4" : "pt-4",
        className
      )}
      {...props} />)
  );
})
CarouselItem.displayName = "CarouselItem"

const CarouselPrevious = React.forwardRef(({ className, variant = "outline",
size = "icon", ...props }, ref) => {
  const { orientation, scrollPrev, canScrollPrev } = useCarousel()

  return (
    (<Button
      ref={ref}
      variant={variant}
      size={size}
      className={cn("absolute  h-8 w-8 rounded-full", orientation ===
"horizontal"
        ? "-left-12 top-1/2 -translate-y-1/2"
        : "-top-12 left-1/2 -translate-x-1/2 rotate-90", className)}
      disabled={!canScrollPrev}
      onClick={scrollPrev}
      {...props}>
      <ArrowLeft className="h-4 w-4" />
      <span className="sr-only">Previous slide</span>
    </Button>)
  );
})
CarouselPrevious.displayName = "CarouselPrevious"

const CarouselNext = React.forwardRef(({ className, variant = "outline", size
= "icon", ...props }, ref) => {
  const { orientation, scrollNext, canScrollNext } = useCarousel()

  return (
    (<Button
      ref={ref}
      variant={variant}
      size={size}
      className={cn("absolute h-8 w-8 rounded-full", orientation ===
"horizontal"
        ? "-right-12 top-1/2 -translate-y-1/2"
```

```
        : "-bottom-12 left-1/2 -translate-x-1/2 rotate-90", className)}
      disabled={!canScrollNext}
      onClick={scrollNext}
      {...props}>
      <ArrowRight className="h-4 w-4" />
      <span className="sr-only">Next slide</span>
    </Button>)
  );
})
CarouselNext.displayName = "CarouselNext"

export { Carousel, CarouselContent, CarouselItem, CarouselPrevious,
CarouselNext };
```

chart.jsx

```
"use client";
import * as React from "react"
import * as RechartsPrimitive from "recharts"

import { cn } from "@/lib/utils"

// Format: { THEME_NAME: CSS_SELECTOR }
const THEMES = {
  light: "",
  dark: ".dark"
}

const ChartContext = React.createContext(null)

function useChart() {
  const context = React.useContext(ChartContext)

  if (!context) {
    throw new Error("useChart must be used within a <ChartContainer />")
  }

  return context
}

const ChartContainer = React.forwardRef(({ id, className, children, config,
...props }, ref) => {
  const uniqueId = React.useId()
  const chartId = `chart-${id || uniqueId.replace(/:/g, "")}`
```

```
  return (
    (<ChartContext.Provider value={{ config }}>
      <div
        data-chart={chartId}
        ref={ref}
        className={cn(
          "flex aspect-video justify-center text-xs
[&_.recharts-cartesian-axis-tick_text]:fill-muted-foreground
[&_.recharts-cartesian-grid_line[stroke='#ccc']]:stroke-border/50
[&_.recharts-curve.recharts-tooltip-cursor]:stroke-border
[&_.recharts-dot[stroke='#fff']]:stroke-transparent
[&_.recharts-layer]:outline-none
[&_.recharts-polar-grid_[stroke='#ccc']]:stroke-border
[&_.recharts-radial-bar-background-sector]:fill-muted
[&_.recharts-rectangle.recharts-tooltip-cursor]:fill-muted
[&_.recharts-reference-line_[stroke='#ccc']]:stroke-border
[&_.recharts-sector[stroke='#fff']]:stroke-transparent
[&_.recharts-sector]:outline-none [&_.recharts-surface]:outline-none",
          className
        )}
        {...props}>
        <ChartStyle id={chartId} config={config} />
        <RechartsPrimitive.ResponsiveContainer>
          {children}
        </RechartsPrimitive.ResponsiveContainer>
      </div>
    </ChartContext.Provider>)
  );
})
ChartContainer.displayName = "Chart"

const ChartStyle = ({
  id,
  config
}) => {
  const colorConfig = Object.entries(config).filter(([, config]) =>
config.theme || config.color)

  if (!colorConfig.length) {
    return null
  }

  return (
    (<style
      dangerouslySetInnerHTML={{
```

```
        __html: Object.entries(THEMES)
          .map(([theme, prefix]) => `
${prefix} [data-chart=${id}] {
${colorConfig
.map(([key, itemConfig]) => {
const color =
  itemConfig.theme?.[theme] ||
  itemConfig.color
return color ? `  --color-${key}: ${color};` : null
})
.join("\n")}
}
`)
          .join("\n"),
      }} />)
  );
}

const ChartTooltip = RechartsPrimitive.Tooltip

const ChartTooltipContent = React.forwardRef((
  {
    active,
    payload,
    className,
    indicator = "dot",
    hideLabel = false,
    hideIndicator = false,
    label,
    labelFormatter,
    labelClassName,
    formatter,
    color,
    nameKey,
    labelKey,
  },
  ref
) => {
  const { config } = useChart()

  const tooltipLabel = React.useMemo(() => {
    if (hideLabel || !payload?.length) {
      return null
    }
```

```
    const [item] = payload
    const key = `${labelKey || item.dataKey || item.name || "value"}`
    const itemConfig = getPayloadConfigFromPayload(config, item, key)
    const value =
      !labelKey && typeof label === "string"
        ? config[label]?.label || label
        : itemConfig?.label

    if (labelFormatter) {
      return (
        (<div className={cn("font-medium", labelClassName)}>
          {labelFormatter(value, payload)}
        </div>)
      );
    }

    if (!value) {
      return null
    }

    return <div className={cn("font-medium", labelClassName)}>{value}</div>;
  }, [
    label,
    labelFormatter,
    payload,
    hideLabel,
    labelClassName,
    config,
    labelKey,
  ])

  if (!active || !payload?.length) {
    return null
  }

  const nestLabel = payload.length === 1 && indicator !== "dot"

  return (
    (<div
      ref={ref}
      className={cn(
        "grid min-w-[8rem] items-start gap-1.5 rounded-lg border
border-border/50 bg-background px-2.5 py-1.5 text-xs shadow-xl",
        className
      )}>
```

```
      {!nestLabel ? tooltipLabel : null}
      <div className="grid gap-1.5">
        {payload.map((item, index) => {
          const key = `${nameKey || item.name || item.dataKey || "value"}`
          const itemConfig = getPayloadConfigFromPayload(config, item, key)
          const indicatorColor = color || item.payload.fill || item.color

          return (
            (<div
              key={item.dataKey}
              className={cn(
                "flex w-full flex-wrap items-stretch gap-2 [&>svg]:h-2.5
[&>svg]:w-2.5 [&>svg]:text-muted-foreground",
                indicator === "dot" && "items-center"
              )}>
              {formatter && item?.value !== undefined && item.name ? (
                formatter(item.value, item.name, item, index, item.payload)
              ) : (
                <>
                  {itemConfig?.icon ? (
                    <itemConfig.icon />
                  ) : (
                    !hideIndicator && (
                      <div
                        className={cn("shrink-0 rounded-[2px]
border-[--color-border] bg-[--color-bg]", {
                          "h-2.5 w-2.5": indicator === "dot",
                          "w-1": indicator === "line",
                          "w-0 border-[1.5px] border-dashed bg-transparent":
                            indicator === "dashed",
                          "my-0.5": nestLabel && indicator === "dashed",
                        })}
                        style={
                          {
                            "--color-bg": indicatorColor,
                            "--color-border": indicatorColor
                          }
                        } />
                    )
                  )}
                  <div
                    className={cn(
                      "flex flex-1 justify-between leading-none",
                      nestLabel ? "items-end" : "items-center"
                    )}>
```

```
                    <div className="grid gap-1.5">
                      {nestLabel ? tooltipLabel : null}
                      <span className="text-muted-foreground">
                        {itemConfig?.label || item.name}
                      </span>
                    </div>
                    {item.value && (
                      <span className="font-mono font-medium tabular-nums
text-foreground">
                        {item.value.toLocaleString()}
                      </span>
                    )}
                  </div>
                </>
              )}
            </div>)
          );
        })}
      </div>
    </div>)
  );
})
ChartTooltipContent.displayName = "ChartTooltip"

const ChartLegend = RechartsPrimitive.Legend

const ChartLegendContent = React.forwardRef((
  { className, hideIcon = false, payload, verticalAlign = "bottom", nameKey
},
  ref
) => {
  const { config } = useChart()

  if (!payload?.length) {
    return null
  }

  return (
    (<div
      ref={ref}
      className={cn(
        "flex items-center justify-center gap-4",
        verticalAlign === "top" ? "pb-3" : "pt-3",
        className
      )}>
```

```
      {payload.map((item) => {
        const key = `${nameKey || item.dataKey || "value"}`
        const itemConfig = getPayloadConfigFromPayload(config, item, key)

        return (
          (<div
            key={item.value}
            className={cn(
              "flex items-center gap-1.5 [&>svg]:h-3 [&>svg]:w-3
[&>svg]:text-muted-foreground"
            )}>
            {itemConfig?.icon && !hideIcon ? (
              <itemConfig.icon />
            ) : (
              <div
                className="h-2 w-2 shrink-0 rounded-[2px]"
                style={{
                  backgroundColor: item.color,
                }} />
            )}
            {itemConfig?.label}
          </div>)
        );
      })}
    </div>)
  );
})
ChartLegendContent.displayName = "ChartLegend"

// Helper to extract item config from a payload.
function getPayloadConfigFromPayload(
  config,
  payload,
  key
) {
  if (typeof payload !== "object" || payload === null) {
    return undefined
  }

  const payloadPayload =
    "payload" in payload &&
    typeof payload.payload === "object" &&
    payload.payload !== null
      ? payload.payload
      : undefined
```

```
  let configLabelKey = key

  if (
    key in payload &&
    typeof payload[key] === "string"
  ) {
    configLabelKey = payload[key]
  } else if (
    payloadPayload &&
    key in payloadPayload &&
    typeof payloadPayload[key] === "string"
  ) {
    configLabelKey = payloadPayload[key]
  }

  return configLabelKey in config
    ? config[configLabelKey]
    : config[key];
}

export {
  ChartContainer,
  ChartTooltip,
  ChartTooltipContent,
  ChartLegend,
  ChartLegendContent,
  ChartStyle,
}
```

checkbox.jsx

```
import * as React from "react"
import * as CheckboxPrimitive from "@radix-ui/react-checkbox"
import { Check } from "lucide-react"

import { cn } from "@/lib/utils"

const Checkbox = React.forwardRef(({ className, ...props }, ref) => (
  <CheckboxPrimitive.Root
    ref={ref}
    className={cn(
      "peer h-4 w-4 shrink-0 rounded-sm border border-primary shadow
focus-visible:outline-none focus-visible:ring-1 focus-visible:ring-ring
disabled:cursor-not-allowed disabled:opacity-50
```

```
data-[state=checked]:bg-primary
data-[state=checked]:text-primary-foreground",
      className
    )}
    {...props}>
    <CheckboxPrimitive.Indicator className={cn("flex items-center
justify-center text-current")}>
      <Check className="h-4 w-4" />
    </CheckboxPrimitive.Indicator>
  </CheckboxPrimitive.Root>
))
Checkbox.displayName = CheckboxPrimitive.Root.displayName

export { Checkbox }
```

collapsible.jsx

```
"use client"

import * as CollapsiblePrimitive from "@radix-ui/react-collapsible"

const Collapsible = CollapsiblePrimitive.Root

const CollapsibleTrigger = CollapsiblePrimitive.CollapsibleTrigger

const CollapsibleContent = CollapsiblePrimitive.CollapsibleContent

export { Collapsible, CollapsibleTrigger, CollapsibleContent }
```

command.jsx

```
import * as React from "react"
import { Command as CommandPrimitive } from "cmdk"
import { Search } from "lucide-react"

import { cn } from "@/lib/utils"
import { Dialog, DialogContent } from "@/components/ui/dialog"

const Command = React.forwardRef(({ className, ...props }, ref) => (
  <CommandPrimitive
    ref={ref}
    className={cn(
      "flex h-full w-full flex-col overflow-hidden rounded-md bg-popover
text-popover-foreground",
      className
    )}
```

```
    {...props} />
))
Command.displayName = CommandPrimitive.displayName

const CommandDialog = ({
  children,
  ...props
}) => {
  return (
    (<Dialog {...props}>
      <DialogContent className="overflow-hidden p-0">
        <Command
          className="[&_[cmdk-group-heading]]:px-2
[&_[cmdk-group-heading]]:font-medium
[&_[cmdk-group-heading]]:text-muted-foreground
[&_[cmdk-group]:not([hidden])_~[cmdk-group]]:pt-0 [&_[cmdk-group]]:px-2
[&_[cmdk-input-wrapper]_svg]:h-5 [&_[cmdk-input-wrapper]_svg]:w-5
[&_[cmdk-input]]:h-12 [&_[cmdk-item]]:px-2 [&_[cmdk-item]]:py-3
[&_[cmdk-item]_svg]:h-5 [&_[cmdk-item]_svg]:w-5">
          {children}
        </Command>
      </DialogContent>
    </Dialog>)
  );
}

const CommandInput = React.forwardRef(({ className, ...props }, ref) => (
  <div className="flex items-center border-b px-3" cmdk-input-wrapper="">
    <Search className="mr-2 h-4 w-4 shrink-0 opacity-50" />
    <CommandPrimitive.Input
      ref={ref}
      className={cn(
        "flex h-10 w-full rounded-md bg-transparent py-3 text-sm outline-none
placeholder:text-muted-foreground disabled:cursor-not-allowed
disabled:opacity-50",
        className
      )}
      {...props} />
  </div>
))

CommandInput.displayName = CommandPrimitive.Input.displayName

const CommandList = React.forwardRef(({ className, ...props }, ref) => (
  <CommandPrimitive.List
```

```
    ref={ref}
    className={cn("max-h-[300px] overflow-y-auto overflow-x-hidden",
classNameName)}
    {...props} />
))

CommandList.displayName = CommandPrimitive.List.displayName

const CommandEmpty = React.forwardRef((props, ref) => (
  <CommandPrimitive.Empty ref={ref} className="py-6 text-center text-sm"
{...props} />
))

CommandEmpty.displayName = CommandPrimitive.Empty.displayName

const CommandGroup = React.forwardRef(({ className, ...props }, ref) => (
  <CommandPrimitive.Group
    ref={ref}
    className={cn(
      "overflow-hidden p-1 text-foreground [&_[cmdk-group-heading]]:px-2
[&_[cmdk-group-heading]]:py-1.5 [&_[cmdk-group-heading]]:text-xs
[&_[cmdk-group-heading]]:font-medium
[&_[cmdk-group-heading]]:text-muted-foreground",
      className
    )}
    {...props} />
))

CommandGroup.displayName = CommandPrimitive.Group.displayName

const CommandSeparator = React.forwardRef(({ className, ...props }, ref) => (
  <CommandPrimitive.Separator ref={ref} className={cn("-mx-1 h-px bg-border",
className)} {...props} />
))
CommandSeparator.displayName = CommandPrimitive.Separator.displayName

const CommandItem = React.forwardRef(({ className, ...props }, ref) => (
  <CommandPrimitive.Item
    ref={ref}
    className={cn(
      "relative flex cursor-default gap-2 select-none items-center rounded-sm
px-2 py-1.5 text-sm outline-none data-[disabled=true]:pointer-events-none
data-[selected=true]:bg-accent data-[selected=true]:text-accent-foreground
data-[disabled=true]:opacity-50 [&_svg]:pointer-events-none [&_svg]:size-4
[&_svg]:shrink-0",
```

```
      className
    )}
    {...props} />
))

CommandItem.displayName = CommandPrimitive.Item.displayName

const CommandShortcut = ({
  className,
  ...props
}) => {
  return (
    (<span
      className={cn("ml-auto text-xs tracking-widest text-muted-foreground",
className)}
      {...props} />)
  );
}
CommandShortcut.displayName = "CommandShortcut"

export {
  Command,
  CommandDialog,
  CommandInput,
  CommandList,
  CommandEmpty,
  CommandGroup,
  CommandItem,
  CommandShortcut,
  CommandSeparator,
}
```

context-menu.jsx

```
import * as React from "react"
import * as ContextMenuPrimitive from "@radix-ui/react-context-menu"
import { Check, ChevronRight, Circle } from "lucide-react"

import { cn } from "@/lib/utils"

const ContextMenu = ContextMenuPrimitive.Root

const ContextMenuTrigger = ContextMenuPrimitive.Trigger

const ContextMenuGroup = ContextMenuPrimitive.Group
```

```
const ContextMenuPortal = ContextMenuPrimitive.Portal

const ContextMenuSub = ContextMenuPrimitive.Sub

const ContextMenuRadioGroup = ContextMenuPrimitive.RadioGroup

const ContextMenuSubTrigger = React.forwardRef(({ className, inset, children,
...props }, ref) => (
  <ContextMenuPrimitive.SubTrigger
    ref={ref}
    className={cn(
      "flex cursor-default select-none items-center rounded-sm px-2 py-1.5
text-sm outline-none focus:bg-accent focus:text-accent-foreground
data-[state=open]:bg-accent data-[state=open]:text-accent-foreground",
      inset && "pl-8",
      className
    )}
    {...props}>
    {children}
    <ChevronRight className="ml-auto h-4 w-4" />
  </ContextMenuPrimitive.SubTrigger>
))
ContextMenuSubTrigger.displayName =
ContextMenuPrimitive.SubTrigger.displayName

const ContextMenuSubContent = React.forwardRef(({ className, ...props }, ref)
=> (
  <ContextMenuPrimitive.SubContent
    ref={ref}
    className={cn(
      "z-50 min-w-[8rem] overflow-hidden rounded-md border bg-popover p-1
text-popover-foreground shadow-lg data-[state=open]:animate-in
data-[state=closed]:animate-out data-[state=closed]:fade-out-0
data-[state=open]:fade-in-0 data-[state=closed]:zoom-out-95
data-[state=open]:zoom-in-95 data-[side=bottom]:slide-in-from-top-2
data-[side=left]:slide-in-from-right-2 data-[side=right]:slide-in-from-left-2
data-[side=top]:slide-in-from-bottom-2",
      className
    )}
    {...props} />
))
ContextMenuSubContent.displayName =
ContextMenuPrimitive.SubContent.displayName
```

```
const ContextMenuContent = React.forwardRef(({ className, ...props }, ref) =>
(
  <ContextMenuPrimitive.Portal>
    <ContextMenuPrimitive.Content
      ref={ref}
      className={cn(
        "z-50 min-w-[8rem] overflow-hidden rounded-md border bg-popover p-1
text-popover-foreground shadow-md data-[state=open]:animate-in
data-[state=closed]:animate-out data-[state=closed]:fade-out-0
data-[state=open]:fade-in-0 data-[state=closed]:zoom-out-95
data-[state=open]:zoom-in-95 data-[side=bottom]:slide-in-from-top-2
data-[side=left]:slide-in-from-right-2 data-[side=right]:slide-in-from-left-2
data-[side=top]:slide-in-from-bottom-2",
        className
      )}
      {...props} />
  </ContextMenuPrimitive.Portal>
))
ContextMenuContent.displayName = ContextMenuPrimitive.Content.displayName

const ContextMenuItem = React.forwardRef(({ className, inset, ...props },
ref) => (
  <ContextMenuPrimitive.Item
    ref={ref}
    className={cn(
      "relative flex cursor-default select-none items-center rounded-sm px-2
py-1.5 text-sm outline-none focus:bg-accent focus:text-accent-foreground
data-[disabled]:pointer-events-none data-[disabled]:opacity-50",
      inset && "pl-8",
      className
    )}
    {...props} />
))
ContextMenuItem.displayName = ContextMenuPrimitive.Item.displayName

const ContextMenuCheckboxItem = React.forwardRef(({ className, children,
checked, ...props }, ref) => (
  <ContextMenuPrimitive.CheckboxItem
    ref={ref}
    className={cn(
      "relative flex cursor-default select-none items-center rounded-sm
py-1.5 pl-8 pr-2 text-sm outline-none focus:bg-accent
focus:text-accent-foreground data-[disabled]:pointer-events-none
data-[disabled]:opacity-50",
      className
```

```
    )}
    checked={checked}
    {...props}>
    <span className="absolute left-2 flex h-3.5 w-3.5 items-center
justify-center">
      <ContextMenuPrimitive.ItemIndicator>
        <Check className="h-4 w-4" />
      </ContextMenuPrimitive.ItemIndicator>
    </span>
    {children}
  </ContextMenuPrimitive.CheckboxItem>
))
ContextMenuCheckboxItem.displayName =
  ContextMenuPrimitive.CheckboxItem.displayName

const ContextMenuRadioItem = React.forwardRef(({ className, children,
...props }, ref) => (
  <ContextMenuPrimitive.RadioItem
    ref={ref}
    className={cn(
      "relative flex cursor-default select-none items-center rounded-sm
py-1.5 pl-8 pr-2 text-sm outline-none focus:bg-accent
focus:text-accent-foreground data-[disabled]:pointer-events-none
data-[disabled]:opacity-50",
      className
    )}
    {...props}>
    <span className="absolute left-2 flex h-3.5 w-3.5 items-center
justify-center">
      <ContextMenuPrimitive.ItemIndicator>
        <Circle className="h-4 w-4 fill-current" />
      </ContextMenuPrimitive.ItemIndicator>
    </span>
    {children}
  </ContextMenuPrimitive.RadioItem>
))
ContextMenuRadioItem.displayName = ContextMenuPrimitive.RadioItem.displayName

const ContextMenuLabel = React.forwardRef(({ className, inset, ...props },
ref) => (
  <ContextMenuPrimitive.Label
    ref={ref}
    className={cn(
      "px-2 py-1.5 text-sm font-semibold text-foreground",
      inset && "pl-8",
```

```
      className
    )}
    {...props} />
))
ContextMenuLabel.displayName = ContextMenuPrimitive.Label.displayName

const ContextMenuSeparator = React.forwardRef(({ className, ...props }, ref)
=> (
  <ContextMenuPrimitive.Separator
    ref={ref}
    className={cn("-mx-1 my-1 h-px bg-border", className)}
    {...props} />
))
ContextMenuSeparator.displayName = ContextMenuPrimitive.Separator.displayName

const ContextMenuShortcut = ({
  className,
  ...props
}) => {
  return (
    (<span
      className={cn("ml-auto text-xs tracking-widest text-muted-foreground",
className)}
      {...props} />)
  );
}
ContextMenuShortcut.displayName = "ContextMenuShortcut"

export {
  ContextMenu,
  ContextMenuTrigger,
  ContextMenuContent,
  ContextMenuItem,
  ContextMenuCheckboxItem,
  ContextMenuRadioItem,
  ContextMenuLabel,
  ContextMenuSeparator,
  ContextMenuShortcut,
  ContextMenuGroup,
  ContextMenuPortal,
  ContextMenuSub,
  ContextMenuSubContent,
  ContextMenuSubTrigger,
  ContextMenuRadioGroup,
}
```

dialog.jsx

```
"use client"

import * as React from "react"
import * as DialogPrimitive from "@radix-ui/react-dialog"
import { X } from "lucide-react"

import { cn } from "@/lib/utils"

const Dialog = DialogPrimitive.Root

const DialogTrigger = DialogPrimitive.Trigger

const DialogPortal = DialogPrimitive.Portal

const DialogClose = DialogPrimitive.Close

const DialogOverlay = React.forwardRef(({ className, ...props }, ref) => (
  <DialogPrimitive.Overlay
    ref={ref}
    className={cn(
      "fixed inset-0 z-50 bg-black/80  data-[state=open]:animate-in
data-[state=closed]:animate-out data-[state=closed]:fade-out-0
data-[state=open]:fade-in-0",
      className
    )}
    {...props} />
))
DialogOverlay.displayName = DialogPrimitive.Overlay.displayName

const DialogContent = React.forwardRef(({ className, children, ...props },
ref) => (
  <DialogPortal>
    <DialogOverlay />
    <DialogPrimitive.Content
      ref={ref}
      className={cn(
        "fixed left-[50%] top-[50%] z-50 grid w-full max-w-lg
translate-x-[-50%] translate-y-[-50%] gap-4 border bg-background p-6
shadow-lg duration-200 data-[state=open]:animate-in
data-[state=closed]:animate-out data-[state=closed]:fade-out-0
data-[state=open]:fade-in-0 data-[state=closed]:zoom-out-95
data-[state=open]:zoom-in-95 data-[state=closed]:slide-out-to-left-1/2
data-[state=closed]:slide-out-to-top-[48%]
```

```
data-[state=open]:slide-in-from-left-1/2
data-[state=open]:slide-in-from-top-[48%] sm:rounded-lg",
        className
      )}
      {...props}>
      {children}
      <DialogPrimitive.Close
        className="absolute right-4 top-4 rounded-sm opacity-70
ring-offset-background transition-opacity hover:opacity-100
focus:outline-none focus:ring-2 focus:ring-ring focus:ring-offset-2
disabled:pointer-events-none data-[state=open]:bg-accent
data-[state=open]:text-muted-foreground">
        <X className="h-4 w-4" />
        <span className="sr-only">Close</span>
      </DialogPrimitive.Close>
    </DialogPrimitive.Content>
  </DialogPortal>
))
DialogContent.displayName = DialogPrimitive.Content.displayName

const DialogHeader = ({
  className,
  ...props
}) => (
  <div
    className={cn("flex flex-col space-y-1.5 text-center sm:text-left",
className)}
    {...props} />
)
DialogHeader.displayName = "DialogHeader"

const DialogFooter = ({
  className,
  ...props
}) => (
  <div
    className={cn("flex flex-col-reverse sm:flex-row sm:justify-end
sm:space-x-2", className)}
    {...props} />
)
DialogFooter.displayName = "DialogFooter"

const DialogTitle = React.forwardRef(({ className, ...props }, ref) => (
  <DialogPrimitive.Title
    ref={ref}
```

```
    className={cn("text-lg font-semibold leading-none tracking-tight",
className)}
    {...props} />
))
DialogTitle.displayName = DialogPrimitive.Title.displayName

const DialogDescription = React.forwardRef(({ className, ...props }, ref) =>
(
  <DialogPrimitive.Description
    ref={ref}
    className={cn("text-sm text-muted-foreground", className)}
    {...props} />
))
DialogDescription.displayName = DialogPrimitive.Description.displayName

export {
  Dialog,
  DialogPortal,
  DialogOverlay,
  DialogTrigger,
  DialogClose,
  DialogContent,
  DialogHeader,
  DialogFooter,
  DialogTitle,
  DialogDescription,
}
```

drawer.jsx

```
"use client"

import * as React from "react"
import { Drawer as DrawerPrimitive } from "vaul"

import { cn } from "@/lib/utils"

const Drawer = ({
  shouldScaleBackground = true,
  ...props
}) => (
  <DrawerPrimitive.Root shouldScaleBackground={shouldScaleBackground}
{...props} />
)
Drawer.displayName = "Drawer"
```

```
const DrawerTrigger = DrawerPrimitive.Trigger

const DrawerPortal = DrawerPrimitive.Portal

const DrawerClose = DrawerPrimitive.Close

const DrawerOverlay = React.forwardRef(({ className, ...props }, ref) => (
  <DrawerPrimitive.Overlay
    ref={ref}
    className={cn("fixed inset-0 z-50 bg-black/80", className)}
    {...props} />
))
DrawerOverlay.displayName = DrawerPrimitive.Overlay.displayName

const DrawerContent = React.forwardRef(({ className, children, ...props },
ref) => (
  <DrawerPortal>
    <DrawerOverlay />
    <DrawerPrimitive.Content
      ref={ref}
      className={cn(
        "fixed inset-x-0 bottom-0 z-50 mt-24 flex h-auto flex-col
rounded-t-[10px] border bg-background",
        className
      )}
      {...props}>
      <div className="mx-auto mt-4 h-2 w-[100px] rounded-full bg-muted" />
      {children}
    </DrawerPrimitive.Content>
  </DrawerPortal>
))
DrawerContent.displayName = "DrawerContent"

const DrawerHeader = ({
  className,
  ...props
}) => (
  <div
    className={cn("grid gap-1.5 p-4 text-center sm:text-left", className)}
    {...props} />
)
DrawerHeader.displayName = "DrawerHeader"

const DrawerFooter = ({
  className,
```

```
  ...props
}) => (
  <div className={cn("mt-auto flex flex-col gap-2 p-4", className)}
{...props} />
)
DrawerFooter.displayName = "DrawerFooter"

const DrawerTitle = React.forwardRef(({ className, ...props }, ref) => (
  <DrawerPrimitive.Title
    ref={ref}
    className={cn("text-lg font-semibold leading-none tracking-tight",
className)}
    {...props} />
))
DrawerTitle.displayName = DrawerPrimitive.Title.displayName

const DrawerDescription = React.forwardRef(({ className, ...props }, ref) =>
(
  <DrawerPrimitive.Description
    ref={ref}
    className={cn("text-sm text-muted-foreground", className)}
    {...props} />
))
DrawerDescription.displayName = DrawerPrimitive.Description.displayName

export {
  Drawer,
  DrawerPortal,
  DrawerOverlay,
  DrawerTrigger,
  DrawerClose,
  DrawerContent,
  DrawerHeader,
  DrawerFooter,
  DrawerTitle,
  DrawerDescription,
}
```

dropdown-menu.jsx

```
import * as React from "react"
import * as DropdownMenuPrimitive from "@radix-ui/react-dropdown-menu"
import { Check, ChevronRight, Circle } from "lucide-react"
```

```
import { cn } from "@/lib/utils"

const DropdownMenu = DropdownMenuPrimitive.Root

const DropdownMenuTrigger = DropdownMenuPrimitive.Trigger

const DropdownMenuGroup = DropdownMenuPrimitive.Group

const DropdownMenuPortal = DropdownMenuPrimitive.Portal

const DropdownMenuSub = DropdownMenuPrimitive.Sub

const DropdownMenuRadioGroup = DropdownMenuPrimitive.RadioGroup

const DropdownMenuSubTrigger = React.forwardRef(({ className, inset,
children, ...props }, ref) => (
  <DropdownMenuPrimitive.SubTrigger
    ref={ref}
    className={cn(
      "flex cursor-default gap-2 select-none items-center rounded-sm px-2
py-1.5 text-sm outline-none focus:bg-accent data-[state=open]:bg-accent
[&_svg]:pointer-events-none [&_svg]:size-4 [&_svg]:shrink-0",
      inset && "pl-8",
      className
    )}
    {...props}>
    {children}
    <ChevronRight className="ml-auto" />
  </DropdownMenuPrimitive.SubTrigger>
))
DropdownMenuSubTrigger.displayName =
  DropdownMenuPrimitive.SubTrigger.displayName

const DropdownMenuSubContent = React.forwardRef(({ className, ...props },
ref) => (
  <DropdownMenuPrimitive.SubContent
    ref={ref}
    className={cn(
      "z-50 min-w-[8rem] overflow-hidden rounded-md border bg-popover p-1
text-popover-foreground shadow-lg data-[state=open]:animate-in
data-[state=closed]:animate-out data-[state=closed]:fade-out-0
data-[state=open]:fade-in-0 data-[state=closed]:zoom-out-95
data-[state=open]:zoom-in-95 data-[side=bottom]:slide-in-from-top-2
data-[side=left]:slide-in-from-right-2 data-[side=right]:slide-in-from-left-2
data-[side=top]:slide-in-from-bottom-2",
```

```
      className
    )}
    {...props} />
))
DropdownMenuSubContent.displayName =
  DropdownMenuPrimitive.SubContent.displayName

const DropdownMenuContent = React.forwardRef(({ className, sideOffset = 4,
...props }, ref) => (
  <DropdownMenuPrimitive.Portal>
    <DropdownMenuPrimitive.Content
      ref={ref}
      sideOffset={sideOffset}
      className={cn(
        "z-50 min-w-[8rem] overflow-hidden rounded-md border bg-popover p-1
text-popover-foreground shadow-md",
        "data-[state=open]:animate-in data-[state=closed]:animate-out
data-[state=closed]:fade-out-0 data-[state=open]:fade-in-0
data-[state=closed]:zoom-out-95 data-[state=open]:zoom-in-95
data-[side=bottom]:slide-in-from-top-2 data-[side=left]:slide-in-from-right-2
data-[side=right]:slide-in-from-left-2
data-[side=top]:slide-in-from-bottom-2",
        className
      )}
      {...props} />
  </DropdownMenuPrimitive.Portal>
))
DropdownMenuContent.displayName = DropdownMenuPrimitive.Content.displayName

const DropdownMenuItem = React.forwardRef(({ className, inset, ...props },
ref) => (
  <DropdownMenuPrimitive.Item
    ref={ref}
    className={cn(
      "relative flex cursor-default select-none items-center gap-2 rounded-sm
px-2 py-1.5 text-sm outline-none transition-colors focus:bg-accent
focus:text-accent-foreground data-[disabled]:pointer-events-none
data-[disabled]:opacity-50 [&>svg]:size-4 [&>svg]:shrink-0",
      inset && "pl-8",
      className
    )}
    {...props} />
))
DropdownMenuItem.displayName = DropdownMenuPrimitive.Item.displayName
```

```
const DropdownMenuCheckboxItem = React.forwardRef(({ className, children,
checked, ...props }, ref) => (
  <DropdownMenuPrimitive.CheckboxItem
    ref={ref}
    className={cn(
      "relative flex cursor-default select-none items-center rounded-sm
py-1.5 pl-8 pr-2 text-sm outline-none transition-colors focus:bg-accent
focus:text-accent-foreground data-[disabled]:pointer-events-none
data-[disabled]:opacity-50",
      className
    )}
    checked={checked}
    {...props}>
    <span className="absolute left-2 flex h-3.5 w-3.5 items-center
justify-center">
      <DropdownMenuPrimitive.ItemIndicator>
        <Check className="h-4 w-4" />
      </DropdownMenuPrimitive.ItemIndicator>
    </span>
    {children}
  </DropdownMenuPrimitive.CheckboxItem>
))
DropdownMenuCheckboxItem.displayName =
  DropdownMenuPrimitive.CheckboxItem.displayName

const DropdownMenuRadioItem = React.forwardRef(({ className, children,
...props }, ref) => (
  <DropdownMenuPrimitive.RadioItem
    ref={ref}
    className={cn(
      "relative flex cursor-default select-none items-center rounded-sm
py-1.5 pl-8 pr-2 text-sm outline-none transition-colors focus:bg-accent
focus:text-accent-foreground data-[disabled]:pointer-events-none
data-[disabled]:opacity-50",
      className
    )}
    {...props}>
    <span className="absolute left-2 flex h-3.5 w-3.5 items-center
justify-center">
      <DropdownMenuPrimitive.ItemIndicator>
        <Circle className="h-2 w-2 fill-current" />
      </DropdownMenuPrimitive.ItemIndicator>
    </span>
    {children}
  </DropdownMenuPrimitive.RadioItem>
```

```
))
DropdownMenuRadioItem.displayName =
DropdownMenuPrimitive.RadioItem.displayName

const DropdownMenuLabel = React.forwardRef(({ className, inset, ...props },
ref) => (
  <DropdownMenuPrimitive.Label
    ref={ref}
    className={cn("px-2 py-1.5 text-sm font-semibold", inset && "pl-8",
className)}
    {...props} />
))
DropdownMenuLabel.displayName = DropdownMenuPrimitive.Label.displayName

const DropdownMenuSeparator = React.forwardRef(({ className, ...props }, ref)
=> (
  <DropdownMenuPrimitive.Separator
    ref={ref}
    className={cn("-mx-1 my-1 h-px bg-muted", className)}
    {...props} />
))
DropdownMenuSeparator.displayName =
DropdownMenuPrimitive.Separator.displayName

const DropdownMenuShortcut = ({
  className,
  ...props
}) => {
  return (
    (<span
      className={cn("ml-auto text-xs tracking-widest opacity-60", className)}
      {...props} />)
  );
}
DropdownMenuShortcut.displayName = "DropdownMenuShortcut"

export {
  DropdownMenu,
  DropdownMenuTrigger,
  DropdownMenuContent,
  DropdownMenuItem,
  DropdownMenuCheckboxItem,
  DropdownMenuRadioItem,
  DropdownMenuLabel,
  DropdownMenuSeparator,
```

```
  DropdownMenuShortcut,
  DropdownMenuGroup,
  DropdownMenuPortal,
  DropdownMenuSub,
  DropdownMenuSubContent,
  DropdownMenuSubTrigger,
  DropdownMenuRadioGroup,
}
```

form.jsx

```
"use client";
import * as React from "react"
import { Slot } from "@radix-ui/react-slot"
import { Controller, FormProvider, useFormContext } from "react-hook-form";

import { cn } from "@/lib/utils"
import { Label } from "@/components/ui/label"

const Form = FormProvider

const FormFieldContext = React.createContext({})

const FormField = (
  {
    ...props
  }
) => {
  return (
    (<FormFieldContext.Provider value={{ name: props.name }}>
      <Controller {...props} />
    </FormFieldContext.Provider>)
  );
}

const useFormField = () => {
  const fieldContext = React.useContext(FormFieldContext)
  const itemContext = React.useContext(FormItemContext)
  const { getFieldState, formState } = useFormContext()

  const fieldState = getFieldState(fieldContext.name, formState)

  if (!fieldContext) {
    throw new Error("useFormField should be used within <FormField>")
  }
```

```
  const { id } = itemContext

  return {
    id,
    name: fieldContext.name,
    formItemId: `${id}-form-item`,
    formDescriptionId: `${id}-form-item-description`,
    formMessageId: `${id}-form-item-message`,
    ...fieldState,
  }
}

const FormItemContext = React.createContext({})

const FormItem = React.forwardRef(({ className, ...props }, ref) => {
  const id = React.useId()

  return (
    (<FormItemContext.Provider value={{ id }}>
      <div ref={ref} className={cn("space-y-2", className)} {...props} />
    </FormItemContext.Provider>)
  );
})
FormItem.displayName = "FormItem"

const FormLabel = React.forwardRef(({ className, ...props }, ref) => {
  const { error, formItemId } = useFormField()

  return (
    (<Label
      ref={ref}
      className={cn(error && "text-destructive", className)}
      htmlFor={formItemId}
      {...props} />)
  );
})
FormLabel.displayName = "FormLabel"

const FormControl = React.forwardRef(({ ...props }, ref) => {
  const { error, formItemId, formDescriptionId, formMessageId } =
useFormField()

  return (
    (<Slot
      ref={ref}
```

```
      id={formItemId}
      aria-describedby={
        !error
          ? `${formDescriptionId}`
          : `${formDescriptionId} ${formMessageId}`
      }
      aria-invalid={!!error}
      {...props} />)
  );
})
FormControl.displayName = "FormControl"

const FormDescription = React.forwardRef(({ className, ...props }, ref) => {
  const { formDescriptionId } = useFormField()

  return (
    (<p
      ref={ref}
      id={formDescriptionId}
      className={cn("text-[0.8rem] text-muted-foreground", className)}
      {...props} />)
  );
})
FormDescription.displayName = "FormDescription"

const FormMessage = React.forwardRef(({ className, children, ...props }, ref)
=> {
  const { error, formMessageId } = useFormField()
  const body = error ? String(error?.message) : children

  if (!body) {
    return null
  }

  return (
    (<p
      ref={ref}
      id={formMessageId}
      className={cn("text-[0.8rem] font-medium text-destructive", className)}
      {...props}>
      {body}
    </p>)
  );
})
FormMessage.displayName = "FormMessage"
```

```
export {
  useFormField,
  Form,
  FormItem,
  FormLabel,
  FormControl,
  FormDescription,
  FormMessage,
  FormField,
}
```

hover-card.jsx

```
"use client"

import * as React from "react"
import * as HoverCardPrimitive from "@radix-ui/react-hover-card"

import { cn } from "@/lib/utils"

const HoverCard = HoverCardPrimitive.Root

const HoverCardTrigger = HoverCardPrimitive.Trigger

const HoverCardContent = React.forwardRef(({ className, align = "center",
sideOffset = 4, ...props }, ref) => (
  <HoverCardPrimitive.Content
    ref={ref}
    align={align}
    sideOffset={sideOffset}
    className={cn(
      "z-50 w-64 rounded-md border bg-popover p-4 text-popover-foreground
shadow-md outline-none data-[state=open]:animate-in
data-[state=closed]:animate-out data-[state=closed]:fade-out-0
data-[state=open]:fade-in-0 data-[state=closed]:zoom-out-95
data-[state=open]:zoom-in-95 data-[side=bottom]:slide-in-from-top-2
data-[side=left]:slide-in-from-right-2 data-[side=right]:slide-in-from-left-2
data-[side=top]:slide-in-from-bottom-2",
      className
    )}
    {...props} />
))
HoverCardContent.displayName = HoverCardPrimitive.Content.displayName

export { HoverCard, HoverCardTrigger, HoverCardContent }
```

image-helpers.js

```
const WIX_MEDIA_HOSTS = {
  "media.base44.com": "/images/public/",
  "static.wixstatic.com": "/media/",
}

export const DEFAULT_TRANSFORM_WIDTH = 1024
export const IMAGE_LOAD_MODE = {
  OPTIMIZED: "optimized",
  ORIGINAL: "original",
  FALLBACK: "fallback",
}

const DEVICE_PIXEL_RATIOS = [1, 2, 3]
const MAX_DIMENSION = 6000

export function splitImageProps(props) {
  const wrapperProps = {}
  const imageProps = {}
  for (const [key, value] of Object.entries(props)) {
    if (key.startsWith("data-")) wrapperProps[key] = value
    else imageProps[key] = value
  }
  return { wrapperProps, imageProps }
}

export function getImagePreviewClassName(className, currentClassName,
baselineClassName) {
  const sourceClasses = new Set((className || "").split(/\s+/))
  const baselineClasses = new Set(baselineClassName.split(/\s+/))
  return currentClassName.split(/\s+/).filter((token) =>
    !["inline-block", "relative"].includes(token) ||
!baselineClasses.has(token) || sourceClasses.has(token)
  ).join(" ")
}

/** Returns transform metadata only for canonical public Wix image URLs. */
export function parseWixMediaUrl(src) {
  try {
    const url = new URL(src)
    if (
      url.protocol !== "https:" ||
      url.username ||
      url.password ||
```

```
      (url.port && url.port !== "443")
    ) {
      return null
    }

    const pathPrefix = WIX_MEDIA_HOSTS[url.hostname]
    if (!pathPrefix) return null

    const transformed =
url.pathname.match(/^(.*)\/v1\/(?:fill|fit)\/[^/]+\/[^/]+$/i)
    const basePath = transformed ? transformed[1] : url.pathname
    const filename = basePath.split("/").pop()
    if (
      !basePath.startsWith(pathPrefix) ||
      !filename ||
      !/\.[a-z0-9]+$/i.test(filename) ||
      /\.svg$/i.test(filename)
    ) {
      return null
    }

    return { baseUrl: `${url.origin}${basePath}`, filename }
  } catch {
    return null
  }
}

const clampDim = (n) => Math.min(Math.max(Math.round(n), 1), MAX_DIMENSION)
const clamp01 = (n) => Math.min(1, Math.max(0, n))

export function buildTransformUrl(
  { baseUrl, filename },
  { width, height, crop, focalPoint, quality }
) {
  const params = [`w_${clampDim(width)}`, `h_${clampDim(height || width)}`]
  if (crop) {
    params.push(
      focalPoint
        ?
`fp_${clamp01(focalPoint.x).toFixed(2)}_${clamp01(focalPoint.y).toFixed(2)}`
        : "al_c"
    )
  }
  // Preserve GIF delivery; negotiate AVIF/WebP for other images via Accept.
  const encoding = /\.gif$/i.test(filename) ? "enc_webp" : "enc_auto"
```

```
  params.push(`q_${quality}`, "usm_0.66_1.00_0.01", encoding, "quality_auto")
  return `${baseUrl}/v1/${crop ? "fill" :
"fit"}/${params.join(",")}/${filename}`
}

export function buildSrcSet(parsed, options) {
  return DEVICE_PIXEL_RATIOS.map(
    (dpr) =>
      `${buildTransformUrl(parsed, {
        ...options,
        width: options.width * dpr,
        height: options.height ? options.height * dpr : undefined,
      })} ${dpr}x`
  ).join(", ")
}

export function getOriginalImageUrl(src, parsed) {
  return parsed?.baseUrl || src
}

export function nextImageLoadMode(mode) {
  return mode === IMAGE_LOAD_MODE.OPTIMIZED
    ? IMAGE_LOAD_MODE.ORIGINAL
    : IMAGE_LOAD_MODE.FALLBACK
}
```

image.jsx

```
import * as React from "react"
import { ResponsiveImage } from "./responsive-image"
import {
  getOriginalImageUrl,
  IMAGE_LOAD_MODE,
  nextImageLoadMode,
  parseWixMediaUrl,
} from "./image-helpers"

const FALLBACK_IMAGE_URL =

"https://static.wixstatic.com/media/12d367_4f26ccd17f8f4e3a8958306ea08c2332~m
v2.png"

/**
 * Image with built-in Wix Media Platform support: canonical public images on
 * media.base44.com and static.wixstatic.com/media are resized to the
rendered
```

```
 * container per device pixel ratio and served as AVIF/WebP when supported;
`fittingType="fill"`
 * crops server-side, optionally anchored at a focal point. Other URLs render
 * as a plain <img>. Failed transforms retry the original URL; only a broken
 * original swaps to the generic fallback image.
 */
const Image = React.forwardRef(
  (
    {
      src: source,
      fittingType = "fill",
      originWidth,
      originHeight,
      focalPointX,
      focalPointY,
      quality = 90,
      onError,
      ...props
    },
    ref
  ) => {
    const [previewSource, setPreviewSource] = React.useState(null)
    const preview = previewSource?.source === source ? previewSource : null
    const src = preview ? preview.value : source
    const replaceSource = (value, className) => setPreviewSource({
      source, value, className, sourceClassName: props.className,
    })
    React.useEffect(() => setPreviewSource(null), [source])
    const parsedSource = src && src !== FALLBACK_IMAGE_URL ?
parseWixMediaUrl(src) : null
    const initialMode = parsedSource ? IMAGE_LOAD_MODE.OPTIMIZED :
IMAGE_LOAD_MODE.ORIGINAL
    const [loadState, setLoadState] = React.useState({ src, mode: initialMode
})
    const mode = loadState.src === src ? loadState.mode : initialMode

    React.useEffect(() => {
      setLoadState({ src, mode: initialMode })
    }, [src, initialMode])

    const handleError = (event) => {
      if (mode === IMAGE_LOAD_MODE.FALLBACK) return
      const nextMode = nextImageLoadMode(mode)
      setLoadState({ src, mode: nextMode })
      if (nextMode === IMAGE_LOAD_MODE.FALLBACK) onError?.(event)
```

```
    }

    const imageProps = {
      ...props,
      className: preview && preview.sourceClassName === props.className ?
preview.className : props.className,
      onError: handleError,
    }

    if (!src) {
      // Renders as a real <img> (not a <div>) — the visual editor's
      // click-to-edit toolbar keys its "Replace Image" action off the DOM
      // tag being `img`, so a placeholder div would be unrecoverable in the
      // editor. FALLBACK_IMAGE_URL doubles as the "no image chosen" graphic.
      return <img ref={ref} src={FALLBACK_IMAGE_URL} {...imageProps}
data-empty-image />
    }

    // A failed transform retries the underlying original as a plain image.
    // Only a failure of that original advances to the generic fallback.
    const parsed = mode === IMAGE_LOAD_MODE.OPTIMIZED ? parsedSource : null

    if (!parsed) {
      const isErrorMode = mode === IMAGE_LOAD_MODE.FALLBACK
      const imageSrc = isErrorMode ? FALLBACK_IMAGE_URL :
getOriginalImageUrl(src, parsedSource)
      return (
        <img ref={ref} src={imageSrc} {...imageProps}
data-error-image={isErrorMode || undefined} />
      )
    }

    const focalPoint =
      typeof focalPointX === "number" && typeof focalPointY === "number"
        ? { x: focalPointX, y: focalPointY }
        : undefined
    // Origin dimensions are optional — when known they stabilize layout via
    // the wrapper's aspect-ratio before the image loads.
    const aspectRatio =
      originWidth && originHeight ? `${originWidth} / ${originHeight}` :
undefined

    return (
      <ResponsiveImage
        ref={ref}
```

```
        src={src}
        parsed={parsed}
        onSourceChange={replaceSource}
        fittingType={fittingType}
        focalPoint={focalPoint}
        quality={quality}
        aspectRatio={aspectRatio}
        {...imageProps}
      />
    )
  }
)
Image.displayName = "Image"

export { Image }
```

input-otp.jsx

```
import * as React from "react"
import { OTPInput, OTPInputContext } from "input-otp"
import { Minus } from "lucide-react"

import { cn } from "@/lib/utils"

const InputOTP = React.forwardRef(({ className, containerClassName, ...props
}, ref) => (
  <OTPInput
    ref={ref}
    containerClassName={cn("flex items-center gap-2
has-[:disabled]:opacity-50", containerClassName)}
    className={cn("disabled:cursor-not-allowed", className)}
    {...props} />
))
InputOTP.displayName = "InputOTP"

const InputOTPGroup = React.forwardRef(({ className, ...props }, ref) => (
  <div ref={ref} className={cn("flex items-center", className)} {...props} />
))
InputOTPGroup.displayName = "InputOTPGroup"

const InputOTPSlot = React.forwardRef(({ index, className, ...props }, ref)
=> {
  const inputOTPContext = React.useContext(OTPInputContext)
  const { char, hasFakeCaret, isActive } = inputOTPContext.slots[index]

  return (
```

```
    (<div
      ref={ref}
      className={cn(
        "relative flex h-9 w-9 items-center justify-center border-y border-r
border-input text-sm shadow-sm transition-all first:rounded-l-md
first:border-l last:rounded-r-md",
        isActive && "z-10 ring-1 ring-ring",
        className
      )}
      {...props}>
      {char}
      {hasFakeCaret && (
        <div
          className="pointer-events-none absolute inset-0 flex items-center
justify-center">
          <div className="h-4 w-px animate-caret-blink bg-foreground
duration-1000" />
        </div>
      )}
    </div>)
  );
})
InputOTPSlot.displayName = "InputOTPSlot"

const InputOTPSeparator = React.forwardRef(({ ...props }, ref) => (
  <div ref={ref} role="separator" {...props}>
    <Minus />
  </div>
))
InputOTPSeparator.displayName = "InputOTPSeparator"

export { InputOTP, InputOTPGroup, InputOTPSlot, InputOTPSeparator }
```

input.jsx

```
import * as React from "react"

import { cn } from "@/lib/utils"

const Input = React.forwardRef(({ className, type, ...props }, ref) => {
  return (
    (<input
      type={type}
      className={cn(
        "flex h-9 w-full rounded-md border border-input bg-transparent px-3
py-1 text-base shadow-sm transition-colors file:border-0 file:bg-transparent
```

```
file:text-sm file:font-medium file:text-foreground
placeholder:text-muted-foreground focus-visible:outline-none
focus-visible:ring-1 focus-visible:ring-ring disabled:cursor-not-allowed
disabled:opacity-50 md:text-sm",
        className
      )}
      ref={ref}
      {...props} />)
  );
})
Input.displayName = "Input"

export { Input }
```

label.jsx

```
import * as React from "react"
import * as LabelPrimitive from "@radix-ui/react-label"
import { cva } from "class-variance-authority";

import { cn } from "@/lib/utils"

const labelVariants = cva(
  "text-sm font-medium leading-none peer-disabled:cursor-not-allowed
peer-disabled:opacity-70"
)

const Label = React.forwardRef(({ className, ...props }, ref) => (
  <LabelPrimitive.Root ref={ref} className={cn(labelVariants(), className)}
{...props} />
))
Label.displayName = LabelPrimitive.Root.displayName

export { Label }
```

menubar.jsx

```
"use client"

import * as React from "react"
import * as MenubarPrimitive from "@radix-ui/react-menubar"
import { Check, ChevronRight, Circle } from "lucide-react"

import { cn } from "@/lib/utils"

function MenubarMenu({
```

```
  ...props
}) {
  return <MenubarPrimitive.Menu {...props} />;
}

function MenubarGroup({
  ...props
}) {
  return <MenubarPrimitive.Group {...props} />;
}

function MenubarPortal({
  ...props
}) {
  return <MenubarPrimitive.Portal {...props} />;
}

function MenubarRadioGroup({
  ...props
}) {
  return <MenubarPrimitive.RadioGroup {...props} />;
}

function MenubarSub({
  ...props
}) {
  return <MenubarPrimitive.Sub data-slot="menubar-sub" {...props} />;
}

const Menubar = React.forwardRef(({ className, ...props }, ref) => (
  <MenubarPrimitive.Root
    ref={ref}
    className={cn(
      "flex h-9 items-center space-x-1 rounded-md border bg-background p-1
shadow-sm",
      className
    )}
    {...props} />
))
Menubar.displayName = MenubarPrimitive.Root.displayName

const MenubarTrigger = React.forwardRef(({ className, ...props }, ref) => (
  <MenubarPrimitive.Trigger
    ref={ref}
    className={cn(
```

```
      "flex cursor-default select-none items-center rounded-sm px-3 py-1
text-sm font-medium outline-none focus:bg-accent focus:text-accent-foreground
data-[state=open]:bg-accent data-[state=open]:text-accent-foreground",
      className
    )}
    {...props} />
))
MenubarTrigger.displayName = MenubarPrimitive.Trigger.displayName

const MenubarSubTrigger = React.forwardRef(({ className, inset, children,
...props }, ref) => (
  <MenubarPrimitive.SubTrigger
    ref={ref}
    className={cn(
      "flex cursor-default select-none items-center rounded-sm px-2 py-1.5
text-sm outline-none focus:bg-accent focus:text-accent-foreground
data-[state=open]:bg-accent data-[state=open]:text-accent-foreground",
      inset && "pl-8",
      className
    )}
    {...props}>
    {children}
    <ChevronRight className="ml-auto h-4 w-4" />
  </MenubarPrimitive.SubTrigger>
))
MenubarSubTrigger.displayName = MenubarPrimitive.SubTrigger.displayName

const MenubarSubContent = React.forwardRef(({ className, ...props }, ref) =>
(
  <MenubarPrimitive.SubContent
    ref={ref}
    className={cn(
      "z-50 min-w-[8rem] overflow-hidden rounded-md border bg-popover p-1
text-popover-foreground shadow-lg data-[state=open]:animate-in
data-[state=closed]:animate-out data-[state=closed]:fade-out-0
data-[state=open]:fade-in-0 data-[state=closed]:zoom-out-95
data-[state=open]:zoom-in-95 data-[side=bottom]:slide-in-from-top-2
data-[side=left]:slide-in-from-right-2 data-[side=right]:slide-in-from-left-2
data-[side=top]:slide-in-from-bottom-2",
      className
    )}
    {...props} />
))
MenubarSubContent.displayName = MenubarPrimitive.SubContent.displayName
```

```
const MenubarContent = React.forwardRef((
  { className, align = "start", alignOffset = -4, sideOffset = 8, ...props },
  ref
) => (
  <MenubarPrimitive.Portal>
    <MenubarPrimitive.Content
      ref={ref}
      align={align}
      alignOffset={alignOffset}
      sideOffset={sideOffset}
      className={cn(
        "z-50 min-w-[12rem] overflow-hidden rounded-md border bg-popover p-1
text-popover-foreground shadow-md data-[state=open]:animate-in
data-[state=closed]:fade-out-0 data-[state=open]:fade-in-0
data-[state=closed]:zoom-out-95 data-[state=open]:zoom-in-95
data-[side=bottom]:slide-in-from-top-2 data-[side=left]:slide-in-from-right-2
data-[side=right]:slide-in-from-left-2
data-[side=top]:slide-in-from-bottom-2",
        className
      )}
      {...props} />
  </MenubarPrimitive.Portal>
))
MenubarContent.displayName = MenubarPrimitive.Content.displayName

const MenubarItem = React.forwardRef(({ className, inset, ...props }, ref) =>
(
  <MenubarPrimitive.Item
    ref={ref}
    className={cn(
      "relative flex cursor-default select-none items-center rounded-sm px-2
py-1.5 text-sm outline-none focus:bg-accent focus:text-accent-foreground
data-[disabled]:pointer-events-none data-[disabled]:opacity-50",
      inset && "pl-8",
      className
    )}
    {...props} />
))
MenubarItem.displayName = MenubarPrimitive.Item.displayName

const MenubarCheckboxItem = React.forwardRef(({ className, children, checked,
...props }, ref) => (
  <MenubarPrimitive.CheckboxItem
    ref={ref}
    className={cn(
```

```
      "relative flex cursor-default select-none items-center rounded-sm
py-1.5 pl-8 pr-2 text-sm outline-none focus:bg-accent
focus:text-accent-foreground data-[disabled]:pointer-events-none
data-[disabled]:opacity-50",
      className
    )}
    checked={checked}
    {...props}>
    <span className="absolute left-2 flex h-3.5 w-3.5 items-center
justify-center">
      <MenubarPrimitive.ItemIndicator>
        <Check className="h-4 w-4" />
      </MenubarPrimitive.ItemIndicator>
    </span>
    {children}
  </MenubarPrimitive.CheckboxItem>
))
MenubarCheckboxItem.displayName = MenubarPrimitive.CheckboxItem.displayName

const MenubarRadioItem = React.forwardRef(({ className, children, ...props },
ref) => (
  <MenubarPrimitive.RadioItem
    ref={ref}
    className={cn(
      "relative flex cursor-default select-none items-center rounded-sm
py-1.5 pl-8 pr-2 text-sm outline-none focus:bg-accent
focus:text-accent-foreground data-[disabled]:pointer-events-none
data-[disabled]:opacity-50",
      className
    )}
    {...props}>
    <span className="absolute left-2 flex h-3.5 w-3.5 items-center
justify-center">
      <MenubarPrimitive.ItemIndicator>
        <Circle className="h-4 w-4 fill-current" />
      </MenubarPrimitive.ItemIndicator>
    </span>
    {children}
  </MenubarPrimitive.RadioItem>
))
MenubarRadioItem.displayName = MenubarPrimitive.RadioItem.displayName

const MenubarLabel = React.forwardRef(({ className, inset, ...props }, ref)
=> (
  <MenubarPrimitive.Label
```

```
    ref={ref}
    className={cn("px-2 py-1.5 text-sm font-semibold", inset && "pl-8",
className)}
    {...props} />
))
MenubarLabel.displayName = MenubarPrimitive.Label.displayName

const MenubarSeparator = React.forwardRef(({ className, ...props }, ref) => (
  <MenubarPrimitive.Separator
    ref={ref}
    className={cn("-mx-1 my-1 h-px bg-muted", className)}
    {...props} />
))
MenubarSeparator.displayName = MenubarPrimitive.Separator.displayName

const MenubarShortcut = ({
  className,
  ...props
}) => {
  return (
    (<span
      className={cn("ml-auto text-xs tracking-widest text-muted-foreground",
className)}
      {...props} />)
  );
}
MenubarShortcut.displayname = "MenubarShortcut"

export {
  Menubar,
  MenubarMenu,
  MenubarTrigger,
  MenubarContent,
  MenubarItem,
  MenubarSeparator,
  MenubarLabel,
  MenubarCheckboxItem,
  MenubarRadioGroup,
  MenubarRadioItem,
  MenubarPortal,
  MenubarSubContent,
  MenubarSubTrigger,
  MenubarGroup,
  MenubarSub,
  MenubarShortcut,
```

```
}
```

navigation-menu.jsx

```
import * as React from "react"
import * as NavigationMenuPrimitive from "@radix-ui/react-navigation-menu"
import { cva } from "class-variance-authority"
import { ChevronDown } from "lucide-react"

import { cn } from "@/lib/utils"

const NavigationMenu = React.forwardRef(({ className, children, ...props },
ref) => (
  <NavigationMenuPrimitive.Root
    ref={ref}
    className={cn(
      "relative z-10 flex max-w-max flex-1 items-center justify-center",
      className
    )}
    {...props}>
    {children}
    <NavigationMenuViewport />
  </NavigationMenuPrimitive.Root>
))
NavigationMenu.displayName = NavigationMenuPrimitive.Root.displayName

const NavigationMenuList = React.forwardRef(({ className, ...props }, ref) =>
(
  <NavigationMenuPrimitive.List
    ref={ref}
    className={cn(
      "group flex flex-1 list-none items-center justify-center space-x-1",
      className
    )}
    {...props} />
))
NavigationMenuList.displayName = NavigationMenuPrimitive.List.displayName

const NavigationMenuItem = NavigationMenuPrimitive.Item

const navigationMenuTriggerStyle = cva(
  "group inline-flex h-9 w-max items-center justify-center rounded-md
bg-background px-4 py-2 text-sm font-medium transition-colors hover:bg-accent
hover:text-accent-foreground focus:bg-accent focus:text-accent-foreground
```

```
focus:outline-none disabled:pointer-events-none disabled:opacity-50
data-[active]:bg-accent/50 data-[state=open]:bg-accent/50"
)

const NavigationMenuTrigger = React.forwardRef(({ className, children,
...props }, ref) => (
  <NavigationMenuPrimitive.Trigger
    ref={ref}
    className={cn(navigationMenuTriggerStyle(), "group", className)}
    {...props}>
    {children}{" "}
    <ChevronDown
      className="relative top-[1px] ml-1 h-3 w-3 transition duration-300
group-data-[state=open]:rotate-180"
      aria-hidden="true" />
  </NavigationMenuPrimitive.Trigger>
))
NavigationMenuTrigger.displayName =
NavigationMenuPrimitive.Trigger.displayName

const NavigationMenuContent = React.forwardRef(({ className, ...props }, ref)
=> (
  <NavigationMenuPrimitive.Content
    ref={ref}
    className={cn(
      "left-0 top-0 w-full data-[motion^=from-]:animate-in
data-[motion^=to-]:animate-out data-[motion^=from-]:fade-in
data-[motion^=to-]:fade-out data-[motion=from-end]:slide-in-from-right-52
data-[motion=from-start]:slide-in-from-left-52
data-[motion=to-end]:slide-out-to-right-52
data-[motion=to-start]:slide-out-to-left-52 md:absolute md:w-auto ",
      className
    )}
    {...props} />
))
NavigationMenuContent.displayName =
NavigationMenuPrimitive.Content.displayName

const NavigationMenuLink = NavigationMenuPrimitive.Link

const NavigationMenuViewport = React.forwardRef(({ className, ...props },
ref) => (
  <div className={cn("absolute left-0 top-full flex justify-center")}>
    <NavigationMenuPrimitive.Viewport
      className={cn(
```

```
        "origin-top-center relative mt-1.5
h-[var(--radix-navigation-menu-viewport-height)] w-full overflow-hidden
rounded-md border bg-popover text-popover-foreground shadow
data-[state=open]:animate-in data-[state=closed]:animate-out
data-[state=closed]:zoom-out-95 data-[state=open]:zoom-in-90
md:w-[var(--radix-navigation-menu-viewport-width)]",
        className
      )}
      ref={ref}
      {...props} />
  </div>
))
NavigationMenuViewport.displayName =
  NavigationMenuPrimitive.Viewport.displayName

const NavigationMenuIndicator = React.forwardRef(({ className, ...props },
ref) => (
  <NavigationMenuPrimitive.Indicator
    ref={ref}
    className={cn(
      "top-full z-[1] flex h-1.5 items-end justify-center overflow-hidden
data-[state=visible]:animate-in data-[state=hidden]:animate-out
data-[state=hidden]:fade-out data-[state=visible]:fade-in",
      className
    )}
    {...props}>
    <div
      className="relative top-[60%] h-2 w-2 rotate-45 rounded-tl-sm bg-border
shadow-md" />
  </NavigationMenuPrimitive.Indicator>
))
NavigationMenuIndicator.displayName =
  NavigationMenuPrimitive.Indicator.displayName

export {
  navigationMenuTriggerStyle,
  NavigationMenu,
  NavigationMenuList,
  NavigationMenuItem,
  NavigationMenuContent,
  NavigationMenuTrigger,
  NavigationMenuLink,
  NavigationMenuIndicator,
  NavigationMenuViewport,
}
```

pagination.jsx

```jsx
import * as React from "react"
import { ChevronLeft, ChevronRight, MoreHorizontal } from "lucide-react"

import { cn } from "@/lib/utils"
import { buttonVariants } from "@/components/ui/button";

const Pagination = ({
  className,
  ...props
}) => (
  <nav
    role="navigation"
    aria-label="pagination"
    className={cn("mx-auto flex w-full justify-center", className)}
    {...props} />
)
Pagination.displayName = "Pagination"

const PaginationContent = React.forwardRef(({ className, ...props }, ref) =>
(
  <ul
    ref={ref}
    className={cn("flex flex-row items-center gap-1", className)}
    {...props} />
))
PaginationContent.displayName = "PaginationContent"

const PaginationItem = React.forwardRef(({ className, ...props }, ref) => (
  <li ref={ref} className={cn("", className)} {...props} />
))
PaginationItem.displayName = "PaginationItem"

const PaginationLink = ({
  className,
  isActive,
  size = "icon",
  ...props
}) => (
  <a
    aria-current={isActive ? "page" : undefined}
    className={cn(buttonVariants({
      variant: isActive ? "outline" : "ghost",
      size,
```

```
    }), className)}
    {...props} />
)
PaginationLink.displayName = "PaginationLink"

const PaginationPrevious = ({
  className,
  ...props
}) => (
  <PaginationLink
    aria-label="Go to previous page"
    size="default"
    className={cn("gap-1 pl-2.5", className)}
    {...props}>
    <ChevronLeft className="h-4 w-4" />
    <span>Previous</span>
  </PaginationLink>
)
PaginationPrevious.displayName = "PaginationPrevious"

const PaginationNext = ({
  className,
  ...props
}) => (
  <PaginationLink
    aria-label="Go to next page"
    size="default"
    className={cn("gap-1 pr-2.5", className)}
    {...props}>
    <span>Next</span>
    <ChevronRight className="h-4 w-4" />
  </PaginationLink>
)
PaginationNext.displayName = "PaginationNext"

const PaginationEllipsis = ({
  className,
  ...props
}) => (
  <span
    aria-hidden
    className={cn("flex h-9 w-9 items-center justify-center", className)}
    {...props}>
    <MoreHorizontal className="h-4 w-4" />
    <span className="sr-only">More pages</span>
```

```
  </span>
)
PaginationEllipsis.displayName = "PaginationEllipsis"

export {
  Pagination,
  PaginationContent,
  PaginationLink,
  PaginationItem,
  PaginationPrevious,
  PaginationNext,
  PaginationEllipsis,
}
```

popover.jsx

```
import * as React from "react"
import * as PopoverPrimitive from "@radix-ui/react-popover"

import { cn } from "@/lib/utils"

const Popover = PopoverPrimitive.Root

const PopoverTrigger = PopoverPrimitive.Trigger

const PopoverAnchor = PopoverPrimitive.Anchor

const PopoverContent = React.forwardRef(({ className, align = "center",
sideOffset = 4, ...props }, ref) => (
  <PopoverPrimitive.Portal>
    <PopoverPrimitive.Content
      ref={ref}
      align={align}
      sideOffset={sideOffset}
      className={cn(
        "z-50 w-72 rounded-md border bg-popover p-4 text-popover-foreground
shadow-md outline-none data-[state=open]:animate-in
data-[state=closed]:animate-out data-[state=closed]:fade-out-0
data-[state=open]:fade-in-0 data-[state=closed]:zoom-out-95
data-[state=open]:zoom-in-95 data-[side=bottom]:slide-in-from-top-2
data-[side=left]:slide-in-from-right-2 data-[side=right]:slide-in-from-left-2
data-[side=top]:slide-in-from-bottom-2",
        className
      )}
      {...props} />
  </PopoverPrimitive.Portal>
```

```
))
PopoverContent.displayName = PopoverPrimitive.Content.displayName

export { Popover, PopoverTrigger, PopoverContent, PopoverAnchor }
```

progress.jsx

```
"use client"

import * as React from "react"
import * as ProgressPrimitive from "@radix-ui/react-progress"

import { cn } from "@/lib/utils"

const Progress = React.forwardRef(({ className, value, ...props }, ref) => (
  <ProgressPrimitive.Root
    ref={ref}
    className={cn(
      "relative h-2 w-full overflow-hidden rounded-full bg-primary/20",
      className
    )}
    {...props}>
    <ProgressPrimitive.Indicator
      className="h-full w-full flex-1 bg-primary transition-all"
      style={{ transform: `translateX(-${100 - (value || 0)}%)` }} />
  </ProgressPrimitive.Root>
))
Progress.displayName = ProgressPrimitive.Root.displayName

export { Progress }
```

radio-group.jsx

```
import * as React from "react"
import * as RadioGroupPrimitive from "@radix-ui/react-radio-group"
import { Circle } from "lucide-react"

import { cn } from "@/lib/utils"

const RadioGroup = React.forwardRef(({ className, ...props }, ref) => {
  return (<RadioGroupPrimitive.Root className={cn("grid gap-2", className)}
{...props} ref={ref} />);
})
RadioGroup.displayName = RadioGroupPrimitive.Root.displayName

const RadioGroupItem = React.forwardRef(({ className, ...props }, ref) => {
```

```
  return (
    (<RadioGroupPrimitive.Item
      ref={ref}
      className={cn(
        "aspect-square h-4 w-4 rounded-full border border-primary
text-primary shadow focus:outline-none focus-visible:ring-1
focus-visible:ring-ring disabled:cursor-not-allowed disabled:opacity-50",
        className
      )}
      {...props}>
      <RadioGroupPrimitive.Indicator className="flex items-center
justify-center">
        <Circle className="h-3.5 w-3.5 fill-primary" />
      </RadioGroupPrimitive.Indicator>
    </RadioGroupPrimitive.Item>)
  );
})
RadioGroupItem.displayName = RadioGroupPrimitive.Item.displayName

export { RadioGroup, RadioGroupItem }
```

resizable.jsx

```
"use client"

import { GripVertical } from "lucide-react"
import * as ResizablePrimitive from "react-resizable-panels"

import { cn } from "@/lib/utils"

const ResizablePanelGroup = ({
  className,
  ...props
}) => (
  <ResizablePrimitive.PanelGroup
    className={cn(
      "flex h-full w-full data-[panel-group-direction=vertical]:flex-col",
      className
    )}
    {...props} />
)

const ResizablePanel = ResizablePrimitive.Panel

const ResizableHandle = ({
  withHandle,
```

```
  className,
  ...props
}) => (
  <ResizablePrimitive.PanelResizeHandle
    className={cn(
      "relative flex w-px items-center justify-center bg-border
after:absolute after:inset-y-0 after:left-1/2 after:w-1
after:-translate-x-1/2 focus-visible:outline-none focus-visible:ring-1
focus-visible:ring-ring focus-visible:ring-offset-1
data-[panel-group-direction=vertical]:h-px
data-[panel-group-direction=vertical]:w-full
data-[panel-group-direction=vertical]:after:left-0
data-[panel-group-direction=vertical]:after:h-1
data-[panel-group-direction=vertical]:after:w-full
data-[panel-group-direction=vertical]:after:-translate-y-1/2
data-[panel-group-direction=vertical]:after:translate-x-0
[&[data-panel-group-direction=vertical]>div]:rotate-90",
      className
    )}
    {...props}>
    {withHandle && (
      <div
        className="z-10 flex h-4 w-3 items-center justify-center rounded-sm
border bg-border">
        <GripVertical className="h-2.5 w-2.5" />
      </div>
    )}
  </ResizablePrimitive.PanelResizeHandle>
)

export { ResizablePanelGroup, ResizablePanel, ResizableHandle }
```

responsive-image.jsx

```
import * as React from "react"
import { cn } from "../../lib/utils"
import { buildSrcSet, buildTransformUrl, splitImageProps } from
"./image-helpers"
import { useResponsiveImage } from "./use-responsive-image"

export const ResponsiveImage = React.forwardRef(
```

```
  ({ src, parsed, fittingType, focalPoint, quality, className, style,
aspectRatio, onLoad, onSourceChange, ...props }, ref) => {
    const { wrapperRef, imgRef, loaded, options, handleLoad } =
useResponsiveImage(
      { parsed, fittingType, focalPoint, quality, className, onLoad,
onSourceChange }, ref
    )
    const { wrapperProps, imageProps } = splitImageProps(props)

    return (
      <span
        ref={wrapperRef}
        className={cn("inline-block relative", className)}
        style={{ aspectRatio, ...style }}
        {...wrapperProps}
        data-base44-image=""
        data-base44-image-src={src}
      >
        {/* Contain both image layers inside the padded content box without
adding an edit target. */}
        <span data-source-location={undefined} className="block relative
w-full h-full overflow-hidden">
          {options && !loaded && (
            <img
              data-source-location={undefined}
              src={buildTransformUrl(parsed, {
                ...options,
                width: 20,
                height: options.height
                  ? Math.max(1, Math.round((20 * options.height) /
options.width))
                  : undefined,
                quality: 20,
              })}
              alt=""
              aria-hidden="true"
              className="w-full h-full inset-0 absolute"
              style={{
                objectFit: fittingType === "fit" ? "contain" : "cover",
                filter: "blur(10px)",
                transform: "scale(1.1)",
              }}
            />
          )}
          {options && (
```

```
            <img
              data-source-location={undefined}
              ref={imgRef}
              src={buildTransformUrl(parsed, options)}
              srcSet={buildSrcSet(parsed, options)}
              loading="lazy"
              className={cn(
                "w-full h-full inset-0 absolute",
                fittingType === "fit" ? "object-contain" : "object-cover"
              )}
              onLoad={handleLoad}
              {...imageProps}
            />
          )}
        </span>
      </span>
    )
  }
)
ResponsiveImage.displayName = "ResponsiveImage"
```

scroll-area.jsx

```
import * as React from "react"
import * as ScrollAreaPrimitive from "@radix-ui/react-scroll-area"

import { cn } from "@/lib/utils"

const ScrollArea = React.forwardRef(({ className, children, ...props }, ref)
=> (
  <ScrollAreaPrimitive.Root
    ref={ref}
    className={cn("relative overflow-hidden", className)}
    {...props}>
    <ScrollAreaPrimitive.Viewport className="h-full w-full
rounded-[inherit]">
      {children}
    </ScrollAreaPrimitive.Viewport>
    <ScrollBar />
    <ScrollAreaPrimitive.Corner />
  </ScrollAreaPrimitive.Root>
))
ScrollArea.displayName = ScrollAreaPrimitive.Root.displayName

const ScrollBar = React.forwardRef(({ className, orientation = "vertical",
...props }, ref) => (
```

```
  <ScrollAreaPrimitive.ScrollAreaScrollbar
    ref={ref}
    orientation={orientation}
    className={cn(
      "flex touch-none select-none transition-colors",
      orientation === "vertical" &&
        "h-full w-2.5 border-l border-l-transparent p-[1px]",
      orientation === "horizontal" &&
        "h-2.5 flex-col border-t border-t-transparent p-[1px]",
      className
    )}
    {...props}>
    <ScrollAreaPrimitive.ScrollAreaThumb className="relative flex-1
rounded-full bg-border" />
  </ScrollAreaPrimitive.ScrollAreaScrollbar>
))
ScrollBar.displayName = ScrollAreaPrimitive.ScrollAreaScrollbar.displayName

export { ScrollArea, ScrollBar }
```

select.jsx

```
"use client"

import * as React from "react"
import * as SelectPrimitive from "@radix-ui/react-select"
import { Check, ChevronDown, ChevronUp } from "lucide-react"

import { cn } from "@/lib/utils"

const Select = SelectPrimitive.Root

const SelectGroup = SelectPrimitive.Group

const SelectValue = SelectPrimitive.Value

const SelectTrigger = React.forwardRef(({ className, children, ...props },
ref) => (
  <SelectPrimitive.Trigger
    ref={ref}
    className={cn(
      "flex h-9 w-full items-center justify-between whitespace-nowrap
rounded-md border border-input bg-transparent px-3 py-2 text-sm shadow-sm
ring-offset-background data-[placeholder]:text-muted-foreground
focus:outline-none focus:ring-1 focus:ring-ring disabled:cursor-not-allowed
disabled:opacity-50 [&>span]:line-clamp-1",
```

```
      className
    )}
    {...props}>
    {children}
    <SelectPrimitive.Icon asChild>
      <ChevronDown className="h-4 w-4 opacity-50" />
    </SelectPrimitive.Icon>
  </SelectPrimitive.Trigger>
))
SelectTrigger.displayName = SelectPrimitive.Trigger.displayName

const SelectScrollUpButton = React.forwardRef(({ className, ...props }, ref)
=> (
  <SelectPrimitive.ScrollUpButton
    ref={ref}
    className={cn("flex cursor-default items-center justify-center py-1",
className)}
    {...props}>
    <ChevronUp className="h-4 w-4" />
  </SelectPrimitive.ScrollUpButton>
))
SelectScrollUpButton.displayName = SelectPrimitive.ScrollUpButton.displayName

const SelectScrollDownButton = React.forwardRef(({ className, ...props },
ref) => (
  <SelectPrimitive.ScrollDownButton
    ref={ref}
    className={cn("flex cursor-default items-center justify-center py-1",
className)}
    {...props}>
    <ChevronDown className="h-4 w-4" />
  </SelectPrimitive.ScrollDownButton>
))
SelectScrollDownButton.displayName =
  SelectPrimitive.ScrollDownButton.displayName

const SelectContent = React.forwardRef(({ className, children, position =
"popper", ...props }, ref) => (
  <SelectPrimitive.Portal>
    <SelectPrimitive.Content
      ref={ref}
      className={cn(
        "relative z-50 max-h-96 min-w-[8rem] overflow-hidden rounded-md
border bg-popover text-popover-foreground shadow-md
data-[state=open]:animate-in data-[state=closed]:animate-out
```

```
data-[state=closed]:fade-out-0 data-[state=open]:fade-in-0
data-[state=closed]:zoom-out-95 data-[state=open]:zoom-in-95
data-[side=bottom]:slide-in-from-top-2 data-[side=left]:slide-in-from-right-2
data-[side=right]:slide-in-from-left-2
data-[side=top]:slide-in-from-bottom-2",
        position === "popper" &&
          "data-[side=bottom]:translate-y-1 data-[side=left]:-translate-x-1
data-[side=right]:translate-x-1 data-[side=top]:-translate-y-1",
        className
      )}
      position={position}
      {...props}>
      <SelectScrollUpButton />
      <SelectPrimitive.Viewport
        className={cn("p-1", position === "popper" &&
          "h-[var(--radix-select-trigger-height)] w-full
min-w-[var(--radix-select-trigger-width)]")}>
        {children}
      </SelectPrimitive.Viewport>
      <SelectScrollDownButton />
    </SelectPrimitive.Content>
  </SelectPrimitive.Portal>
))
SelectContent.displayName = SelectPrimitive.Content.displayName

const SelectLabel = React.forwardRef(({ className, ...props }, ref) => (
  <SelectPrimitive.Label
    ref={ref}
    className={cn("px-2 py-1.5 text-sm font-semibold", className)}
    {...props} />
))
SelectLabel.displayName = SelectPrimitive.Label.displayName

const SelectItem = React.forwardRef(({ className, children, ...props }, ref)
=> (
  <SelectPrimitive.Item
    ref={ref}
    className={cn(
      "relative flex w-full cursor-default select-none items-center
rounded-sm py-1.5 pl-2 pr-8 text-sm outline-none focus:bg-accent
focus:text-accent-foreground data-[disabled]:pointer-events-none
data-[disabled]:opacity-50",
      className
    )}
    {...props}>
```

```
    <span className="absolute right-2 flex h-3.5 w-3.5 items-center
justify-center">
      <SelectPrimitive.ItemIndicator>
        <Check className="h-4 w-4" />
      </SelectPrimitive.ItemIndicator>
    </span>
    <SelectPrimitive.ItemText>{children}</SelectPrimitive.ItemText>
  </SelectPrimitive.Item>
))
SelectItem.displayName = SelectPrimitive.Item.displayName

const SelectSeparator = React.forwardRef(({ className, ...props }, ref) => (
  <SelectPrimitive.Separator
    ref={ref}
    className={cn("-mx-1 my-1 h-px bg-muted", className)}
    {...props} />
))
SelectSeparator.displayName = SelectPrimitive.Separator.displayName

export {
  Select,
  SelectGroup,
  SelectValue,
  SelectTrigger,
  SelectContent,
  SelectLabel,
  SelectItem,
  SelectSeparator,
  SelectScrollUpButton,
  SelectScrollDownButton,
}
```

separator.jsx

```
import * as React from "react"
import * as SeparatorPrimitive from "@radix-ui/react-separator"

import { cn } from "@/lib/utils"

const Separator = React.forwardRef((
  { className, orientation = "horizontal", decorative = true, ...props },
  ref
) => (
  <SeparatorPrimitive.Root
    ref={ref}
    decorative={decorative}
```

```
    orientation={orientation}
    className={cn(
      "shrink-0 bg-border",
      orientation === "horizontal" ? "h-[1px] w-full" : "h-full w-[1px]",
      className
    )}
    {...props} />
))
Separator.displayName = SeparatorPrimitive.Root.displayName

export { Separator }
```

sheet.jsx

```
"use client";
import * as React from "react"
import * as SheetPrimitive from "@radix-ui/react-dialog"
import { cva } from "class-variance-authority";
import { X } from "lucide-react"

import { cn } from "@/lib/utils"

const Sheet = SheetPrimitive.Root

const SheetTrigger = SheetPrimitive.Trigger

const SheetClose = SheetPrimitive.Close

const SheetPortal = SheetPrimitive.Portal

const SheetOverlay = React.forwardRef(({ className, ...props }, ref) => (
  <SheetPrimitive.Overlay
    className={cn(
      "fixed inset-0 z-50 bg-black/80  data-[state=open]:animate-in
data-[state=closed]:animate-out data-[state=closed]:fade-out-0
data-[state=open]:fade-in-0",
      className
    )}
    {...props}
    ref={ref} />
))
SheetOverlay.displayName = SheetPrimitive.Overlay.displayName

const sheetVariants = cva(
```

```
  "fixed z-50 gap-4 bg-background p-6 shadow-lg transition ease-in-out
data-[state=closed]:duration-300 data-[state=open]:duration-500
data-[state=open]:animate-in data-[state=closed]:animate-out",
  {
    variants: {
      side: {
        top: "inset-x-0 top-0 border-b data-[state=closed]:slide-out-to-top
data-[state=open]:slide-in-from-top",
        bottom:
          "inset-x-0 bottom-0 border-t
data-[state=closed]:slide-out-to-bottom
data-[state=open]:slide-in-from-bottom",
        left: "inset-y-0 left-0 h-full w-3/4 border-r
data-[state=closed]:slide-out-to-left data-[state=open]:slide-in-from-left
sm:max-w-sm",
        right:
          "inset-y-0 right-0 h-full w-3/4 border-l
data-[state=closed]:slide-out-to-right data-[state=open]:slide-in-from-right
sm:max-w-sm",
      },
    },
    defaultVariants: {
      side: "right",
    },
  }
)

const SheetContent = React.forwardRef(({ side = "right", className, children,
...props }, ref) => (
  <SheetPortal>
    <SheetOverlay />
    <SheetPrimitive.Content ref={ref} className={cn(sheetVariants({ side }),
className)} {...props}>
      <SheetPrimitive.Close
        className="absolute right-4 top-4 rounded-sm opacity-70
ring-offset-background transition-opacity hover:opacity-100
focus:outline-none focus:ring-2 focus:ring-ring focus:ring-offset-2
disabled:pointer-events-none data-[state=open]:bg-secondary">
        <X className="h-4 w-4" />
        <span className="sr-only">Close</span>
      </SheetPrimitive.Close>
      {children}
    </SheetPrimitive.Content>
  </SheetPortal>
))
```

```
SheetContent.displayName = SheetPrimitive.Content.displayName

const SheetHeader = ({
  className,
  ...props
}) => (
  <div
    className={cn("flex flex-col space-y-2 text-center sm:text-left",
className)}
    {...props} />
)
SheetHeader.displayName = "SheetHeader"

const SheetFooter = ({
  className,
  ...props
}) => (
  <div
    className={cn("flex flex-col-reverse sm:flex-row sm:justify-end
sm:space-x-2", className)}
    {...props} />
)
SheetFooter.displayName = "SheetFooter"

const SheetTitle = React.forwardRef(({ className, ...props }, ref) => (
  <SheetPrimitive.Title
    ref={ref}
    className={cn("text-lg font-semibold text-foreground", className)}
    {...props} />
))
SheetTitle.displayName = SheetPrimitive.Title.displayName

const SheetDescription = React.forwardRef(({ className, ...props }, ref) => (
  <SheetPrimitive.Description
    ref={ref}
    className={cn("text-sm text-muted-foreground", className)}
    {...props} />
))
SheetDescription.displayName = SheetPrimitive.Description.displayName

export {
  Sheet,
  SheetPortal,
  SheetOverlay,
  SheetTrigger,
```

```
  SheetClose,
  SheetContent,
  SheetHeader,
  SheetFooter,
  SheetTitle,
  SheetDescription,
}
```

sidebar.jsx

```
import * as React from "react"
import { Slot } from "@radix-ui/react-slot"
import { cva } from "class-variance-authority";
import { PanelLeft } from "lucide-react"

import { useIsMobile } from "@/hooks/use-mobile"
import { cn } from "@/lib/utils"
import { Button } from "@/components/ui/button"
import { Input } from "@/components/ui/input"
import { Separator } from "@/components/ui/separator"
import { Sheet, SheetContent } from "@/components/ui/sheet"
import { Skeleton } from "@/components/ui/skeleton"
import {
  Tooltip,
  TooltipContent,
  TooltipProvider,
  TooltipTrigger,
} from "@/components/ui/tooltip"

const SIDEBAR_COOKIE_NAME = "sidebar_state"
const SIDEBAR_COOKIE_MAX_AGE = 60 * 60 * 24 * 7
const SIDEBAR_WIDTH = "16rem"
const SIDEBAR_WIDTH_MOBILE = "18rem"
const SIDEBAR_WIDTH_ICON = "3rem"
const SIDEBAR_KEYBOARD_SHORTCUT = "b"

const SidebarContext = React.createContext(null)

function useSidebar() {
  const context = React.useContext(SidebarContext)
  if (!context) {
    throw new Error("useSidebar must be used within a SidebarProvider.")
  }

  return context
}
```

```
const SidebarProvider = React.forwardRef((
  {
    defaultOpen = true,
    open: openProp,
    onOpenChange: setOpenProp,
    className,
    style,
    children,
    ...props
  },
  ref
) => {
  const isMobile = useIsMobile()
  const [openMobile, setOpenMobile] = React.useState(false)

  // This is the internal state of the sidebar.
  // We use openProp and setOpenProp for control from outside the component.
  const [_open, _setOpen] = React.useState(defaultOpen)
  const open = openProp ?? _open
  const setOpen = React.useCallback((value) => {
    const openState = typeof value === "function" ? value(open) : value
    if (setOpenProp) {
      setOpenProp(openState)
    } else {
      _setOpen(openState)
    }

    // This sets the cookie to keep the sidebar state.
    document.cookie = `${SIDEBAR_COOKIE_NAME}=${openState}; path=/;
max-age=${SIDEBAR_COOKIE_MAX_AGE}`
  }, [setOpenProp, open])

  // Helper to toggle the sidebar.
  const toggleSidebar = React.useCallback(() => {
    return isMobile
      ? setOpenMobile((open) => !open)
      : setOpen((open) => !open);
  }, [isMobile, setOpen, setOpenMobile])

  // Adds a keyboard shortcut to toggle the sidebar.
  React.useEffect(() => {
    const handleKeyDown = (event) => {
      if (
        event.key === SIDEBAR_KEYBOARD_SHORTCUT &&
```

```
        (event.metaKey || event.ctrlKey)
      ) {
        event.preventDefault()
        toggleSidebar()
      }
    }

    window.addEventListener("keydown", handleKeyDown)
    return () => window.removeEventListener("keydown", handleKeyDown);
  }, [toggleSidebar])

  // We add a state so that we can do data-state="expanded" or "collapsed".
  // This makes it easier to style the sidebar with Tailwind classes.
  const state = open ? "expanded" : "collapsed"

  const contextValue = React.useMemo(() => ({
    state,
    open,
    setOpen,
    isMobile,
    openMobile,
    setOpenMobile,
    toggleSidebar,
  }), [state, open, setOpen, isMobile, openMobile, setOpenMobile,
toggleSidebar])

  return (
    (<SidebarContext.Provider value={contextValue}>
      <TooltipProvider delayDuration={0}>
        <div
          style={
            {
              "--sidebar-width": SIDEBAR_WIDTH,
              "--sidebar-width-icon": SIDEBAR_WIDTH_ICON,
              ...style
            }
          }
          className={cn(
            "group/sidebar-wrapper flex min-h-svh w-full
has-[[data-variant=inset]]:bg-sidebar",
            className
          )}
          ref={ref}
          {...props}>
          {children}
```

```
        </div>
      </TooltipProvider>
    </SidebarContext.Provider>)
  );
})
SidebarProvider.displayName = "SidebarProvider"

const Sidebar = React.forwardRef((
  {
    side = "left",
    variant = "sidebar",
    collapsible = "offcanvas",
    className,
    children,
    ...props
  },
  ref
) => {
  const { isMobile, state, openMobile, setOpenMobile } = useSidebar()

  if (collapsible === "none") {
    return (
      (<div
        className={cn(
          "flex h-full w-[--sidebar-width] flex-col bg-sidebar
text-sidebar-foreground",
          className
        )}
        ref={ref}
        {...props}>
        {children}
      </div>)
    );
  }

  if (isMobile) {
    return (
      (<Sheet open={openMobile} onOpenChange={setOpenMobile} {...props}>
        <SheetContent
          data-sidebar="sidebar"
          data-mobile="true"
          className="w-[--sidebar-width] bg-sidebar p-0
text-sidebar-foreground [&>button]:hidden"
          style={
            {
```

```
              "--sidebar-width": SIDEBAR_WIDTH_MOBILE
            }
          }
          side={side}>
          <div className="flex h-full w-full flex-col">{children}</div>
        </SheetContent>
      </Sheet>)
    );
  }

  return (
    (<div
      ref={ref}
      className="group peer hidden text-sidebar-foreground md:block"
      data-state={state}
      data-collapsible={state === "collapsed" ? collapsible : ""}
      data-variant={variant}
      data-side={side}>
      {/* This is what handles the sidebar gap on desktop */}
      <div
        className={cn(
          "relative h-svh w-[--sidebar-width] bg-transparent
transition-[width] duration-200 ease-linear",
          "group-data-[collapsible=offcanvas]:w-0",
          "group-data-[side=right]:rotate-180",
          variant === "floating" || variant === "inset"
            ?
"group-data-[collapsible=icon]:w-[calc(var(--sidebar-width-icon)_+_theme(spac
ing.4))]"
            : "group-data-[collapsible=icon]:w-[--sidebar-width-icon]"
        )} />
      <div
        className={cn(
          "fixed inset-y-0 z-10 hidden h-svh w-[--sidebar-width]
transition-[left,right,width] duration-200 ease-linear md:flex",
          side === "left"
            ? "left-0
group-data-[collapsible=offcanvas]:left-[calc(var(--sidebar-width)*-1)]"
            : "right-0
group-data-[collapsible=offcanvas]:right-[calc(var(--sidebar-width)*-1)]",
          // Adjust the padding for floating and inset variants.
          variant === "floating" || variant === "inset"
            ? "p-2
group-data-[collapsible=icon]:w-[calc(var(--sidebar-width-icon)_+_theme(spaci
ng.4)_+2px)]"
```

```
            : "group-data-[collapsible=icon]:w-[--sidebar-width-icon]
group-data-[side=left]:border-r group-data-[side=right]:border-l",
          className
        )}
        {...props}>
        <div
          data-sidebar="sidebar"
          className="flex h-full w-full flex-col bg-sidebar
group-data-[variant=floating]:rounded-lg group-data-[variant=floating]:border
group-data-[variant=floating]:border-sidebar-border
group-data-[variant=floating]:shadow">
          {children}
        </div>
      </div>
    </div>)
  );
})
Sidebar.displayName = "Sidebar"

const SidebarTrigger = React.forwardRef(({ className, onClick, asChild =
false, ...props }, ref) => {
  const { toggleSidebar } = useSidebar()

  return (
    (<Button
      ref={ref}
      data-sidebar="trigger"
      variant="ghost"
      size="icon"
      className={cn("h-7 w-7", className)}
      onClick={(event) => {
        onClick?.(event)
        toggleSidebar()
      }}
      asChild={asChild}
      {...props}>
      {asChild ? (
        <PanelLeft />
      ) : (
        <>
          <PanelLeft />
          <span className="sr-only">Toggle Sidebar</span>
        </>
      )}
    </Button>)
```

```
  );
})
SidebarTrigger.displayName = "SidebarTrigger"

const SidebarRail = React.forwardRef(({ className, ...props }, ref) => {
  const { toggleSidebar } = useSidebar()

  return (
    (<button
      ref={ref}
      data-sidebar="rail"
      aria-label="Toggle Sidebar"
      tabIndex={-1}
      onClick={toggleSidebar}
      title="Toggle Sidebar"
      className={cn(
        "absolute inset-y-0 z-20 hidden w-4 -translate-x-1/2 transition-all
ease-linear after:absolute after:inset-y-0 after:left-1/2 after:w-[2px]
hover:after:bg-sidebar-border group-data-[side=left]:-right-4
group-data-[side=right]:left-0 sm:flex",
        "[[data-side=left]_&]:cursor-w-resize
[[data-side=right]_&]:cursor-e-resize",
        "[[data-side=left][data-state=collapsed]_&]:cursor-e-resize
[[data-side=right][data-state=collapsed]_&]:cursor-w-resize",
        "group-data-[collapsible=offcanvas]:translate-x-0
group-data-[collapsible=offcanvas]:after:left-full
group-data-[collapsible=offcanvas]:hover:bg-sidebar",
        "[[data-side=left][data-collapsible=offcanvas]_&]:-right-2",
        "[[data-side=right][data-collapsible=offcanvas]_&]:-left-2",
        className
      )}
      {...props} />)
  );
})
SidebarRail.displayName = "SidebarRail"

const SidebarInset = React.forwardRef(({ className, ...props }, ref) => {
  return (
    (<main
      ref={ref}
      className={cn(
        "relative flex min-h-svh flex-1 flex-col bg-background",
        "peer-data-[variant=inset]:min-h-[calc(100svh-theme(spacing.4))]
md:peer-data-[variant=inset]:m-2
md:peer-data-[state=collapsed]:peer-data-[variant=inset]:ml-2
```

```
md:peer-data-[variant=inset]:ml-0 md:peer-data-[variant=inset]:rounded-xl
md:peer-data-[variant=inset]:shadow",
        className
      )}
      {...props} />)
  );
})
SidebarInset.displayName = "SidebarInset"

const SidebarInput = React.forwardRef(({ className, ...props }, ref) => {
  return (
    (<Input
      ref={ref}
      data-sidebar="input"
      className={cn(
        "h-8 w-full bg-background shadow-none focus-visible:ring-2
focus-visible:ring-sidebar-ring",
        className
      )}
      {...props} />)
  );
})
SidebarInput.displayName = "SidebarInput"

const SidebarHeader = React.forwardRef(({ className, ...props }, ref) => {
  return (
    (<div
      ref={ref}
      data-sidebar="header"
      className={cn("flex flex-col gap-2 p-2", className)}
      {...props} />)
  );
})
SidebarHeader.displayName = "SidebarHeader"

const SidebarFooter = React.forwardRef(({ className, ...props }, ref) => {
  return (
    (<div
      ref={ref}
      data-sidebar="footer"
      className={cn("flex flex-col gap-2 p-2", className)}
      {...props} />)
  );
})
SidebarFooter.displayName = "SidebarFooter"
```

```
const SidebarSeparator = React.forwardRef(({ className, ...props }, ref) => {
  return (
    (<Separator
      ref={ref}
      data-sidebar="separator"
      className={cn("mx-2 w-auto bg-sidebar-border", className)}
      {...props} />)
  );
})
SidebarSeparator.displayName = "SidebarSeparator"

const SidebarContent = React.forwardRef(({ className, ...props }, ref) => {
  return (
    (<div
      ref={ref}
      data-sidebar="content"
      className={cn(
        "flex min-h-0 flex-1 flex-col gap-2 overflow-auto
group-data-[collapsible=icon]:overflow-hidden",
        className
      )}
      {...props} />)
  );
})
SidebarContent.displayName = "SidebarContent"

const SidebarGroup = React.forwardRef(({ className, ...props }, ref) => {
  return (
    (<div
      ref={ref}
      data-sidebar="group"
      className={cn("relative flex w-full min-w-0 flex-col p-2", className)}
      {...props} />)
  );
})
SidebarGroup.displayName = "SidebarGroup"

const SidebarGroupLabel = React.forwardRef(({ className, asChild = false,
...props }, ref) => {
  const Comp = asChild ? Slot : "div"

  return (
    (<Comp
      ref={ref}
```

```
      data-sidebar="group-label"
      className={cn(
        "flex h-8 shrink-0 items-center rounded-md px-2 text-xs font-medium
text-sidebar-foreground/70 outline-none ring-sidebar-ring
transition-[margin,opacity] duration-200 ease-linear focus-visible:ring-2
[&>svg]:size-4 [&>svg]:shrink-0",
        "group-data-[collapsible=icon]:-mt-8
group-data-[collapsible=icon]:opacity-0",
        className
      )}
      {...props} />)
  );
})
SidebarGroupLabel.displayName = "SidebarGroupLabel"

const SidebarGroupAction = React.forwardRef(({ className, asChild = false,
...props }, ref) => {
  const Comp = asChild ? Slot : "button"

  return (
    (<Comp
      ref={ref}
      data-sidebar="group-action"
      className={cn(
        "absolute right-3 top-3.5 flex aspect-square w-5 items-center
justify-center rounded-md p-0 text-sidebar-foreground outline-none
ring-sidebar-ring transition-transform hover:bg-sidebar-accent
hover:text-sidebar-accent-foreground focus-visible:ring-2 [&>svg]:size-4
[&>svg]:shrink-0",
        // Increases the hit area of the button on mobile.
        "after:absolute after:-inset-2 after:md:hidden",
        "group-data-[collapsible=icon]:hidden",
        className
      )}
      {...props} />)
  );
})
SidebarGroupAction.displayName = "SidebarGroupAction"

const SidebarGroupContent = React.forwardRef(({ className, ...props }, ref)
=> (
  <div
    ref={ref}
    data-sidebar="group-content"
    className={cn("w-full text-sm", className)}
```

```
    {...props} />
))
SidebarGroupContent.displayName = "SidebarGroupContent"

const SidebarMenu = React.forwardRef(({ className, ...props }, ref) => (
  <ul
    ref={ref}
    data-sidebar="menu"
    className={cn("flex w-full min-w-0 flex-col gap-1", className)}
    {...props} />
))
SidebarMenu.displayName = "SidebarMenu"

const SidebarMenuItem = React.forwardRef(({ className, ...props }, ref) => (
  <li
    ref={ref}
    data-sidebar="menu-item"
    className={cn("group/menu-item relative", className)}
    {...props} />
))
SidebarMenuItem.displayName = "SidebarMenuItem"

const sidebarMenuButtonVariants = cva(
  "peer/menu-button flex w-full items-center gap-2 overflow-hidden rounded-md
p-2 text-left text-sm outline-none ring-sidebar-ring
transition-[width,height,padding] hover:bg-sidebar-accent
hover:text-sidebar-accent-foreground focus-visible:ring-2
active:bg-sidebar-accent active:text-sidebar-accent-foreground
disabled:pointer-events-none disabled:opacity-50
group-has-[[data-sidebar=menu-action]]/menu-item:pr-8
aria-disabled:pointer-events-none aria-disabled:opacity-50
data-[active=true]:bg-sidebar-accent data-[active=true]:font-medium
data-[active=true]:text-sidebar-accent-foreground
data-[state=open]:hover:bg-sidebar-accent
data-[state=open]:hover:text-sidebar-accent-foreground
group-data-[collapsible=icon]:!size-8 group-data-[collapsible=icon]:!p-2
[&>span:last-child]:truncate [&>svg]:size-4 [&>svg]:shrink-0",
  {
    variants: {
      variant: {
        default: "hover:bg-sidebar-accent
hover:text-sidebar-accent-foreground",
        outline:
```

```
          "bg-background shadow-[0_0_0_1px_hsl(var(--sidebar-border))]
hover:bg-sidebar-accent hover:text-sidebar-accent-foreground
hover:shadow-[0_0_0_1px_hsl(var(--sidebar-accent))]",
      },
      size: {
        default: "h-8 text-sm",
        sm: "h-7 text-xs",
        lg: "h-12 text-sm group-data-[collapsible=icon]:!p-0",
      },
    },
    defaultVariants: {
      variant: "default",
      size: "default",
    },
  }
)

const SidebarMenuButton = React.forwardRef((
  {
    asChild = false,
    isActive = false,
    variant = "default",
    size = "default",
    tooltip,
    className,
    ...props
  },
  ref
) => {
  const Comp = asChild ? Slot : "button"
  const { isMobile, state } = useSidebar()

  const button = (
    <Comp
      ref={ref}
      data-sidebar="menu-button"
      data-size={size}
      data-active={isActive}
      className={cn(sidebarMenuButtonVariants({ variant, size }), className)}
      {...props} />
  )

  if (!tooltip) {
    return button
  }
```

```
  if (typeof tooltip === "string") {
    tooltip = {
      children: tooltip,
    }
  }

  return (
    (<Tooltip>
      <TooltipTrigger asChild>{button}</TooltipTrigger>
      <TooltipContent
        side="right"
        align="center"
        hidden={state !== "collapsed" || isMobile}
        {...tooltip} />
    </Tooltip>)
  );
})
SidebarMenuButton.displayName = "SidebarMenuButton"

const SidebarMenuAction = React.forwardRef(({ className, asChild = false,
showOnHover = false, ...props }, ref) => {
  const Comp = asChild ? Slot : "button"

  return (
    (<Comp
      ref={ref}
      data-sidebar="menu-action"
      className={cn(
        "absolute right-1 top-1.5 flex aspect-square w-5 items-center
justify-center rounded-md p-0 text-sidebar-foreground outline-none
ring-sidebar-ring transition-transform hover:bg-sidebar-accent
hover:text-sidebar-accent-foreground focus-visible:ring-2
peer-hover/menu-button:text-sidebar-accent-foreground [&>svg]:size-4
[&>svg]:shrink-0",
        // Increases the hit area of the button on mobile.
        "after:absolute after:-inset-2 after:md:hidden",
        "peer-data-[size=sm]/menu-button:top-1",
        "peer-data-[size=default]/menu-button:top-1.5",
        "peer-data-[size=lg]/menu-button:top-2.5",
        "group-data-[collapsible=icon]:hidden",
        showOnHover &&
        "group-focus-within/menu-item:opacity-100
group-hover/menu-item:opacity-100 data-[state=open]:opacity-100
```

```
peer-data-[active=true]/menu-button:text-sidebar-accent-foreground
md:opacity-0",
        className
      )}
      {...props} />)
  );
})
SidebarMenuAction.displayName = "SidebarMenuAction"

const SidebarMenuBadge = React.forwardRef(({ className, ...props }, ref) => (
  <div
    ref={ref}
    data-sidebar="menu-badge"
    className={cn(
      "pointer-events-none absolute right-1 flex h-5 min-w-5 select-none
items-center justify-center rounded-md px-1 text-xs font-medium tabular-nums
text-sidebar-foreground",
      "peer-hover/menu-button:text-sidebar-accent-foreground
peer-data-[active=true]/menu-button:text-sidebar-accent-foreground",
      "peer-data-[size=sm]/menu-button:top-1",
      "peer-data-[size=default]/menu-button:top-1.5",
      "peer-data-[size=lg]/menu-button:top-2.5",
      "group-data-[collapsible=icon]:hidden",
      className
    )}
    {...props} />
))
SidebarMenuBadge.displayName = "SidebarMenuBadge"

const SidebarMenuSkeleton = React.forwardRef(({ className, showIcon = false,
...props }, ref) => {
  // Random width between 50 to 90%.
  const width = React.useMemo(() => {
    return `${Math.floor(Math.random() * 40) + 50}%`;
  }, [])

  return (
    (<div
      ref={ref}
      data-sidebar="menu-skeleton"
      className={cn("flex h-8 items-center gap-2 rounded-md px-2",
className)}
      {...props}>
      {showIcon && (
```

```
        <Skeleton className="size-4 rounded-md"
data-sidebar="menu-skeleton-icon" />
      )}
      <Skeleton
        className="h-4 max-w-[--skeleton-width] flex-1"
        data-sidebar="menu-skeleton-text"
        style={
          {
            "--skeleton-width": width
          }
        } />
    </div>)
  );
})
SidebarMenuSkeleton.displayName = "SidebarMenuSkeleton"

const SidebarMenuSub = React.forwardRef(({ className, ...props }, ref) => (
  <ul
    ref={ref}
    data-sidebar="menu-sub"
    className={cn(
      "mx-3.5 flex min-w-0 translate-x-px flex-col gap-1 border-l
border-sidebar-border px-2.5 py-0.5",
      "group-data-[collapsible=icon]:hidden",
      className
    )}
    {...props} />
))
SidebarMenuSub.displayName = "SidebarMenuSub"

const SidebarMenuSubItem = React.forwardRef(({ ...props }, ref) => <li
ref={ref} {...props} />)
SidebarMenuSubItem.displayName = "SidebarMenuSubItem"

const SidebarMenuSubButton = React.forwardRef(
  ({ asChild = false, size = "md", isActive, className, ...props }, ref) => {
    const Comp = asChild ? Slot : "a"

    return (
      (<Comp
        ref={ref}
        data-sidebar="menu-sub-button"
        data-size={size}
        data-active={isActive}
        className={cn(
```

```
          "flex h-7 min-w-0 -translate-x-px items-center gap-2
overflow-hidden rounded-md px-2 text-sidebar-foreground outline-none
ring-sidebar-ring hover:bg-sidebar-accent
hover:text-sidebar-accent-foreground focus-visible:ring-2
active:bg-sidebar-accent active:text-sidebar-accent-foreground
disabled:pointer-events-none disabled:opacity-50
aria-disabled:pointer-events-none aria-disabled:opacity-50
[&>span:last-child]:truncate [&>svg]:size-4 [&>svg]:shrink-0
[&>svg]:text-sidebar-accent-foreground",
          "data-[active=true]:bg-sidebar-accent
data-[active=true]:text-sidebar-accent-foreground",
          size === "sm" && "text-xs",
          size === "md" && "text-sm",
          "group-data-[collapsible=icon]:hidden",
          className
        )}
        {...props} />)
    );
  }
)
SidebarMenuSubButton.displayName = "SidebarMenuSubButton"

export {
  Sidebar,
  SidebarContent,
  SidebarFooter,
  SidebarGroup,
  SidebarGroupAction,
  SidebarGroupContent,
  SidebarGroupLabel,
  SidebarHeader,
  SidebarInput,
  SidebarInset,
  SidebarMenu,
  SidebarMenuAction,
  SidebarMenuBadge,
  SidebarMenuButton,
  SidebarMenuItem,
  SidebarMenuSkeleton,
  SidebarMenuSub,
  SidebarMenuSubButton,
  SidebarMenuSubItem,
  SidebarProvider,
  SidebarRail,
  SidebarSeparator,
```

```
  SidebarTrigger,
  useSidebar,
}
```

skeleton.jsx

```
import { cn } from "@/lib/utils"

function Skeleton({
  className,
  ...props
}) {
  return (
    (<div
      className={cn("animate-pulse rounded-md bg-primary/10", className)}
      {...props} />)
  );
}

export { Skeleton }
```

slider.jsx

```
import * as React from "react"
import * as SliderPrimitive from "@radix-ui/react-slider"

import { cn } from "@/lib/utils"

const Slider = React.forwardRef(({ className, ...props }, ref) => (
  <SliderPrimitive.Root
    ref={ref}
    className={cn("relative flex w-full touch-none select-none items-center",
className)}
    {...props}>
    <SliderPrimitive.Track
      className="relative h-1.5 w-full grow overflow-hidden rounded-full
bg-primary/20">
      <SliderPrimitive.Range className="absolute h-full bg-primary" />
    </SliderPrimitive.Track>
    <SliderPrimitive.Thumb
      className="block h-4 w-4 rounded-full border border-primary/50
bg-background shadow transition-colors focus-visible:outline-none
focus-visible:ring-1 focus-visible:ring-ring disabled:pointer-events-none
disabled:opacity-50" />
  </SliderPrimitive.Root>
))
```

```
Slider.displayName = SliderPrimitive.Root.displayName

export { Slider }
```

sonner.jsx

```
"use client";
import { useTheme } from "next-themes"
import { Toaster as Sonner } from "sonner"

const Toaster = ({
  ...props
}) => {
  const { theme = "system" } = useTheme()

  return (
    (<Sonner
      theme={theme}
      className="toaster group"
      toastOptions={{
        classNames: {
          toast:
            "group toast group-[.toaster]:bg-background
group-[.toaster]:text-foreground group-[.toaster]:border-border
group-[.toaster]:shadow-lg",
          description: "group-[.toast]:text-muted-foreground",
          actionButton:
            "group-[.toast]:bg-primary
group-[.toast]:text-primary-foreground",
          cancelButton:
            "group-[.toast]:bg-muted group-[.toast]:text-muted-foreground",
        },
      }}
      {...props} />)
  );
}

export { Toaster }
```

switch.jsx

```
import * as React from "react"
import * as SwitchPrimitives from "@radix-ui/react-switch"

import { cn } from "@/lib/utils"
```

```
const Switch = React.forwardRef(({ className, ...props }, ref) => (
  <SwitchPrimitives.Root
    className={cn(
      "peer inline-flex h-5 w-9 shrink-0 cursor-pointer items-center
rounded-full border-2 border-transparent shadow-sm transition-colors
focus-visible:outline-none focus-visible:ring-2 focus-visible:ring-ring
focus-visible:ring-offset-2 focus-visible:ring-offset-background
disabled:cursor-not-allowed disabled:opacity-50
data-[state=checked]:bg-primary data-[state=unchecked]:bg-input",
      className
    )}
    {...props}
    ref={ref}>
    <SwitchPrimitives.Thumb
      className={cn(
        "pointer-events-none block h-4 w-4 rounded-full bg-background
shadow-lg ring-0 transition-transform data-[state=checked]:translate-x-4
data-[state=unchecked]:translate-x-0"
      )} />
  </SwitchPrimitives.Root>
))
Switch.displayName = SwitchPrimitives.Root.displayName

export { Switch }
```

table.jsx

```
import * as React from "react"

import { cn } from "@/lib/utils"

const Table = React.forwardRef(({ className, ...props }, ref) => (
  <div className="relative w-full overflow-auto">
    <table
      ref={ref}
      className={cn("w-full caption-bottom text-sm", className)}
      {...props} />
  </div>
))
Table.displayName = "Table"

const TableHeader = React.forwardRef(({ className, ...props }, ref) => (
  <thead ref={ref} className={cn("[&_tr]:border-b", className)} {...props} />
))
TableHeader.displayName = "TableHeader"
```

```
const TableBody = React.forwardRef(({ className, ...props }, ref) => (
  <tbody
    ref={ref}
    className={cn("[&_tr:last-child]:border-0", className)}
    {...props} />
))
TableBody.displayName = "TableBody"

const TableFooter = React.forwardRef(({ className, ...props }, ref) => (
  <tfoot
    ref={ref}
    className={cn("border-t bg-muted/50 font-medium [&>tr]:last:border-b-0",
className)}
    {...props} />
))
TableFooter.displayName = "TableFooter"

const TableRow = React.forwardRef(({ className, ...props }, ref) => (
  <tr
    ref={ref}
    className={cn(
      "border-b transition-colors hover:bg-muted/50
data-[state=selected]:bg-muted",
      className
    )}
    {...props} />
))
TableRow.displayName = "TableRow"

const TableHead = React.forwardRef(({ className, ...props }, ref) => (
  <th
    ref={ref}
    className={cn(
      "h-10 px-2 text-left align-middle font-medium text-muted-foreground
[&:has([role=checkbox])]:pr-0 [&>[role=checkbox]]:translate-y-[2px]",
      className
    )}
    {...props} />
))
TableHead.displayName = "TableHead"

const TableCell = React.forwardRef(({ className, ...props }, ref) => (
  <td
    ref={ref}
```

```
    className={cn(
      "p-2 align-middle [&:has([role=checkbox])]:pr-0
[&>[role=checkbox]]:translate-y-[2px]",
      className
    )}
    {...props} />
))
TableCell.displayName = "TableCell"

const TableCaption = React.forwardRef(({ className, ...props }, ref) => (
  <caption
    ref={ref}
    className={cn("mt-4 text-sm text-muted-foreground", className)}
    {...props} />
))
TableCaption.displayName = "TableCaption"

export {
  Table,
  TableHeader,
  TableBody,
  TableFooter,
  TableHead,
  TableRow,
  TableCell,
  TableCaption,
}
```

tabs.jsx

```
import * as React from "react"
import * as TabsPrimitive from "@radix-ui/react-tabs"

import { cn } from "@/lib/utils"

const Tabs = TabsPrimitive.Root

const TabsList = React.forwardRef(({ className, ...props }, ref) => (
  <TabsPrimitive.List
    ref={ref}
    className={cn(
      "inline-flex h-9 items-center justify-center rounded-lg bg-muted p-1
text-muted-foreground",
      className
    )}
    {...props} />
```

```
))
TabsList.displayName = TabsPrimitive.List.displayName

const TabsTrigger = React.forwardRef(({ className, ...props }, ref) => (
  <TabsPrimitive.Trigger
    ref={ref}
    className={cn(
      "inline-flex items-center justify-center whitespace-nowrap rounded-md
px-3 py-1 text-sm font-medium ring-offset-background transition-all
focus-visible:outline-none focus-visible:ring-2 focus-visible:ring-ring
focus-visible:ring-offset-2 disabled:pointer-events-none disabled:opacity-50
data-[state=active]:bg-background data-[state=active]:text-foreground
data-[state=active]:shadow",
      className
    )}
    {...props} />
))
TabsTrigger.displayName = TabsPrimitive.Trigger.displayName

const TabsContent = React.forwardRef(({ className, ...props }, ref) => (
  <TabsPrimitive.Content
    ref={ref}
    className={cn(
      "mt-2 ring-offset-background focus-visible:outline-none
focus-visible:ring-2 focus-visible:ring-ring focus-visible:ring-offset-2",
      className
    )}
    {...props} />
))
TabsContent.displayName = TabsPrimitive.Content.displayName

export { Tabs, TabsList, TabsTrigger, TabsContent }
```

textarea.jsx

```
import * as React from "react"

import { cn } from "@/lib/utils"

const Textarea = React.forwardRef(({ className, ...props }, ref) => {
  return (
    (<textarea
      className={cn(
        "flex min-h-[60px] w-full rounded-md border border-input
bg-transparent px-3 py-2 text-base shadow-sm
placeholder:text-muted-foreground focus-visible:outline-none
```

```
focus-visible:ring-1 focus-visible:ring-ring disabled:cursor-not-allowed
disabled:opacity-50 md:text-sm",
        className
      )}
      ref={ref}
      {...props} />)
  );
})
Textarea.displayName = "Textarea"

export { Textarea }
```

toast.jsx

```
import * as React from "react";
import * as ToastPrimitives from "@radix-ui/react-toast";
import { cva } from "class-variance-authority";
import { X } from "lucide-react";
import { cn } from "@/lib/utils";

const ToastProvider = ToastPrimitives.Provider;

const ToastViewport = React.forwardRef(({ className, ...props }, ref) => (
  <ToastPrimitives.Viewport
    ref={ref}
    className={cn(
      "fixed top-0 z-[100] flex max-h-screen w-full flex-col-reverse p-4
sm:bottom-0 sm:right-0 sm:top-auto sm:flex-col md:max-w-[420px]",
      className
    )}
    {...props}
  />
));
ToastViewport.displayName = ToastPrimitives.Viewport.displayName;

const toastVariants = cva(
  "group pointer-events-auto relative flex w-full items-center
justify-between space-x-4 overflow-hidden rounded-md border p-6 pr-8
shadow-lg transition-all data-[swipe=cancel]:translate-x-0
data-[swipe=end]:translate-x-[var(--radix-toast-swipe-end-x)]
data-[swipe=move]:translate-x-[var(--radix-toast-swipe-move-x)]
data-[swipe=move]:transition-none data-[state=open]:animate-in
data-[state=closed]:animate-out data-[swipe=end]:animate-out
data-[state=closed]:fade-out-80 data-[state=closed]:slide-out-to-right-full
data-[state=open]:slide-in-from-top-full
data-[state=open]:sm:slide-in-from-bottom-full",
```

```
  {
    variants: {
      variant: {
        default: "border bg-background text-foreground",
        destructive:
          "destructive group border-destructive bg-destructive
text-destructive-foreground",
      },
    },
    defaultVariants: {
      variant: "default",
    },
  }
);

const Toast = React.forwardRef(({ className, variant, ...props }, ref) => {
  return (
    <ToastPrimitives.Root
      ref={ref}
      className={cn(toastVariants({ variant }), className)}
      {...props}
    />
  );
});
Toast.displayName = ToastPrimitives.Root.displayName;

const ToastAction = React.forwardRef(({ className, ...props }, ref) => (
  <ToastPrimitives.Action
    ref={ref}
    className={cn(
      "inline-flex h-8 shrink-0 items-center justify-center rounded-md border
bg-transparent px-3 text-sm font-medium ring-offset-background
transition-colors hover:bg-secondary focus:outline-none focus:ring-2
focus:ring-ring focus:ring-offset-2 disabled:pointer-events-none
disabled:opacity-50 group-[.destructive]:border-muted/40
group-[.destructive]:hover:border-destructive/30
group-[.destructive]:hover:bg-destructive
group-[.destructive]:hover:text-destructive-foreground
group-[.destructive]:focus:ring-destructive",
      className
    )}
    {...props}
  />
));
ToastAction.displayName = ToastPrimitives.Action.displayName;
```

```
const ToastClose = React.forwardRef(({ className, ...props }, ref) => (
  <ToastPrimitives.Close
    ref={ref}
    className={cn(
      "absolute right-2 top-2 rounded-md p-1 text-foreground/50 opacity-0
transition-opacity hover:text-foreground focus:opacity-100 focus:outline-none
focus:ring-2 group-hover:opacity-100 group-[.destructive]:text-red-300
group-[.destructive]:hover:text-red-50
group-[.destructive]:focus:ring-red-400
group-[.destructive]:focus:ring-offset-red-600",
      className
    )}
    toast-close=""
    {...props}
  >
    <X className="h-4 w-4" />
  </ToastPrimitives.Close>
));
ToastClose.displayName = ToastPrimitives.Close.displayName;

const ToastTitle = React.forwardRef(({ className, ...props }, ref) => (
  <ToastPrimitives.Title
    ref={ref}
    className={cn("text-sm font-semibold", className)}
    {...props}
  />
));
ToastTitle.displayName = ToastPrimitives.Title.displayName;

const ToastDescription = React.forwardRef(({ className, ...props }, ref) => (
  <ToastPrimitives.Description
    ref={ref}
    className={cn("text-sm opacity-90", className)}
    {...props}
  />
));
ToastDescription.displayName = ToastPrimitives.Description.displayName;

export {
  ToastProvider,
  ToastViewport,
  Toast,
  ToastTitle,
  ToastDescription,
```

```
  ToastClose,
  ToastAction,
};
```

toaster.jsx

```
import { useToast } from "@/components/ui/use-toast";
import {
  Toast,
  ToastClose,
  ToastDescription,
  ToastProvider,
  ToastTitle,
  ToastViewport,
} from "@/components/ui/toast";

export function Toaster() {
  const { toasts } = useToast();

  return (
    <ToastProvider>
      {toasts.map(function ({ id, title, description, action, ...props }) {
        return (
          <Toast key={id} {...props}>
            <div className="grid gap-1">
              {title && <ToastTitle>{title}</ToastTitle>}
              {description && (
                <ToastDescription>{description}</ToastDescription>
              )}
            </div>
            {action}
            <ToastClose />
          </Toast>
        );
      })}
      <ToastViewport />
    </ToastProvider>
  );
}
```

toggle-group.jsx

```
"use client";
import * as React from "react"
import * as ToggleGroupPrimitive from "@radix-ui/react-toggle-group"
```

```
import { cn } from "@/lib/utils"
import { toggleVariants } from "@/components/ui/toggle"

const ToggleGroupContext = React.createContext({
  size: "default",
  variant: "default",
})

const ToggleGroup = React.forwardRef(({ className, variant, size, children,
...props }, ref) => (
  <ToggleGroupPrimitive.Root
    ref={ref}
    className={cn("flex items-center justify-center gap-1", className)}
    {...props}>
    <ToggleGroupContext.Provider value={{ variant, size }}>
      {children}
    </ToggleGroupContext.Provider>
  </ToggleGroupPrimitive.Root>
))

ToggleGroup.displayName = ToggleGroupPrimitive.Root.displayName

const ToggleGroupItem = React.forwardRef(({ className, children, variant,
size, ...props }, ref) => {
  const context = React.useContext(ToggleGroupContext)

  return (
    (<ToggleGroupPrimitive.Item
      ref={ref}
      className={cn(toggleVariants({
        variant: context.variant || variant,
        size: context.size || size,
      }), className)}
      {...props}>
      {children}
    </ToggleGroupPrimitive.Item>)
  );
})

ToggleGroupItem.displayName = ToggleGroupPrimitive.Item.displayName

export { ToggleGroup, ToggleGroupItem }
```

toggle.jsx

```jsx
import * as React from "react"
import * as TogglePrimitive from "@radix-ui/react-toggle"
import { cva } from "class-variance-authority";

import { cn } from "@/lib/utils"

const toggleVariants = cva(
  "inline-flex items-center justify-center gap-2 rounded-md text-sm
font-medium transition-colors hover:bg-muted hover:text-muted-foreground
focus-visible:outline-none focus-visible:ring-1 focus-visible:ring-ring
disabled:pointer-events-none disabled:opacity-50 data-[state=on]:bg-accent
data-[state=on]:text-accent-foreground [&_svg]:pointer-events-none
[&_svg]:size-4 [&_svg]:shrink-0",
  {
    variants: {
      variant: {
        default: "bg-transparent",
        outline:
          "border border-input bg-transparent shadow-sm hover:bg-accent
hover:text-accent-foreground",
      },
      size: {
        default: "h-9 px-2 min-w-9",
        sm: "h-8 px-1.5 min-w-8",
        lg: "h-10 px-2.5 min-w-10",
      },
    },
    defaultVariants: {
      variant: "default",
      size: "default",
    },
  }
)

const Toggle = React.forwardRef(({ className, variant, size, ...props }, ref)
=> (
  <TogglePrimitive.Root
    ref={ref}
    className={cn(toggleVariants({ variant, size, className }))}
    {...props} />
))

Toggle.displayName = TogglePrimitive.Root.displayName

export { Toggle, toggleVariants }
```

tooltip.jsx

```
"use client"

import * as React from "react"
import * as TooltipPrimitive from "@radix-ui/react-tooltip"

import { cn } from "@/lib/utils"

const TooltipProvider = TooltipPrimitive.Provider

const Tooltip = TooltipPrimitive.Root

const TooltipTrigger = TooltipPrimitive.Trigger

const TooltipContent = React.forwardRef(({ className, sideOffset = 4,
...props }, ref) => (
  <TooltipPrimitive.Portal>
    <TooltipPrimitive.Content
      ref={ref}
      sideOffset={sideOffset}
      className={cn(
        "z-50 overflow-hidden rounded-md bg-primary px-3 py-1.5 text-xs
text-primary-foreground animate-in fade-in-0 zoom-in-95
data-[state=closed]:animate-out data-[state=closed]:fade-out-0
data-[state=closed]:zoom-out-95 data-[side=bottom]:slide-in-from-top-2
data-[side=left]:slide-in-from-right-2 data-[side=right]:slide-in-from-left-2
data-[side=top]:slide-in-from-bottom-2",
        className
      )}
      {...props} />
  </TooltipPrimitive.Portal>
))
TooltipContent.displayName = TooltipPrimitive.Content.displayName

export { Tooltip, TooltipTrigger, TooltipContent, TooltipProvider }
```

use-responsive-image.jsx

```
import * as React from "react"
import { useSize } from "../../hooks/use-size"
import { cn } from "../../lib/utils"
import { DEFAULT_TRANSFORM_WIDTH, getImagePreviewClassName } from
"./image-helpers"
```

```
export function useResponsiveImage({ parsed, fittingType, focalPoint,
quality, className, onLoad, onSourceChange }, parentRef) {
  const wrapperRef = React.useRef(null)
  const imgRef = React.useRef(null)
  const size = useSize(wrapperRef)
  const [loaded, setLoaded] = React.useState(false)

  React.useImperativeHandle(parentRef, () => imgRef.current)
  React.useEffect(() => setLoaded(false), [parsed.baseUrl])
  React.useEffect(() => {
    const wrapper = wrapperRef.current
    const replace = (event) => onSourceChange(
      event.detail.src, getImagePreviewClassName(className,
wrapper.className, cn("inline-block relative", className))
    )
    wrapper.addEventListener("base44:image-replace", replace)
    return () => wrapper.removeEventListener("base44:image-replace", replace)
  }, [className, onSourceChange])

  const crop = fittingType !== "fit"
  // Wait for useSize's pre-paint measurement before requesting a transform.
  const options = size && {
    width: size.width || DEFAULT_TRANSFORM_WIDTH,
    height: size.height || undefined,
    crop,
    focalPoint: crop ? focalPoint : undefined,
    quality,
  }

  return {
    wrapperRef, imgRef, loaded, options,
    handleLoad: (event) => {
      setLoaded(true)
      onLoad?.(event)
    },
  }
}
```

use-toast.jsx

```
// Inspired by react-hot-toast library
import { useState, useEffect } from "react";

const TOAST_LIMIT = 20;
const TOAST_REMOVE_DELAY = 1000000;
```

```
const actionTypes = {
  ADD_TOAST: "ADD_TOAST",
  UPDATE_TOAST: "UPDATE_TOAST",
  DISMISS_TOAST: "DISMISS_TOAST",
  REMOVE_TOAST: "REMOVE_TOAST",
};

let count = 0;

function genId() {
  count = (count + 1) % Number.MAX_VALUE;
  return count.toString();
}

const toastTimeouts = new Map();

const addToRemoveQueue = (toastId) => {
  if (toastTimeouts.has(toastId)) {
    return;
  }

  const timeout = setTimeout(() => {
    toastTimeouts.delete(toastId);
    dispatch({
      type: actionTypes.REMOVE_TOAST,
      toastId,
    });
  }, TOAST_REMOVE_DELAY);

  toastTimeouts.set(toastId, timeout);
};

const _clearFromRemoveQueue = (toastId) => {
  const timeout = toastTimeouts.get(toastId);
  if (timeout) {
    clearTimeout(timeout);
    toastTimeouts.delete(toastId);
  }
};

export const reducer = (state, action) => {
  switch (action.type) {
    case actionTypes.ADD_TOAST:
      return {
        ...state,
```

```
        toasts: [action.toast, ...state.toasts].slice(0, TOAST_LIMIT),
      };

    case actionTypes.UPDATE_TOAST:
      return {
        ...state,
        toasts: state.toasts.map((t) =>
          t.id === action.toast.id ? { ...t, ...action.toast } : t
        ),
      };

    case actionTypes.DISMISS_TOAST: {
      const { toastId } = action;

      // ! Side effects ! - This could be extracted into a dismissToast()
action,
      // but I'll keep it here for simplicity
      if (toastId) {
        addToRemoveQueue(toastId);
      } else {
        state.toasts.forEach((toast) => {
          addToRemoveQueue(toast.id);
        });
      }

      return {
        ...state,
        toasts: state.toasts.map((t) =>
          t.id === toastId || toastId === undefined
            ? {
                ...t,
                open: false,
              }
            : t
        ),
      };
    }
    case actionTypes.REMOVE_TOAST:
      if (action.toastId === undefined) {
        return {
          ...state,
          toasts: [],
        };
      }
      return {
```

```
        ...state,
        toasts: state.toasts.filter((t) => t.id !== action.toastId),
      };
  }
};

const listeners = [];

let memoryState = { toasts: [] };

function dispatch(action) {
  memoryState = reducer(memoryState, action);
  listeners.forEach((listener) => {
    listener(memoryState);
  });
}

function toast({ ...props }) {
  const id = genId();

  const update = (props) =>
    dispatch({
      type: actionTypes.UPDATE_TOAST,
      toast: { ...props, id },
    });

  const dismiss = () =>
    dispatch({ type: actionTypes.DISMISS_TOAST, toastId: id });

  dispatch({
    type: actionTypes.ADD_TOAST,
    toast: {
      ...props,
      id,
      open: true,
      onOpenChange: (open) => {
        if (!open) dismiss();
      },
    },
  });

  return {
    id,
    dismiss,
    update,
```

```
  };
}

function useToast() {
  const [state, setState] = useState(memoryState);

  useEffect(() => {
    listeners.push(setState);
    return () => {
      const index = listeners.indexOf(setState);
      if (index > -1) {
        listeners.splice(index, 1);
      }
    };
  }, [state]);

  return {
    ...state,
    toast,
    dismiss: (toastId) => dispatch({ type: actionTypes.DISMISS_TOAST, toastId
}),
  };
}

export { useToast, toast };
```

/components

AnalysisDashboard.jsx

```
import { BarChart, Bar, XAxis, YAxis, ResponsiveContainer, Tooltip, Cell }
from 'recharts';
import LandCoverPie from './LandCoverPie';
import ScoreGauge from './ScoreGauge';
import StatCard from './StatCard';
import { CATEGORY_COLORS, formatNumber, formatArea, ratingTone } from
'@/lib/geoUtils';
import { Sprout, Leaf, ArrowRight } from 'lucide-react';

export default function AnalysisDashboard({ data, onSave, saving, saved }) {
  const breakdown = data.land_use_breakdown && data.land_use_breakdown.length
    ? data.land_use_breakdown
    : [
        { category: 'Vegetation', pct: data.vegetation_pct, color:
CATEGORY_COLORS.Vegetation },
        { category: 'Buildings', pct: data.buildings_pct, color:
CATEGORY_COLORS.Buildings },
```

```
        { category: 'Roads', pct: data.roads_pct, color:
CATEGORY_COLORS.Roads },
        { category: 'Open Land', pct: data.open_land_pct, color:
CATEGORY_COLORS['Open Land'] },
        { category: 'Water', pct: data.water_pct, color:
CATEGORY_COLORS.Water },
      ];

  const tone = ratingTone(data.green_cover_rating);
  const barData = breakdown.map((d) => ({
    name: d.category,
    pct: Math.max(0, Number(d.pct) || 0),
    color: d.color || CATEGORY_COLORS[d.category] || '#8a8275',
  }));

  return (
    <div className="space-y-6">
      {/* Header */}
      <div className="flex flex-col gap-4 rounded-3xl border border-border
bg-card p-6 sm:flex-row sm:items-center sm:justify-between">
        <div>
          <div className="flex items-center gap-2 text-xs uppercase
tracking-[0.18em] text-muted-foreground">
            <Leaf className="h-3.5 w-3.5" />
            Environmental Report
          </div>
          <h2 className="mt-2 font-display text-2xl font-semibold
text-foreground sm:text-3xl">
            {data.location_name}
          </h2>
          <p className="mt-1 text-sm text-muted-foreground">
            {formatArea(data.area_hectares)} analyzed ·
{data.lat?.toFixed(4)}, {data.lng?.toFixed(4)}
          </p>
        </div>
        <div className="flex items-center gap-3">
          <div className={`flex items-center gap-2 rounded-full px-3.5 py-2
${tone.bg}`}>
            <span className="h-2 w-2 rounded-full" style={{ background:
tone.dot }} />
            <span className={`text-sm font-medium
${tone.text}`}>{data.green_cover_rating} green cover</span>
          </div>
        </div>
      </div>
```

```
      {/* Stat cards */}
      <div className="grid grid-cols-2 gap-4 lg:grid-cols-4">
        <StatCard type="vegetation" label="Vegetation cover"
value={`${formatNumber(data.vegetation_pct, 1)}`} suffix="%" accent="#5f7d56"
hint="Trees, shrubs, grass" />
        <StatCard type="buildings" label="Built structures"
value={`${formatNumber(data.buildings_pct, 1)}`} suffix="%" accent="#5a6168"
hint="Buildings & rooftops" />
        <StatCard type="absorption" label="CO₂ absorbed"
value={formatNumber(data.carbon_absorption_tons_year, 1)} suffix="t/yr"
accent="#4f8a93" hint="By local vegetation" />
        <StatCard type="trees" label="Tree equivalents"
value={formatNumber(data.tree_canopy_estimate)} accent="#6f8a64"
hint="Mature-tree canopy" />
      </div>

      {/* Charts row */}
      <div className="grid grid-cols-1 gap-6 lg:grid-cols-3">
        <div className="rounded-3xl border border-border bg-card p-6">
          <h3 className="font-display text-lg font-semibold
text-foreground">Land-cover distribution</h3>
          <p className="mt-0.5 text-xs text-muted-foreground">Indicative
share of the analyzed area</p>
          <div className="mt-5">
            <LandCoverPie breakdown={breakdown} />
          </div>
        </div>

        <div className="rounded-3xl border border-border bg-card p-6">
          <h3 className="font-display text-lg font-semibold
text-foreground">Sustainability profile</h3>
          <p className="mt-0.5 text-xs text-muted-foreground">Composite
environmental indicators</p>
          <div className="mt-6 flex items-center justify-around gap-4">
            <ScoreGauge score={data.sustainability_score}
label="Sustainability" />
            <ScoreGauge score={data.carbon_impact_score} label="Carbon
impact" invert />
          </div>
        </div>

        <div className="rounded-3xl border border-border bg-card p-6">
          <h3 className="font-display text-lg font-semibold
text-foreground">Cover comparison</h3>
```

```
          <p className="mt-0.5 text-xs text-muted-foreground">Percentage by
land-use class</p>
          <div className="mt-5 h-56">
            <ResponsiveContainer width="100%" height="100%">
              <BarChart data={barData} margin={{ top: 8, right: 8, left: -16,
bottom: 0 }}>
                <XAxis dataKey="name" tick={{ fontSize: 10, fill:
'hsl(var(--muted-foreground))' }} axisLine={false} tickLine={false}
interval={0} angle={-12} textAnchor="end" height={40} />
                <YAxis tick={{ fontSize: 10, fill:
'hsl(var(--muted-foreground))' }} axisLine={false} tickLine={false} />
                <Tooltip
                  cursor={{ fill: 'hsl(var(--muted))' }}
                  formatter={(v) => [`${Number(v).toFixed(1)}%`, 'Cover']}
                  contentStyle={{ background: 'hsl(var(--card))', border:
'1px solid hsl(var(--border))', borderRadius: '0.5rem', fontSize: '12px' }}
                />
                <Bar dataKey="pct" radius={[6, 6, 0, 0]}>
                  {barData.map((entry, i) => (
                    <Cell key={i} fill={entry.color} />
                  ))}
                </Bar>
              </BarChart>
            </ResponsiveContainer>
          </div>
        </div>
      </div>

      {/* Summary + recommendations */}
      <div className="grid grid-cols-1 gap-6 lg:grid-cols-2">
        <div className="rounded-3xl border border-border bg-card p-6">
          <h3 className="font-display text-lg font-semibold
text-foreground">Assessment</h3>
          <p className="mt-3 text-sm leading-relaxed
text-foreground/80">{data.summary}</p>
          <div className="mt-5 grid grid-cols-2 gap-3">
            <div className="rounded-xl bg-muted/60 p-4">
              <p className="text-xs uppercase tracking-[0.14em]
text-muted-foreground">Carbon impact</p>
              <p className="mt-1 font-display text-xl font-semibold
text-foreground">{formatNumber(data.carbon_impact_score, 0)}<span
className="text-sm text-muted-foreground">/100</span></p>
            </div>
            <div className="rounded-xl bg-muted/60 p-4">
```

```
              <p className="text-xs uppercase tracking-[0.14em]
text-muted-foreground">Sustainability</p>
              <p className="mt-1 font-display text-xl font-semibold
text-foreground">{formatNumber(data.sustainability_score, 0)}<span
className="text-sm text-muted-foreground">/100</span></p>
            </div>
          </div>
        </div>

        <div className="rounded-3xl border border-border bg-card p-6">
          <h3 className="font-display text-lg font-semibold
text-foreground">Recommendations</h3>
          <ul className="mt-4 space-y-3">
            {(data.recommendations || []).map((rec, i) => (
              <li key={i} className="flex gap-3 text-sm leading-relaxed
text-foreground/80">
                <span className="mt-0.5 flex h-5 w-5 flex-none items-center
justify-center rounded-full bg-land-vegetation/15 text-land-vegetation">
                  <Sprout className="h-3 w-3" />
                </span>
                <span>{rec}</span>
              </li>
            ))}
            {!data.recommendations?.length && <li className="text-sm
text-muted-foreground">No recommendations returned.</li>}
          </ul>
        </div>
      </div>

      {/* Save action */}
      {onSave && (
        <div className="flex flex-col items-center justify-between gap-4
rounded-3xl border border-border bg-gradient-to-br from-land-vegetation/10
via-card to-card p-6 sm:flex-row">
          <div>
            <h3 className="font-display text-lg font-semibold
text-foreground">Save this report</h3>
            <p className="mt-0.5 text-sm text-muted-foreground">Keep this
analysis in your history to revisit or compare later.</p>
          </div>
          <button
            onClick={onSave}
            disabled={saving || saved}
```

```
            className="inline-flex items-center gap-2 rounded-full bg-primary
px-6 py-3 text-sm font-medium text-primary-foreground transition-all
hover:opacity-90 disabled:opacity-50"
          >
            {saved ? 'Saved to history' : saving ? 'Saving…' : 'Save report'}
            {!saving && !saved && <ArrowRight className="h-4 w-4" />}
          </button>
        </div>
      )}
    </div>
  );
}
```

AppLayout.jsx

```
import { Outlet } from 'react-router-dom';
import SiteNav from './SiteNav';

export default function AppLayout() {
  return (
    <div className="flex min-h-screen flex-col bg-background">
      <SiteNav />
      <main className="flex-1">
        <Outlet />
      </main>
      <footer className="border-t border-border/60 bg-card/40">
        <div className="mx-auto max-w-7xl px-4 py-8 sm:px-6 lg:px-8">
          <p className="text-xs text-muted-foreground">
            GreenScope AI · Indicative environmental assessments for planning
purposes. Land-cover and carbon figures are model-based estimates, not field
measurements.
          </p>
        </div>
      </footer>
    </div>
  );
}
```

AuthLayout.jsx

```
import React from "react";

export default function AuthLayout({ icon: Icon, title, subtitle, footer,
children }) {
  return (
    <div className="min-h-screen flex items-center justify-center
bg-background px-4">
```

```
      <div className="w-full max-w-md">
        <div className="text-center mb-10">
          <div className="inline-flex items-center justify-center w-14 h-14
rounded-2xl bg-primary mb-4">
            <Icon className="w-7 h-7 text-primary-foreground"
aria-hidden="true" />
          </div>
          <h1 className="text-3xl font-bold tracking-tight
text-foreground">{title}</h1>
          {subtitle && <p className="text-muted-foreground
mt-2">{subtitle}</p>}
        </div>
        <div className="bg-card rounded-2xl shadow-sm border border-border
p-8">
          {children}
        </div>
        {footer && (
          <p className="text-center text-sm text-muted-foreground
mt-6">{footer}</p>
        )}
      </div>
    </div>
  );
}
```

GoogleIcon.jsx

```
import React from "react";

export default function GoogleIcon({ className = "w-5 h-5" }) {
  return (
    <svg className={className} viewBox="0 0 24 24" aria-hidden="true">
      <path d="M22.56 12.25c0-.78-.07-1.53-.2-2.25H12v4.26h5.92a5.06 5.06 0
01-2.2 3.32v2.77h3.57c2.08-1.92 3.28-4.74 3.28-8.1z" fill="#4285F4" />
      <path d="M12 23c2.97 0 5.46-.98 7.28-2.66l-3.57-2.77c-.98.66-2.23
1.06-3.71 1.06-2.86 0-5.29-1.93-6.16-4.53H2.18v2.84C3.99 20.53 7.7 23 12 23z"
fill="#34A853" />
      <path d="M5.84
14.09c-.22-.66-.35-1.36-.35-2.09s.13-1.43.35-2.09V7.07H2.18C1.43 8.55 1 10.22
1 12s.43 3.45 1.18 4.93l2.85-2.22.81-.62z" fill="#FBBC05" />
      <path d="M12 5.38c1.62 0 3.06.56 4.21 1.64l3.15-3.15C17.45 2.09 14.97 1
12 1 7.7 1 3.99 3.47 2.18 7.07l3.66 2.84c.87-2.6 3.3-4.53 6.16-4.53z"
fill="#EA4335" />
    </svg>
  );
}
```

LandCoverPie.jsx

```
import { PieChart, Pie, Cell, ResponsiveContainer, Tooltip } from 'recharts';
import { CATEGORY_COLORS } from '@/lib/geoUtils';

export default function LandCoverPie({ breakdown }) {
  const data = (breakdown || []).map((d) => ({
    name: d.category,
    value: Math.max(0, Number(d.pct) || 0),
    color: d.color || CATEGORY_COLORS[d.category] || '#8a8275',
  }));

  if (!data.length) return null;

  return (
    <div className="flex flex-col items-center gap-5">
      <div className="relative h-44 w-44">
        <ResponsiveContainer width="100%" height="100%">
          <PieChart>
            <Pie
              data={data}
              dataKey="value"
              nameKey="name"
              innerRadius={56}
              outerRadius={84}
              paddingAngle={2}
              stroke="hsl(var(--card))"
              strokeWidth={2}
            >
              {data.map((entry, i) => (
                <Cell key={i} fill={entry.color} />
              ))}
            </Pie>
            <Tooltip
              formatter={(value, name) => [`${Number(value).toFixed(1)}%`,
name]}
              contentStyle={{
                background: 'hsl(var(--card))',
                border: '1px solid hsl(var(--border))',
                borderRadius: '0.5rem',
                fontSize: '12px',
              }}
            />
          </PieChart>
        </ResponsiveContainer>
```

```
        <div className="pointer-events-none absolute inset-0 flex flex-col
items-center justify-center">
          <span className="font-display text-3xl font-semibold
text-foreground">
            {data.reduce((s, d) => s + d.value, 0).toFixed(0)}%
          </span>
          <span className="text-[10px] uppercase tracking-[0.18em]
text-muted-foreground">Cover</span>
        </div>
      </div>
      <ul className="grid w-full grid-cols-1 gap-2">
        {data.map((d) => (
          <li key={d.name} className="flex items-center justify-between
text-sm">
            <span className="flex items-center gap-2.5">
              <span className="h-2.5 w-2.5 rounded-full" style={{ background:
d.color }} />
              <span className="text-foreground/80">{d.name}</span>
            </span>
            <span className="font-medium tabular-nums
text-foreground">{d.value.toFixed(1)}%</span>
          </li>
        ))}
      </ul>
    </div>
  );
}
```

LocationSearch.jsx

```
import { useState } from 'react';
import { base44 } from '@/api/base44Client';
import { Search, MapPin, Loader2 } from 'lucide-react';

export default function LocationSearch({ onSelect }) {
  const [query, setQuery] = useState('');
  const [searching, setSearching] = useState(false);
  const [results, setResults] = useState([]);
  const [showResults, setShowResults] = useState(false);
  const [error, setError] = useState(null);

  const handleSearch = async () => {
    const q = query.trim();
    if (q.length < 2) return;
    setSearching(true);
    setError(null);
    try {
```

```
      const res = await base44.functions.invoke('geocodeLocation', { query: q
});
      setResults(res.data?.results || []);
      setShowResults(true);
    } catch (e) {
      setError('Search failed. Try again.');
    } finally {
      setSearching(false);
    }
  };

  const select = (r) => {
    onSelect({ lat: r.lat, lng: r.lng, name: r.display_name });
    setShowResults(false);
    setQuery(r.display_name.split(',')[0]);
  };

  return (
    <div>
      <div className="flex gap-2">
        <div className="relative flex-1">
          <Search className="pointer-events-none absolute left-3 top-1/2 h-4
w-4 -translate-y-1/2 text-muted-foreground" />
          <input
            value={query}
            onChange={(e) => setQuery(e.target.value)}
            onKeyDown={(e) => e.key === 'Enter' && handleSearch()}
            placeholder="Campus, city, address…"
            className="w-full rounded-xl border border-input bg-background
py-2.5 pl-9 pr-3 text-sm text-foreground outline-none transition-colors
focus:border-primary focus:ring-1 focus:ring-primary"
          />
        </div>
        <button
          onClick={handleSearch}
          disabled={searching}
          className="inline-flex items-center justify-center rounded-xl
bg-secondary px-4 text-sm font-medium text-foreground transition-colors
hover:bg-secondary/70 disabled:opacity-50"
        >
          {searching ? <Loader2 className="h-4 w-4 animate-spin" /> :
'Search'}
        </button>
      </div>
      {showResults && results.length > 0 && (
```

```
        <ul className="mt-2 max-h-64 overflow-auto rounded-xl border
border-border bg-background">
          {results.map((r, i) => (
            <li key={i}>
              <button
                onClick={() => select(r)}
                className="flex w-full items-start gap-2.5 border-b
border-border/60 px-3 py-2.5 text-left text-sm transition-colors
last:border-0 hover:bg-muted/60"
              >
                <MapPin className="mt-0.5 h-4 w-4 flex-none text-primary" />
                <span className="text-foreground/85">{r.display_name}</span>
              </button>
            </li>
          ))}
        </ul>
      )}
      {showResults && results.length === 0 && !searching && (
        <p className="mt-2 text-xs text-muted-foreground">No matches found.
Try a more specific name.</p>
      )}
      {error && <p className="mt-2 text-xs text-destructive">{error}</p>}
    </div>
  );
}
```

MapPanel.jsx

```
import { memo, useEffect } from 'react';
import { MapContainer, TileLayer, Polygon, CircleMarker, Marker, useMap,
useMapEvents } from 'react-leaflet';
import 'leaflet/dist/leaflet.css';
import L from 'leaflet';

// Fix default marker icons (Leaflet's bundled URLs break under bundlers)
const DefaultIcon = L.icon({
  iconUrl: 'https://unpkg.com/leaflet@1.9.4/dist/images/marker-icon.png',
  iconRetinaUrl:
'https://unpkg.com/leaflet@1.9.4/dist/images/marker-icon-2x.png',
  shadowUrl: 'https://unpkg.com/leaflet@1.9.4/dist/images/marker-shadow.png',
  iconSize: [25, 41],
  iconAnchor: [12, 41],
  popupAnchor: [1, -34],
});
L.Marker.prototype.options.icon = DefaultIcon;

const STROKE = '#2f6a52';
```

```
const SATELLITE_URL =
'https://server.arcgisonline.com/ArcGIS/rest/services/World_Imagery/MapServer
/tile/{z}/{y}/{x}';
const LABELS_URL =
'https://server.arcgisonline.com/ArcGIS/rest/services/Reference/World_Boundar
ies_and_Places/MapServer/tile/{z}/{y}/{x}';
const STREET_URL = 'https://{s}.tile.openstreetmap.org/{z}/{x}/{y}.png';

function FlyController({ center }) {
  const map = useMap();
  useEffect(() => {
    if (center && !isNaN(center.lat) && !isNaN(center.lng)) {
      map.flyTo([center.lat, center.lng], Math.max(map.getZoom(), 15), {
duration: 0.9 });
    }
  }, [center?.lat, center?.lng]);
  return null;
}

function ClickCapture({ enabled, onClick }) {
  useMapEvents({
    click(e) {
      if (enabled) onClick([e.latlng.lat, e.latlng.lng]);
    },
  });
  return null;
}

function MapPanel({ center, vertices, onVertexClick, drawEnabled, basemap =
'satellite' }) {
  const tileUrl = basemap === 'street' ? STREET_URL : SATELLITE_URL;
  return (
    <MapContainer
      center={[20, 0]}
      zoom={3}
      minZoom={2}
      maxZoom={19}
      className="h-full w-full"
      zoomControl={true}
      scrollWheelZoom={true}
      worldCopyJump
    >
      <TileLayer url={tileUrl} attribution='&copy; Esri, Maxar, Earthstar
Geographics' maxZoom={19} />
```

```
      {basemap === 'satellite' && <TileLayer url={LABELS_URL} maxZoom={19}
/>}
      <FlyController center={center} />
      <ClickCapture enabled={drawEnabled} onClick={onVertexClick} />
      {vertices.length >= 3 && (
        <Polygon positions={vertices} pathOptions={{ color: STROKE, weight:
2.5, fillColor: STROKE, fillOpacity: 0.2 }} />
      )}
      {vertices.length >= 2 && vertices.length < 3 && (
        <Polygon positions={vertices} pathOptions={{ color: STROKE, weight:
2, dashArray: '6 6', fillOpacity: 0 }} />
      )}
      {vertices.map((v, i) => (
        <CircleMarker key={i} center={v} radius={5} pathOptions={{ color:
'#1a201d', fillColor: STROKE, fillOpacity: 1, weight: 2 }} />
      ))}
      {center && <Marker position={[center.lat, center.lng]}
interactive={false} keyboard={false} />}
    </MapContainer>
  );
}

// memoized so typing in the search box / unrelated state changes don't
re-render the map
export default memo(MapPanel);
```

ProtectedRoute.jsx

```
import { useEffect } from 'react';
import { Outlet } from 'react-router-dom';
import { useAuth } from '@/lib/AuthContext';
import UserNotRegisteredError from '@/components/UserNotRegisteredError';

const DefaultFallback = () => (
  <div className="fixed inset-0 flex items-center justify-center">
    <div className="w-8 h-8 border-4 border-slate-200 border-t-slate-800
rounded-full animate-spin"></div>
  </div>
);

export default function ProtectedRoute({ fallback = <DefaultFallback />,
unauthenticatedElement }) {
  const { isAuthenticated, isLoadingAuth, authChecked, authError,
checkUserAuth } = useAuth();

  useEffect(() => {
    if (!authChecked && !isLoadingAuth) {
```

```
      checkUserAuth();
    }
  }, [authChecked, isLoadingAuth, checkUserAuth]);

  if (isLoadingAuth || !authChecked) {
    return fallback;
  }

  if (authError) {
    if (authError.type === 'user_not_registered') {
      return <UserNotRegisteredError />;
    }
    return unauthenticatedElement;
  }

  if (!isAuthenticated) {
    return unauthenticatedElement;
  }

  return <Outlet />;
}
```

ScoreGauge.jsx

```
import { RadialBarChart, RadialBar, PolarAngleAxis, ResponsiveContainer }
from 'recharts';
import { scoreTone } from '@/lib/geoUtils';

export default function ScoreGauge({ score = 0, label = 'Score', invert =
false }) {
  const value = Math.max(0, Math.min(100, Number(score) || 0));
  const display = invert ? 100 - value : value;
  const tone = scoreTone(display);
  const data = [{ name: label, value, fill: tone.color }];

  return (
    <div className="flex flex-col items-center">
      <div className="relative h-36 w-36">
        <ResponsiveContainer width="100%" height="100%">
          <RadialBarChart
            innerRadius="74%"
            outerRadius="100%"
            data={data}
            startAngle={90}
            endAngle={-270}
          >
```

```
            <PolarAngleAxis type="number" domain={[0, 100]} angleAxisId={0}
tick={false} />
            <RadialBar
              background={{ fill: 'hsl(var(--muted))' }}
              dataKey="value"
              cornerRadius={20}
              angleAxisId={0}
            />
          </RadialBarChart>
        </ResponsiveContainer>
        <div className="pointer-events-none absolute inset-0 flex flex-col
items-center justify-center">
          <span className="font-display text-3xl font-semibold" style={{
color: tone.color }}>
            {Math.round(display)}
          </span>
          <span className="text-[10px] uppercase tracking-[0.16em]
text-muted-foreground">{tone.label}</span>
        </div>
      </div>
      <span className="mt-1 text-xs uppercase tracking-[0.16em]
text-muted-foreground">{label}</span>
    </div>
  );
}
```

ScrollToTop.jsx

```
import { useEffect } from "react";
import { useLocation, useNavigationType } from "react-router-dom";

const getHashId = (hash) => {
  const rawId = hash.slice(1);

  try {
    return decodeURIComponent(rawId);
  } catch {
    return rawId;
  }
};

export default function ScrollToTop() {
  const { pathname, hash } = useLocation();
  const navigationType = useNavigationType();

  useEffect(() => {
    if (navigationType === "POP") return;
```

```
    if (hash) {
      const id = getHashId(hash);
      const timer = window.setTimeout(() => {
        document.getElementById(id)?.scrollIntoView({ behavior: "smooth" });
      }, 50);
      return () => window.clearTimeout(timer);
    }

    window.scrollTo({ top: 0, left: 0, behavior: "instant" });
  }, [pathname, hash, navigationType]);

  return null;
}
```

SiteNav.jsx

```
import { useEffect, useState } from 'react';
import { Link, NavLink } from 'react-router-dom';
import { History, LogOut, Menu, X } from 'lucide-react';
import { base44 } from '@/api/base44Client';

function BrandMark() {
  return (
    <span className="flex h-9 w-9 items-center justify-center rounded-xl
bg-primary text-primary-foreground">
      <svg viewBox="0 0 24 24" className="h-5 w-5" fill="none"
stroke="currentColor" strokeWidth="1.6" strokeLinecap="round"
strokeLinejoin="round">
        <path d="M4 20c0-8 5.5-13.5 15-15 0 9.5-5.5 15-14 15" />
        <path d="M4 20c2.5-4.5 6-8 10-10.5" />
      </svg>
    </span>
  );
}

export default function SiteNav() {
  const [user, setUser] = useState(null);
  const [open, setOpen] = useState(false);
  const [menuOpen, setMenuOpen] = useState(false);

  useEffect(() => {
    base44.auth.me().then(setUser).catch(() => {});
  }, []);

  const handleLogout = async () => {
```

```
    await base44.auth.logout();
  };

  const links = [
    { to: '/', label: 'Overview', end: true },
    { to: '/analyze', label: 'Analyze' },
    { to: '/history', label: 'History' },
  ];

  return (
    <header className="sticky top-0 z-40 border-b border-border/60 glass">
      <div className="mx-auto flex h-16 max-w-7xl items-center
justify-between px-4 sm:px-6 lg:px-8">
        <Link to="/" className="flex items-center gap-2.5">
          <BrandMark />
          <span className="font-display text-lg font-semibold tracking-tight
text-foreground">
            GreenScope
          </span>
        </Link>

        <nav className="hidden items-center gap-1 md:flex">
          {links.map((l) => (
            <NavLink
              key={l.to}
              to={l.to}
              end={l.end}
              className={({ isActive }) =>
                `rounded-full px-4 py-2 text-sm font-medium transition-colors
${
                  isActive ? 'bg-secondary text-foreground' :
'text-muted-foreground hover:text-foreground'
                }`
              }
            >
              {l.label}
            </NavLink>
          ))}
        </nav>

        <div className="hidden items-center gap-3 md:flex">
          <Link
            to="/analyze"
```

```
            className="inline-flex items-center gap-2 rounded-full bg-primary
px-5 py-2.5 text-sm font-medium text-primary-foreground transition-all
hover:opacity-90"
          >
            New analysis
          </Link>
          {user && (
            <div className="relative">
              <button
                onClick={() => setMenuOpen((v) => !v)}
                className="flex h-9 w-9 items-center justify-center
rounded-full border border-border bg-card text-sm font-medium
text-foreground"
              >
                {(user.full_name || user.email ||
'?').charAt(0).toUpperCase()}
              </button>
              {menuOpen && (
                <div className="absolute right-0 mt-2 w-56 overflow-hidden
rounded-2xl border border-border bg-card shadow-lg">
                  <div className="border-b border-border px-4 py-3">
                    <p className="truncate text-sm font-medium
text-foreground">{user.full_name || 'User'}</p>
                    <p className="truncate text-xs
text-muted-foreground">{user.email}</p>
                  </div>
                  <button
                    onClick={handleLogout}
                    className="flex w-full items-center gap-2 px-4 py-3
text-sm text-foreground/80 transition-colors hover:bg-muted"
                  >
                    <LogOut className="h-4 w-4" /> Sign out
                  </button>
                </div>
              )}
            </div>
          )}
        </div>

        <button onClick={() => setOpen((v) => !v)} className="md:hidden">
          {open ? <X className="h-6 w-6" /> : <Menu className="h-6 w-6" />}
        </button>
      </div>

      {open && (
```

```
        <div className="border-t border-border bg-card md:hidden">
          <div className="space-y-1 px-4 py-3">
            {links.map((l) => (
              <NavLink
                key={l.to}
                to={l.to}
                end={l.end}
                onClick={() => setOpen(false)}
                className={({ isActive }) =>
                  `block rounded-xl px-4 py-3 text-sm font-medium ${
                    isActive ? 'bg-secondary text-foreground' :
'text-muted-foreground'
                  }`
                }
              >
                {l.label}
              </NavLink>
            ))}
            <Link
              to="/analyze"
              onClick={() => setOpen(false)}
              className="mt-2 block rounded-xl bg-primary px-4 py-3
text-center text-sm font-medium text-primary-foreground"
            >
              New analysis
            </Link>
            <button
              onClick={handleLogout}
              className="mt-1 flex w-full items-center gap-2 rounded-xl px-4
py-3 text-sm text-muted-foreground"
            >
              <LogOut className="h-4 w-4" /> Sign out
            </button>
          </div>
        </div>
      )}
    </header>
  );
}
```

StatCard.jsx

```
import { Leaf, Building2, Route as Road, Map, Droplets, TreePine, CloudOff,
Sprout } from 'lucide-react';
import { formatNumber } from '@/lib/geoUtils';

const ICONS = {
```

```
  vegetation: Leaf,
  buildings: Building2,
  roads: Road,
  open_land: Map,
  water: Droplets,
  trees: TreePine,
  carbon_impact: CloudOff,
  absorption: Sprout,
};

export default function StatCard({ type, label, value, suffix, accent =
'#5f7d56', hint }) {
  const Icon = ICONS[type] || Leaf;
  return (
    <div className="group relative overflow-hidden rounded-2xl border
border-border bg-card p-5 transition-all
hover:shadow-[0_8px_30px_-12px_rgba(31,27,22,0.18)]">
      <div
        className="absolute -right-6 -top-6 h-20 w-20 rounded-full
opacity-[0.07] transition-opacity group-hover:opacity-[0.12]"
        style={{ background: accent }}
      />
      <div className="flex items-start justify-between">
        <div
          className="flex h-10 w-10 items-center justify-center rounded-xl"
          style={{ background: `${accent}1a`, color: accent }}
        >
          <Icon className="h-5 w-5" strokeWidth={1.6} />
        </div>
      </div>
      <div className="mt-4">
        <div className="flex items-baseline gap-1">
          <span className="font-display text-3xl font-semibold
text-foreground tabular-nums">
            {value}
          </span>
          {suffix && <span className="text-sm
text-muted-foreground">{suffix}</span>}
        </div>
        <p className="mt-1 text-sm font-medium
text-foreground/80">{label}</p>
        {hint && <p className="mt-0.5 text-xs
text-muted-foreground">{hint}</p>}
      </div>
    </div>
```

```
  );
}
```

UserNotRegisteredError.jsx

```
import React from 'react';

const UserNotRegisteredError = () => {
  return (
    <div className="flex flex-col items-center justify-center min-h-screen bg-gradient-to-b from-white to-slate-50">
      <div className="max-w-md w-full p-8 bg-white rounded-lg shadow-lg border border-slate-100">
        <div className="text-center">
          <div className="inline-flex items-center justify-center w-16 h-16 mb-6 rounded-full bg-orange-100">
            <svg className="w-8 h-8 text-orange-600" fill="none" stroke="currentColor" viewBox="0 0 24 24">
              <path strokeLinecap="round" strokeLinejoin="round" strokeWidth="2" d="M12 9v2m0 4h.01m-6.938 4h13.856c1.54 0 2.502-1.667 1.732-3L13.732 4c-.77-1.333-2.694-1.333-3.464 0L3.34 16c-.77 1.333.192 3 1.732 3z" />
            </svg>
          </div>
          <h1 className="text-3xl font-bold text-slate-900 mb-4">Access Restricted</h1>
          <p className="text-slate-600 mb-8">
            You are not registered to use this application. Please contact the app administrator to request access.
          </p>
          <div className="p-4 bg-slate-50 rounded-md text-sm text-slate-600">
            <p>If you believe this is an error, you can:</p>
            <ul className="list-disc list-inside mt-2 space-y-1">
              <li>Verify you are logged in with the correct account</li>
              <li>Contact the app administrator for access</li>
              <li>Try logging out and back in again</li>
            </ul>
          </div>
        </div>
      </div>
    </div>
  );
};

export default UserNotRegisteredError;
```

/hooks

use-mobile.jsx

```
import * as React from "react"

const MOBILE_BREAKPOINT = 768

export function useIsMobile() {
  const [isMobile, setIsMobile] = React.useState(undefined)

  React.useEffect(() => {
    const mql = window.matchMedia(`(max-width: ${MOBILE_BREAKPOINT - 1}px)`)
    const onChange = () => {
      setIsMobile(window.innerWidth < MOBILE_BREAKPOINT)
    }
    mql.addEventListener("change", onChange)
    setIsMobile(window.innerWidth < MOBILE_BREAKPOINT)
    return () => mql.removeEventListener("change", onChange);
  }, [])

  return !!isMobile
}
```

use-size.jsx

```
import * as React from "react"

export function useSize(ref) {
  const [size, setSize] = React.useState(null)

  // useLayoutEffect (not useEffect): the initial measurement must land
before
  // the browser paints, so consumers can render their real content on the
  // very first painted frame instead of a guess. A ResizeObserver's first
  // callback arrives too late for that — by then an <img> src guess has
  // already been dispatched to the network.
  React.useLayoutEffect(() => {
    const element = ref.current
    if (!element) return

    const rect = element.getBoundingClientRect()
    setSize({ width: rect.width, height: rect.height })

    const observer = new ResizeObserver(([entry]) => {
      const { width, height } = entry.contentRect
      setSize({ width, height })
```

```
    })

    observer.observe(element)
    return () => observer.disconnect()
  }, [ref])

  return size
}
```

/lib

app-params.js

```
import { getAccessToken } from '@base44/sdk';

const isNode = typeof window === 'undefined';

const isClearAccessTokenRequested = () =>
    !isNode && new
URLSearchParams(window.location.search).get("clear_access_token") === 'true';

const clearStoredAccessToken = () => {
    window.localStorage.removeItem('base44_access_token');
    window.localStorage.removeItem('token');
}

const getAppParams = () => {
    if (isClearAccessTokenRequested()) {
        clearStoredAccessToken();
    }
    return {
        appId: import.meta.env.VITE_BASE44_APP_ID,
        token: getAccessToken(),
        functionsVersion: import.meta.env.VITE_BASE44_FUNCTIONS_VERSION,
        appBaseUrl: import.meta.env.VITE_BASE44_APP_BASE_URL,
    }
}


export const appParams = {
    ...getAppParams()
}
```

AuthContext.jsx

```
import React, { createContext, useState, useContext, useEffect } from
'react';
```

```
import { base44 } from '@/api/base44Client';
import { appParams } from '@/lib/app-params';

const AuthContext = createContext();

export const AuthProvider = ({ children }) => {
  const [user, setUser] = useState(null);
  const [isAuthenticated, setIsAuthenticated] = useState(false);
  const [isLoadingAuth, setIsLoadingAuth] = useState(true);
  const [isLoadingPublicSettings, setIsLoadingPublicSettings] =
useState(true);
  const [authError, setAuthError] = useState(null);
  const [authChecked, setAuthChecked] = useState(false);
  const [appPublicSettings, setAppPublicSettings] = useState(null); //
Contains only { id, public_settings }

  useEffect(() => {
    checkAppState();
  }, []);

  const checkAppState = async () => {
    try {
      setIsLoadingPublicSettings(true);
      setAuthError(null);

      try {
        const publicSettings = await base44.app.getPublicSettings();
        setAppPublicSettings(publicSettings);

        // If we got the app public settings successfully, check if user is
authenticated
        if (appParams.token) {
          await checkUserAuth();
        } else {
          setIsLoadingAuth(false);
          setIsAuthenticated(false);
          setAuthChecked(true);
        }
        setIsLoadingPublicSettings(false);
      } catch (appError) {
        console.error('App state check failed:', appError);

        // Handle app-level errors
        if (appError.status === 403 && appError.data?.extra_data?.reason) {
          const reason = appError.data.extra_data.reason;
```

```
        if (reason === 'auth_required') {
          setAuthError({
            type: 'auth_required',
            message: 'Authentication required'
          });
        } else if (reason === 'user_not_registered') {
          setAuthError({
            type: 'user_not_registered',
            message: 'User not registered for this app'
          });
        } else {
          setAuthError({
            type: reason,
            message: appError.message
          });
        }
      } else {
        setAuthError({
          type: 'unknown',
          message: appError.message || 'Failed to load app'
        });
      }
      setIsLoadingPublicSettings(false);
      setIsLoadingAuth(false);
    }
  } catch (error) {
    console.error('Unexpected error:', error);
    setAuthError({
      type: 'unknown',
      message: error.message || 'An unexpected error occurred'
    });
    setIsLoadingPublicSettings(false);
    setIsLoadingAuth(false);
  }
};

const checkUserAuth = async () => {
  try {
    // Now check if the user is authenticated
    setIsLoadingAuth(true);
    const currentUser = await base44.auth.me();
    setUser(currentUser);
    setIsAuthenticated(true);
    setIsLoadingAuth(false);
    setAuthChecked(true);
```

```
    } catch (error) {
      console.error('User auth check failed:', error);
      setIsLoadingAuth(false);
      setIsAuthenticated(false);
      setAuthChecked(true);

      // If user auth fails, it might be an expired token
      if (error.status === 401 || error.status === 403) {
        setAuthError({
          type: 'auth_required',
          message: 'Authentication required'
        });
      }
    }
  };

  const logout = (shouldRedirect = true) => {
    setUser(null);
    setIsAuthenticated(false);

    if (shouldRedirect) {
      // Use the SDK's logout method which handles token cleanup and redirect
      base44.auth.logout(window.location.href);
    } else {
      // Just remove the token without redirect
      base44.auth.logout();
    }
  };

  const navigateToLogin = () => {
    // Use the SDK's redirectToLogin method
    base44.auth.redirectToLogin(window.location.href);
  };

  return (
    <AuthContext.Provider value={{
      user,
      isAuthenticated,
      isLoadingAuth,
      isLoadingPublicSettings,
      authError,
      appPublicSettings,
      authChecked,
      logout,
      navigateToLogin,
```

```
      checkUserAuth,
      checkAppState
    }}>
      {children}
    </AuthContext.Provider>
  );
};

export const useAuth = () => {
  const context = useContext(AuthContext);
  if (!context) {
    throw new Error('useAuth must be used within an AuthProvider');
  }
  return context;
};
```

authReturnTo.js

```
// Shared by the auth pages (Login, Register, and any page that resumes a
flow
// after sign-in, e.g. the MCP OAuth consent page). Keep the redirect
// validation in one place — it is security-sensitive and easy to drift.

// Resolve ?returnTo= to a safe same-origin path, else "/".
//
// The same-origin check alone is not enough: a value like /.//evil.com or
// /\evil.com parses same-origin but normalizes to a protocol-relative
// //evil.com when assigned to location.href — an open redirect. So require
the
// resolved path to be exactly one leading slash (no "//" prefix, no
backslash).
export function safeReturnTo() {
  const raw = new URLSearchParams(window.location.search).get("returnTo");
  if (!raw) return "/";
  try {
    const url = new URL(raw, window.location.origin);
    if (url.origin !== window.location.origin) return "/";
    // Only access_token/clear_access_token are still URL-read by
app-params.js, but the
    // whole bootstrap set stays stripped: one going back to a URL read must
not silently
    // become injectable again. Normal app-flow params (e.g. the OAuth
consent ctx) are kept.
    for (const p of ["access_token", "clear_access_token", "app_id",
"app_base_url", "functions_version", "from_url"]) {
      url.searchParams.delete(p);
```

```
    }
    const path = url.pathname + url.search;
    if (!path.startsWith("/") || path.startsWith("//") ||
path.includes("\\")) return "/";
    return path;
  } catch {
    return "/";
  }
}
```

geoUtils.js

```
// Geospatial helpers for GreenScope AI

// Spherical polygon area in hectares using an equirectangular projection.
// latlngs: array of [lat, lng] pairs. Returns 0 for fewer than 3 vertices.
export function polygonAreaHectares(latlngs) {
  if (!latlngs || latlngs.length < 3) return 0;
  const R = 6378137; // earth radius (m)
  const pts = latlngs.map(([lat, lng]) => {
    const latRad = (lat * Math.PI) / 180;
    const x = ((lng * Math.PI) / 180) * R * Math.cos(latRad);
    const y = latRad * R;
    return [x, y];
  });
  let area = 0;
  for (let i = 0; i < pts.length; i++) {
    const [x1, y1] = pts[i];
    const [x2, y2] = pts[(i + 1) % pts.length];
    area += x1 * y2 - x2 * y1;
  }
  return Math.abs(area) / 2 / 10000; // m^2 -> hectares
}

export function formatNumber(n, digits = 0) {
  if (n == null || isNaN(n)) return '—';
  return Number(n).toLocaleString(undefined, {
    maximumFractionDigits: digits,
    minimumFractionDigits: 0,
  });
}

export function formatArea(ha) {
  if (ha == null || isNaN(ha)) return '—';
  if (ha < 1) return `${(ha * 10000).toFixed(0)} m²`;
  if (ha < 100) return `${ha.toFixed(2)} ha`;
```

```
  return `${ha.toFixed(0)} ha`;
}

// Default chart colors per land-cover category
export const CATEGORY_COLORS = {
  Vegetation: '#5f7d56',
  Buildings: '#a86a3e',
  Roads: '#3a3530',
  'Open Land': '#cdbb93',
  Water: '#4f8a93',
};

export function ratingTone(rating) {
  switch (rating) {
    case 'Excellent': return { bg: 'bg-land-vegetation/15', text:
'text-land-vegetation', dot: '#5f7d56' };
    case 'Good': return { bg: 'bg-land-vegetation/10', text:
'text-land-vegetation', dot: '#6f8a64' };
    case 'Moderate': return { bg: 'bg-amber-400/15', text: 'text-amber-700',
dot: '#c8924a' };
    case 'Low': return { bg: 'bg-orange-400/15', text: 'text-orange-700',
dot: '#b56b3a' };
    case 'Poor': return { bg: 'bg-red-400/15', text: 'text-red-700', dot:
'#a64a3a' };
    default: return { bg: 'bg-muted', text: 'text-muted-foreground', dot:
'#8a8275' };
  }
}

export function scoreTone(score) {
  if (score >= 75) return { label: 'Strong', color: '#5f7d56' };
  if (score >= 55) return { label: 'Moderate', color: '#c8924a' };
  if (score >= 35) return { label: 'Limited', color: '#b56b3a' };
  return { label: 'Critical', color: '#a64a3a' };
}
```

PageNotFound.jsx

```
import { useLocation } from 'react-router-dom';
import { base44 } from '@/api/base44Client';
import { useQuery } from '@tanstack/react-query';


export default function PageNotFound({}) {
    const location = useLocation();
    const pageName = location.pathname.substring(1);
```

```
    const { data: authData, isFetched } = useQuery({
        queryKey: ['user'],
        queryFn: async () => {
            try {
                const user = await base44.auth.me();
                return { user, isAuthenticated: true };
            } catch (error) {
                return { user: null, isAuthenticated: false };
            }
        }
    });

    return (
        <div className="min-h-screen flex items-center justify-center p-6
bg-slate-50">
            <div className="max-w-md w-full">
                <div className="text-center space-y-6">
                    {/* 404 Error Code */}
                    <div className="space-y-2">
                        <h1 className="text-7xl font-light
text-slate-300">404</h1>
                        <div className="h-0.5 w-16 bg-slate-200
mx-auto"></div>
                    </div>

                    {/* Main Message */}
                    <div className="space-y-3">
                        <h2 className="text-2xl font-medium text-slate-800">
                            Page Not Found
                        </h2>
                        <p className="text-slate-600 leading-relaxed">
                            The page <span className="font-medium
text-slate-700">"{pageName}"</span> could not be found in this application.
                        </p>
                    </div>

                    {/* Admin Note */}
                    {isFetched && authData.isAuthenticated &&
authData.user?.role === 'admin' && (
                        <div className="mt-8 p-4 bg-slate-100 rounded-lg
border border-slate-200">
                            <div className="flex items-start space-x-3">
                                <div className="flex-shrink-0 w-5 h-5
rounded-full bg-orange-100 flex items-center justify-center mt-0.5">
```

```
                                    <div className="w-2 h-2 rounded-full
bg-orange-400"></div>
                                </div>
                                <div className="text-left space-y-1">
                                    <p className="text-sm font-medium
text-slate-700">Admin Note</p>
                                    <p className="text-sm text-slate-600
leading-relaxed">
                                        This could mean that the AI hasn't
implemented this page yet. Ask it to implement it in the chat.
                                    </p>
                                </div>
                            </div>
                        </div>
                    )}

                    {/* Action Button */}
                    <div className="pt-6">
                        <button
                            onClick={() => window.location.href = '/'}
                            className="inline-flex items-center px-4 py-2
text-sm font-medium text-slate-700 bg-white border border-slate-200
rounded-lg hover:bg-slate-50 hover:border-slate-300 transition-colors
duration-200 focus:outline-none focus:ring-2 focus:ring-offset-2
focus:ring-slate-500"
                        >
                            <svg className="w-4 h-4 mr-2" fill="none"
stroke="currentColor" viewBox="0 0 24 24">
                                <path strokeLinecap="round"
strokeLinejoin="round" strokeWidth={2} d="M3 12l2-2m0 0l7-7 7 7M5 10v10a1 1 0
001 1h3m10-11l2 2m-2-2v10a1 1 0 01-1 1h-3m-6 0a1 1 0 001-1v-4a1 1 0 011-1h2a1
1 0 011 1v4a1 1 0 001 1m-6 0h6" />
                            </svg>
                            Go Home
                        </button>
                    </div>
                </div>
            </div>
        </div>
    )
}
```

query-client.js

```
import { QueryClient } from '@tanstack/react-query';
```

```
export const queryClientInstance = new QueryClient({
    defaultOptions: {
        queries: {
            refetchOnWindowFocus: false,
            retry: 1,
        },
    },
});
```

utils.js

```
import { clsx } from "clsx"
import { twMerge } from "tailwind-merge"

export function cn(...inputs) {
  return twMerge(clsx(inputs))
}


export const isIframe = window.self !== window.top;
```

/pages

AnalysisDetail.jsx

```
import { useEffect, useState } from 'react';
import { useParams, Link } from 'react-router-dom';
import { base44 } from '@/api/base44Client';
import MapPanel from '@/components/MapPanel';
import AnalysisDashboard from '@/components/AnalysisDashboard';
import { Loader2, ArrowLeft, Trash2 } from 'lucide-react';

export default function AnalysisDetail() {
  const { id } = useParams();
  const [record, setRecord] = useState(null);
  const [loading, setLoading] = useState(true);
  const [notFound, setNotFound] = useState(false);
  const [deleting, setDeleting] = useState(false);

  useEffect(() => {
    let active = true;
    (async () => {
      try {
        const r = await base44.entities.Analysis.get(id);
        if (active) setRecord(r);
      } catch (e) {
        if (active) setNotFound(true);
      } finally {
```

```
        if (active) setLoading(false);
      }
    })();
    return () => { active = false; };
  }, [id]);

  const handleDelete = async () => {
    if (!record) return;
    setDeleting(true);
    try {
      await base44.entities.Analysis.delete(record.id);
      window.location.href = '/history';
    } catch (e) {
      setDeleting(false);
    }
  };

  if (loading) {
    return (
      <div className="flex h-[60vh] items-center justify-center">
        <Loader2 className="h-7 w-7 animate-spin text-primary" />
      </div>
    );
  }
  if (notFound || !record) {
    return (
      <div className="mx-auto max-w-2xl px-4 py-24 text-center">
        <p className="font-display text-2xl font-semibold
text-foreground">Report not found</p>
        <p className="mt-2 text-muted-foreground">It may have been deleted or
doesn't belong to you.</p>
        <Link to="/history" className="mt-6 inline-flex items-center gap-2
rounded-full bg-primary px-5 py-2.5 text-sm font-medium
text-primary-foreground">
          <ArrowLeft className="h-4 w-4" /> Back to history
        </Link>
      </div>
    );
  }

  const center = { lat: record.lat, lng: record.lng };
  const boundary = Array.isArray(record.boundary) ? record.boundary : [];

  return (
    <div className="mx-auto max-w-7xl px-4 py-10 sm:px-6 lg:px-8">
```

```
      <div className="mb-6 flex items-center justify-between">
        <Link to="/history" className="inline-flex items-center gap-2 text-sm
text-muted-foreground transition-colors hover:text-foreground">
          <ArrowLeft className="h-4 w-4" /> History
        </Link>
        <button
          onClick={handleDelete}
          disabled={deleting}
          className="inline-flex items-center gap-2 rounded-full border
border-border bg-card px-4 py-2 text-sm font-medium text-foreground
transition-colors hover:bg-secondary disabled:opacity-50"
        >
          <Trash2 className="h-4 w-4" /> Delete
        </button>
      </div>

      <div className="mb-8 h-[40vh] min-h-[300px] overflow-hidden rounded-3xl
border border-border">
        <MapPanel center={center} vertices={boundary} drawEnabled={false}
basemap="satellite" />
      </div>

      <AnalysisDashboard data={record} />
    </div>
  );
}
```

Analyze.jsx

```
import { useCallback, useState } from 'react';
import { base44 } from '@/api/base44Client';
import MapPanel from '@/components/MapPanel';
import LocationSearch from '@/components/LocationSearch';
import AnalysisDashboard from '@/components/AnalysisDashboard';
import { polygonAreaHectares, formatArea } from '@/lib/geoUtils';
import { PenLine, Undo2, Trash2, Satellite, Route as RoadIcon, Loader2, Leaf,
AlertCircle, Check } from 'lucide-react';

export default function Analyze() {
  const [location, setLocation] = useState(null);
  const [vertices, setVertices] = useState([]);
  const [drawMode, setDrawMode] = useState(false);
  const [basemap, setBasemap] = useState('satellite');

  const [analyzing, setAnalyzing] = useState(false);
  const [result, setResult] = useState(null);
  const [error, setError] = useState(null);
```

```
const [saving, setSaving] = useState(false);
const [saved, setSaved] = useState(false);
const [savedId, setSavedId] = useState(null);

const area = polygonAreaHectares(vertices);

const handleSelectLocation = useCallback((loc) => {
  setLocation(loc);
  setVertices([]);
  setResult(null);
  setSaved(false);
  setSavedId(null);
  setDrawMode(true);
}, []);

const addVertex = useCallback((v) => {
  setVertices((prev) => [...prev, v]);
  setResult(null);
  setSaved(false);
  setSavedId(null);
}, []);

const undoVertex = () => setVertices((prev) => prev.slice(0, -1));
const clearVertices = () => {
  setVertices([]);
  setResult(null);
  setSaved(false);
};

const handleAnalyze = async () => {
  if (vertices.length < 3 || !location) return;
  setAnalyzing(true);
  setError(null);
  setResult(null);
  try {
    const res = await base44.functions.invoke('analyzeArea', {
      location_name: location.name,
      lat: location.lat,
      lng: location.lng,
      area_hectares: area,
    });
    if (res.data?.error) throw new Error(res.data.error);
    const analysis = res.data?.analysis;
    if (!analysis) throw new Error('No analysis returned.');
```

```
      setResult(analysis);
      setSaved(false);
      setSavedId(null);
    } catch (e) {
      setError(e.message || 'Analysis failed.');
    } finally {
      setAnalyzing(false);
    }
  };

  const handleSave = async () => {
    if (!result) return;
    setSaving(true);
    try {
      const record = await base44.entities.Analysis.create({
        location_name: location.name,
        lat: location.lat,
        lng: location.lng,
        area_hectares: area,
        boundary: vertices,
        vegetation_pct: result.vegetation_pct,
        buildings_pct: result.buildings_pct,
        roads_pct: result.roads_pct,
        open_land_pct: result.open_land_pct,
        water_pct: result.water_pct,
        carbon_impact_score: result.carbon_impact_score,
        carbon_absorption_tons_year: result.carbon_absorption_tons_year,
        tree_canopy_estimate: result.tree_canopy_estimate,
        sustainability_score: result.sustainability_score,
        green_cover_rating: result.green_cover_rating,
        summary: result.summary,
        recommendations: result.recommendations,
        land_use_breakdown: result.land_use_breakdown,
      });
      setSavedId(record.id);
      setSaved(true);
    } catch (e) {
      setError('Could not save the report.');
    } finally {
      setSaving(false);
    }
  };

  const canAnalyze = vertices.length >= 3 && location && !analyzing;
```

```
  return (
    <div className="mx-auto max-w-7xl px-4 py-10 sm:px-6 lg:px-8">
      {/* Header */}
      <div className="mb-8">
        <p className="text-xs font-medium uppercase tracking-[0.2em]
text-primary">Analysis workspace</p>
        <h1 className="mt-2 font-display text-3xl font-semibold
tracking-tight text-foreground sm:text-4xl">
          Analyze an area
        </h1>
        <p className="mt-2 max-w-2xl text-muted-foreground">
          Search a location, draw its boundary on the satellite map, and run
an AI land-cover and carbon assessment.
        </p>
      </div>

      <div className="grid grid-cols-1 gap-6 lg:grid-cols-[380px_1fr]">
        {/* Control panel */}
        <div className="space-y-4">
          {/* Search */}
          <div className="rounded-2xl border border-border bg-card p-5">
            <label className="text-xs font-medium uppercase tracking-[0.16em]
text-muted-foreground">
              1 · Find a location
            </label>
            <div className="mt-3">
              <LocationSearch onSelect={handleSelectLocation} />
            </div>
          </div>

          {/* Draw tools */}
          <div className="rounded-2xl border border-border bg-card p-5">
            <label className="text-xs font-medium uppercase tracking-[0.16em]
text-muted-foreground">
              2 · Draw the boundary
            </label>
            <p className="mt-2 text-xs text-muted-foreground">
              {location ? 'Click on the map to drop vertices. At least 3
close the area.' : 'Search for a location first to begin drawing.'}
            </p>
            <div className="mt-3 flex flex-wrap gap-2">
              <button
                onClick={() => setDrawMode((v) => !v)}
                disabled={!location}
```

```
                className={`inline-flex items-center gap-2 rounded-xl px-3.5
py-2 text-sm font-medium transition-colors disabled:opacity-40 ${
                  drawMode ? 'bg-primary text-primary-foreground' :
'bg-secondary text-foreground hover:bg-secondary/70'
                }`}
              >
                <PenLine className="h-4 w-4" />
                {drawMode ? 'Drawing' : 'Draw'}
              </button>
              <button onClick={undoVertex} disabled={vertices.length === 0}
className="inline-flex items-center gap-2 rounded-xl bg-secondary px-3.5 py-2
text-sm font-medium text-foreground transition-colors hover:bg-secondary/70
disabled:opacity-40">
                <Undo2 className="h-4 w-4" /> Undo
              </button>
              <button onClick={clearVertices} disabled={vertices.length ===
0} className="inline-flex items-center gap-2 rounded-xl bg-secondary px-3.5
py-2 text-sm font-medium text-foreground transition-colors
hover:bg-secondary/70 disabled:opacity-40">
                <Trash2 className="h-4 w-4" /> Clear
              </button>
            </div>

            <div className="mt-4 grid grid-cols-2 gap-3">
              <div className="rounded-xl bg-muted/50 px-4 py-3">
                <p className="text-xs uppercase tracking-[0.14em]
text-muted-foreground">Area</p>
                <p className="mt-1 font-display text-lg font-semibold
text-foreground">{formatArea(area)}</p>
              </div>
              <div className="rounded-xl bg-muted/50 px-4 py-3">
                <p className="text-xs uppercase tracking-[0.14em]
text-muted-foreground">Vertices</p>
                <p className="mt-1 font-display text-lg font-semibold
text-foreground">{vertices.length}</p>
              </div>
            </div>

            <div className="mt-3 flex gap-2">
              <button
                onClick={() => setBasemap('satellite')}
                className={`inline-flex flex-1 items-center justify-center
gap-2 rounded-xl px-3 py-2 text-xs font-medium transition-colors ${
```

```
                  basemap === 'satellite' ? 'bg-primary
text-primary-foreground' : 'bg-secondary text-foreground
hover:bg-secondary/70'
                }`}
              >
                <Satellite className="h-3.5 w-3.5" /> Satellite
              </button>
              <button
                onClick={() => setBasemap('street')}
                className={`inline-flex flex-1 items-center justify-center
gap-2 rounded-xl px-3 py-2 text-xs font-medium transition-colors ${
                  basemap === 'street' ? 'bg-primary text-primary-foreground'
: 'bg-secondary text-foreground hover:bg-secondary/70'
                }`}
              >
                <RoadIcon className="h-3.5 w-3.5" /> Street
              </button>
            </div>
          </div>

          {/* Analyze */}
          <div className="rounded-2xl border border-border bg-gradient-to-br
from-primary/10 via-card to-card p-5">
            <label className="text-xs font-medium uppercase tracking-[0.16em]
text-muted-foreground">
              3 · Run assessment
            </label>
            <p className="mt-2 text-xs text-muted-foreground">
              {canAnalyze ? 'Ready to analyze.' : vertices.length < 3 ? 'Draw
at least 3 vertices to define an area.' : 'Select a location first.'}
            </p>
            <button
              onClick={handleAnalyze}
              disabled={!canAnalyze}
              className="mt-3 inline-flex w-full items-center justify-center
gap-2 rounded-xl bg-primary px-5 py-3 text-sm font-medium
text-primary-foreground transition-all hover:opacity-90 disabled:opacity-40"
            >
              {analyzing ? <Loader2 className="h-4 w-4 animate-spin" /> :
<Leaf className="h-4 w-4" />}
              {analyzing ? 'Analyzing…' : 'Analyze area'}
            </button>
          </div>
        </div>
```

```
        {/* Map */}
        <div className="relative">
          <div className="h-[58vh] min-h-[420px] overflow-hidden rounded-3xl
border border-border shadow-[0_20px_60px_-30px_rgba(26,32,29,0.35)]">
            <MapPanel
              center={location}
              vertices={vertices}
              onVertexClick={addVertex}
              drawEnabled={drawMode}
              basemap={basemap}
            />
          </div>
          {drawMode && location && (
            <div className="pointer-events-none absolute left-4 top-4
rounded-full glass px-3.5 py-1.5 text-xs font-medium text-foreground">
              Click the map to add vertices
            </div>
          )}
        </div>
      </div>

      {/* Error */}
      {error && (
        <div className="mt-6 flex items-center gap-3 rounded-2xl border
border-destructive/30 bg-destructive/5 px-5 py-4 text-sm text-destructive">
          <AlertCircle className="h-5 w-5" />
          {error}
        </div>
      )}

      {/* Loading */}
      {analyzing && (
        <div className="mt-8 grid grid-cols-1 gap-6 lg:grid-cols-4">
          {[...Array(4)].map((_, i) => (
            <div key={i} className="h-40 animate-pulse rounded-2xl border
border-border bg-muted/40" />
          ))}
          <div className="lg:col-span-4 h-64 animate-pulse rounded-3xl border
border-border bg-muted/40" />
        </div>
      )}

      {/* Result */}
      {result && location && (
        <div className="mt-10 animate-fade-up">
```

```
          <AnalysisDashboard
            data={{ ...result, location_name: location.name, area_hectares:
area, lat: location.lat, lng: location.lng }}
            onSave={handleSave}
            saving={saving}
            saved={saved}
          />
          {saved && savedId && (
            <div className="mt-4 flex items-center gap-2 rounded-2xl border
border-land-vegetation/30 bg-land-vegetation/10 px-5 py-3 text-sm
text-land-vegetation">
              <Check className="h-4 w-4" />
              Report saved. <a href={`/analysis/${savedId}`}
className="underline">View it</a>.
            </div>
          )}
        </div>
      )}
    </div>
  );
}
```

ForgotPassword.jsx

```
import React, { useState } from "react";
import { Link } from "react-router-dom";
import { base44 } from "@/api/base44Client";
import { Button } from "@/components/ui/button";
import { Input } from "@/components/ui/input";
import { Label } from "@/components/ui/label";
import { Mail, ArrowLeft, Loader2 } from "lucide-react";
import AuthLayout from "@/components/AuthLayout";

export default function ForgotPassword() {
  const [email, setEmail] = useState("");
  const [loading, setLoading] = useState(false);
  const [sent, setSent] = useState(false);

  const handleSubmit = async (e) => {
    e.preventDefault();
    setLoading(true);
    try {
      await base44.auth.resetPasswordRequest(email);
    } catch {
      // Always show success regardless
    } finally {
      setLoading(false);
```

```
      setSent(true);
    }
  };

  return (
    <AuthLayout
      icon={Mail}
      title="Reset password"
      subtitle="We'll send you a link to reset it"
      footer={
        <Link to="/login" className="text-primary font-medium
hover:underline">
          <ArrowLeft className="w-3 h-3 inline mr-1" />Back to log in
        </Link>
      }
    >
      {sent ? (
        <p className="text-sm text-foreground text-center">
          If an account exists with that email, you'll receive a password
reset link shortly.
        </p>
      ) : (
        <form onSubmit={handleSubmit} className="space-y-4">
          <div className="space-y-2">
            <Label htmlFor="email">Email address</Label>
            <div className="relative">
              <Mail className="absolute left-3 top-1/2 -translate-y-1/2 w-4
h-4 text-muted-foreground" aria-hidden="true" />
              <Input
                id="email"
                type="email"
                autoComplete="email"
                autoFocus
                placeholder="you@example.com"
                value={email}
                onChange={(e) => setEmail(e.target.value)}
                className="pl-10 h-12"
                required
              />
            </div>
          </div>
          <Button type="submit" className="w-full h-12 font-medium"
disabled={loading}>
            {loading ? (
              <>
```

```
                <Loader2 className="w-4 h-4 mr-2 animate-spin" />
                Sending...
              </>
            ) : (
              "Send reset link"
            )}
          </Button>
        </form>
      )}
    </AuthLayout>
  );
}
```

History.jsx

```
import { useEffect, useState } from 'react';
import { Link } from 'react-router-dom';
import { base44 } from '@/api/base44Client';
import { formatArea, formatNumber, ratingTone } from '@/lib/geoUtils';
import { Loader2, Leaf, ArrowRight, MapPin } from 'lucide-react';

export default function History() {
  const [items, setItems] = useState([]);
  const [loading, setLoading] = useState(true);

  useEffect(() => {
    let active = true;
    (async () => {
      try {
        const page = await base44.entities.Analysis.filter({}, { sort:
'-created_date', limit: 50 });
        if (active) setItems(page.items || []);
      } catch (e) {
        // ignore
      } finally {
        if (active) setLoading(false);
      }
    })();
    return () => { active = false; };
  }, []);

  return (
    <div className="mx-auto max-w-7xl px-4 py-10 sm:px-6 lg:px-8">
      <div className="mb-8 flex flex-col gap-4 sm:flex-row sm:items-end
sm:justify-between">
        <div>
```

```
          <p className="text-xs font-medium uppercase tracking-[0.2em]
text-primary">Your library</p>
          <h1 className="mt-2 font-display text-3xl font-semibold
tracking-tight text-foreground sm:text-4xl">
            Saved reports
          </h1>
          <p className="mt-2 text-muted-foreground">Revisit and compare your
past environmental analyses.</p>
        </div>
        <Link to="/analyze" className="inline-flex items-center gap-2
rounded-full bg-primary px-5 py-2.5 text-sm font-medium
text-primary-foreground transition-all hover:opacity-90">
          <Leaf className="h-4 w-4" /> New analysis
        </Link>
      </div>

      {loading ? (
        <div className="flex h-64 items-center justify-center">
          <Loader2 className="h-7 w-7 animate-spin text-primary" />
        </div>
      ) : items.length === 0 ? (
        <div className="rounded-3xl border border-dashed border-border
bg-card/50 p-16 text-center">
          <span className="mx-auto flex h-14 w-14 items-center justify-center
rounded-2xl bg-primary/10 text-primary">
            <MapPin className="h-7 w-7" />
          </span>
          <p className="mt-5 font-display text-xl font-semibold
text-foreground">No reports yet</p>
          <p className="mt-2 text-sm text-muted-foreground">Run your first
area analysis to see it saved here.</p>
          <Link to="/analyze" className="mt-6 inline-flex items-center gap-2
rounded-full bg-primary px-5 py-2.5 text-sm font-medium
text-primary-foreground">
            Analyze an area <ArrowRight className="h-4 w-4" />
          </Link>
        </div>
      ) : (
        <div className="grid grid-cols-1 gap-5 sm:grid-cols-2
lg:grid-cols-3">
          {items.map((it) => {
            const tone = ratingTone(it.green_cover_rating);
            return (
              <Link
                key={it.id}
```

```
                to={`/analysis/${it.id}`}
                className="group flex flex-col rounded-2xl border
border-border bg-card p-5 transition-all hover:-translate-y-1
hover:shadow-[0_14px_44px_-20px_rgba(31,27,22,0.28)]"
              >
                <div className="flex items-center justify-between">
                  <span className={`flex items-center gap-2 rounded-full px-3
py-1 text-xs font-medium ${tone.bg} ${tone.text}`}>
                    <span className="h-2 w-2 rounded-full" style={{
background: tone.dot }} />
                    {it.green_cover_rating || '—'}
                  </span>
                  <span className="text-xs text-muted-foreground">
                    {new Date(it.created_date).toLocaleDateString(undefined,
{ day: 'numeric', month: 'short', year: 'numeric' })}
                  </span>
                </div>
                <h3 className="mt-4 line-clamp-2 font-display text-lg
font-semibold text-foreground">{it.location_name}</h3>
                <p className="mt-1 text-xs
text-muted-foreground">{formatArea(it.area_hectares)} analyzed</p>

                <div className="mt-4 grid grid-cols-3 gap-2 text-center">
                  <div className="rounded-lg bg-muted/50 py-2">
                    <p className="font-display text-lg font-semibold
text-land-vegetation">{formatNumber(it.vegetation_pct, 0)}%</p>
                    <p className="text-[10px] uppercase tracking-wide
text-muted-foreground">Green</p>
                  </div>
                  <div className="rounded-lg bg-muted/50 py-2">
                    <p className="font-display text-lg font-semibold
text-foreground">{formatNumber(it.sustainability_score, 0)}</p>
                    <p className="text-[10px] uppercase tracking-wide
text-muted-foreground">Sustain</p>
                  </div>
                  <div className="rounded-lg bg-muted/50 py-2">
                    <p className="font-display text-lg font-semibold
text-foreground">{formatNumber(it.carbon_absorption_tons_year, 0)}</p>
                    <p className="text-[10px] uppercase tracking-wide
text-muted-foreground">t CO₂/yr</p>
                  </div>
                </div>

                <div className="mt-4 flex items-center gap-1 text-sm
font-medium text-primary">
```

```
                  Open report
                  <ArrowRight className="h-4 w-4 transition-transform
group-hover:translate-x-1" />
                </div>
              </Link>
            );
          })}
        </div>
      )}
    </div>
  );
}
```

Home.jsx

```
import { Link } from 'react-router-dom';
import { Image } from '@/components/ui/image';
import { ScanLine, BarChart3, MapPin, ArrowRight, Globe2, Layers, Leaf } from
'lucide-react';

const HERO_IMG =
'https://media.base44.com/images/public/6ace50ef1a61b33024edc57e/9cb44d9f2_generated_image.png';

const features = [
  {
    icon: ScanLine,
    title: 'Interactive area selection',
    body: 'Search any location worldwide and draw a precise boundary on live
satellite imagery to define exactly what gets assessed.',
    featured: true,
  },
  {
    icon: Layers,
    title: 'AI land-cover classification',
    body: 'Vegetation, buildings, roads, open land and water — identified and
quantified as indicative cover percentages.',
  },
  {
    icon: BarChart3,
    title: 'Carbon & sustainability dashboard',
    body: 'Green-cover statistics, estimated carbon impact, absorption
potential and a composite sustainability score.',
  },
  {
    icon: Globe2,
    title: 'Built for planners',
```

```
    body: 'Colleges, industries, developers and environmental organizations
get a fast, shareable environmental read on any region.',
    wide: true,
  },
];

const steps = [
  { n: '01', title: 'Locate', body: 'Search for a campus, neighbourhood, site
or city and fly to it on the map.' },
  { n: '02', title: 'Outline', body: 'Draw the boundary of the area you want
assessed directly on the satellite view.' },
  { n: '03', title: 'Assess', body: 'Get an indicative land-cover, carbon and
sustainability report in seconds.' },
];

export default function Home() {
  return (
    <div>
      {/* Hero */}
      <section className="relative overflow-hidden border-b
border-border/60">
        <div className="bg-grid pointer-events-none absolute inset-0
opacity-50" />
        <div className="relative mx-auto max-w-7xl px-4 pb-20 pt-16 sm:px-6
sm:pt-24 lg:px-8">
          <div className="grid items-center gap-12 lg:grid-cols-2">
            <div className="animate-fade-up">
              <div className="flex items-center gap-3">
                <span className="h-px w-8 bg-primary" />
                <span className="text-xs font-medium uppercase
tracking-[0.22em] text-primary">
                  Geospatial environmental intelligence
                </span>
              </div>
              <h1 className="mt-6 font-display text-5xl font-semibold
leading-[1.04] tracking-tight text-foreground text-balance sm:text-6xl
lg:text-[4.25rem]">
                See the green, the grey, and the carbon behind any place.
              </h1>
              <p className="mt-6 max-w-xl text-lg leading-relaxed
text-muted-foreground">
                GreenScope turns satellite imagery and a drawn boundary into
a clear environmental report — land-cover distribution, carbon impact and
sustainability, in seconds.
              </p>
```

```
              <div className="mt-9 flex flex-col gap-3 sm:flex-row">
                <Link
                  to="/analyze"
                  className="inline-flex items-center justify-center gap-2
rounded-full bg-primary px-7 py-3.5 text-sm font-medium
text-primary-foreground transition-all hover:opacity-90"
                >
                  <MapPin className="h-4 w-4" />
                  Analyze an area
                </Link>
                <Link
                  to="/history"
                  className="inline-flex items-center justify-center gap-2
rounded-full border border-border bg-card px-7 py-3.5 text-sm font-medium
text-foreground transition-all hover:bg-secondary"
                >
                  View past reports
                </Link>
              </div>
            </div>

            <div className="relative animate-fade-in">
              <div className="absolute -inset-4 rounded-[2rem]
bg-gradient-to-br from-land-vegetation/25 via-primary/10 to-transparent
blur-2xl" />
              <div className="relative overflow-hidden rounded-[1.75rem]
border border-border shadow-[0_30px_80px_-30px_rgba(26,32,29,0.4)]">
                <Image src={HERO_IMG} alt="Aerial view of a varied landscape"
className="aspect-[4/3] w-full" fittingType="fill" />
                <div className="absolute bottom-4 left-4 right-4 flex
items-center justify-between rounded-2xl glass px-4 py-3">
                  <div className="flex items-center gap-2.5">
                    <span className="flex h-8 w-8 items-center justify-center
rounded-lg bg-land-vegetation/20 text-land-vegetation">
                      <Leaf className="h-4 w-4" />
                    </span>
                    <div>
                      <p className="text-xs font-medium text-foreground">Live
classification</p>
                      <p className="text-[10px]
text-muted-foreground">Vegetation · Built · Roads · Open · Water</p>
                    </div>
                  </div>
                  <span className="rounded-full bg-land-vegetation/15 px-2.5
py-1 text-[10px] font-medium text-land-vegetation">42% green</span>
```

```
                </div>
              </div>
            </div>
          </div>

          {/* Stat row */}
          <div className="mt-16 grid grid-cols-3 divide-x divide-border
border-y border-border">
            <div className="px-2 py-7 text-center sm:px-6">
              <p className="font-display text-3xl font-semibold
text-foreground sm:text-4xl">5</p>
              <p className="mt-1 text-xs uppercase tracking-[0.16em]
text-muted-foreground sm:text-sm">land-cover classes</p>
            </div>
            <div className="px-2 py-7 text-center sm:px-6">
              <p className="font-display text-3xl font-semibold
text-foreground sm:text-4xl">∞</p>
              <p className="mt-1 text-xs uppercase tracking-[0.16em]
text-muted-foreground sm:text-sm">locations worldwide</p>
            </div>
            <div className="px-2 py-7 text-center sm:px-6">
              <p className="font-display text-3xl font-semibold
text-foreground sm:text-4xl">~10s</p>
              <p className="mt-1 text-xs uppercase tracking-[0.16em]
text-muted-foreground sm:text-sm">to a full report</p>
            </div>
          </div>
        </div>
      </section>

      {/* Bento features */}
      <section className="bg-background">
        <div className="mx-auto max-w-7xl px-4 py-20 sm:px-6 lg:px-8">
          <div className="max-w-2xl">
            <p className="text-xs font-medium uppercase tracking-[0.2em]
text-primary">What it does</p>
            <h2 className="mt-3 font-display text-3xl font-semibold
tracking-tight text-foreground sm:text-4xl">
              From a boundary on a map to an environmental read.
            </h2>
          </div>
          <div className="mt-12 grid grid-cols-1 gap-5 md:grid-cols-6">
            {features.map((f) => (
              <div
                key={f.title}
```

```
                className={`group relative overflow-hidden rounded-2xl border
border-border bg-card p-7 transition-all hover:-translate-y-1
hover:shadow-[0_14px_44px_-20px_rgba(26,32,29,0.28)] ${
                  f.featured ? 'md:col-span-3 md:row-span-2' : f.wide ?
'md:col-span-6' : 'md:col-span-3'
                }`}
              >
                {f.featured && (
                  <div className="bg-grid-fine pointer-events-none absolute
inset-0 opacity-40" />
                )}
                <div className="relative">
                  <span className="flex h-11 w-11 items-center justify-center
rounded-xl bg-primary/10 text-primary">
                    <f.icon className="h-5 w-5" strokeWidth={1.7} />
                  </span>
                  <h3 className={`mt-5 font-display font-semibold
text-foreground ${f.featured ? 'text-2xl' : 'text-lg'}`}>{f.title}</h3>
                  <p className={`mt-2 leading-relaxed text-muted-foreground
${f.featured ? 'text-base max-w-sm' : 'text-sm'}`}>{f.body}</p>
                  {f.featured && (
                    <Link
                      to="/analyze"
                      className="mt-6 inline-flex items-center gap-2
rounded-full bg-primary px-5 py-2.5 text-sm font-medium
text-primary-foreground transition-all hover:opacity-90"
                    >
                      Try it now <ArrowRight className="h-4 w-4" />
                    </Link>
                  )}
                </div>
              </div>
            ))}
          </div>
        </div>
      </section>

      {/* How it works */}
      <section className="border-t border-border/60 bg-card/30">
        <div className="mx-auto max-w-7xl px-4 py-20 sm:px-6 lg:px-8">
          <div className="max-w-2xl">
            <p className="text-xs font-medium uppercase tracking-[0.2em]
text-primary">How it works</p>
            <h2 className="mt-3 font-display text-3xl font-semibold
tracking-tight text-foreground sm:text-4xl">
```

```
              Three steps to an environmental report.
            </h2>
          </div>
          <div className="mt-12 grid grid-cols-1 gap-6 md:grid-cols-3">
            {steps.map((s) => (
              <div key={s.n} className="relative rounded-2xl border
border-border bg-card p-7">
                <span className="font-display text-5xl font-semibold
text-primary/20">{s.n}</span>
                <h3 className="mt-4 font-display text-xl font-semibold
text-foreground">{s.title}</h3>
                <p className="mt-2 text-sm leading-relaxed
text-muted-foreground">{s.body}</p>
              </div>
            ))}
          </div>
        </div>
      </section>

      {/* CTA */}
      <section className="border-t border-border/60">
        <div className="mx-auto max-w-5xl px-4 py-20 sm:px-6 lg:px-8">
          <div className="relative overflow-hidden rounded-[2rem] border
border-border bg-gradient-to-br from-land-vegetation/15 via-card
to-primary/10 p-10 text-center sm:p-16">
            <div className="bg-grid pointer-events-none absolute inset-0
opacity-40" />
            <div className="relative">
              <h2 className="font-display text-3xl font-semibold
tracking-tight text-foreground text-balance sm:text-4xl">
                Put a number on the environment of any place.
              </h2>
              <p className="mx-auto mt-4 max-w-xl text-muted-foreground">
                Draw a boundary. Get green cover, land use and carbon
indicators. Save and compare over time.
              </p>
              <Link
                to="/analyze"
                className="mt-8 inline-flex items-center gap-2 rounded-full
bg-primary px-7 py-3.5 text-sm font-medium text-primary-foreground
transition-all hover:opacity-90"
              >
                Start an analysis <ArrowRight className="h-4 w-4" />
              </Link>
            </div>
```

```
          </div>
        </div>
      </section>
    </div>
  );
}
```

Login.jsx

```
import React, { useState } from "react";
import { Link } from "react-router-dom";
import { base44 } from "@/api/base44Client";
import { Button } from "@/components/ui/button";
import { Input } from "@/components/ui/input";
import { Label } from "@/components/ui/label";
import { LogIn, Mail, Lock, Loader2 } from "lucide-react";
import AuthLayout from "@/components/AuthLayout";
import GoogleIcon from "@/components/GoogleIcon";
import { safeReturnTo } from "@/lib/authReturnTo";

export default function Login() {
  const [email, setEmail] = useState("");
  const [password, setPassword] = useState("");
  const [error, setError] = useState("");
  const [loading, setLoading] = useState(false);
  // Post-login destination (e.g. the MCP OAuth consent page sends users here
  // with returnTo so the grant flow can resume). Same-origin paths only.
  const returnTo = safeReturnTo();

  const handleSubmit = async (e) => {
    e.preventDefault();
    setError("");
    setLoading(true);
    try {
      await base44.auth.loginViaEmailPassword(email, password);
      window.location.href = returnTo;
    } catch (err) {
      setError(err.message || "Invalid email or password");
    } finally {
      setLoading(false);
    }
  };

  const handleGoogle = () => {
    base44.auth.loginWithProvider("google", returnTo);
  };
```

```
  return (
    <AuthLayout
      icon={LogIn}
      title="Welcome back"
      subtitle="Log in to your account"
      footer={
        <>
          Don't have an account?{" "}
          <Link
            to={"/register" + (returnTo !== "/" ? "?returnTo=" +
encodeURIComponent(returnTo) : "")}
            className="text-primary font-medium hover:underline"
          >
            Create one
          </Link>
        </>
      }
    >
      <Button
        variant="outline"
        className="w-full h-12 text-sm font-medium mb-6"
        onClick={handleGoogle}
      >
        <GoogleIcon className="w-5 h-5 mr-2" />
        Continue with Google
      </Button>

      <div className="relative mb-6">
        <div className="absolute inset-0 flex items-center">
          <div className="w-full border-t border-border" />
        </div>
        <div className="relative flex justify-center text-xs uppercase">
          <span className="bg-card px-3 text-muted-foreground">or</span>
        </div>
      </div>

      {error && (
        <div className="mb-4 p-3 rounded-lg bg-destructive/10
text-destructive text-sm">
          {error}
        </div>
      )}

      <form onSubmit={handleSubmit} className="space-y-4">
        <div className="space-y-2">
```

```
          <Label htmlFor="email">Email</Label>
          <div className="relative">
            <Mail className="absolute left-3 top-1/2 -translate-y-1/2 w-4 h-4
text-muted-foreground" aria-hidden="true" />
            <Input
              id="email"
              type="email"
              autoComplete="email"
              autoFocus
              placeholder="you@example.com"
              value={email}
              onChange={(e) => setEmail(e.target.value)}
              className="pl-10 h-12"
              required
            />
          </div>
        </div>
        <div className="space-y-2">
          <div className="flex items-center justify-between">
            <Label htmlFor="password">Password</Label>
            <Link to="/forgot-password" className="text-xs text-primary
hover:underline">
              Forgot password?
            </Link>
          </div>
          <div className="relative">
            <Lock className="absolute left-3 top-1/2 -translate-y-1/2 w-4 h-4
text-muted-foreground" aria-hidden="true" />
            <Input
              id="password"
              type="password"
              autoComplete="current-password"
              placeholder="••••••••"
              value={password}
              onChange={(e) => setPassword(e.target.value)}
              className="pl-10 h-12"
              required
            />
          </div>
        </div>
        <Button type="submit" className="w-full h-12 font-medium"
disabled={loading}>
          {loading ? (
            <>
              <Loader2 className="w-4 h-4 mr-2 animate-spin" />
```

```
              Logging in...
            </>
          ) : (
            "Log in"
          )}
        </Button>
      </form>
    </AuthLayout>
  );
}
```

OAuthConsent.jsx

```
import React, { useEffect, useState } from "react";
import { appParams } from "@/lib/app-params";
import { Button } from "@/components/ui/button";
import { ShieldCheck, Loader2 } from "lucide-react";
import AuthLayout from "@/components/AuthLayout";

const OPERATION_VERBS = { read: "view", create: "create", update: "edit",
delete: "delete" };
const COLLAPSED_DATA_ROWS = 6;

// Entity tools arrive one per operation; show each entity once, naming only
the
// operations it actually grants.
function groupTools(tools) {
  const operationsByEntity = new Map();
  const agents = [];
  const actions = [];
  for (const tool of tools) {
    if (tool.kind === "entity" && tool.entity) {
      if (!operationsByEntity.has(tool.entity))
operationsByEntity.set(tool.entity, []);
      operationsByEntity.get(tool.entity).push(tool.operation);
    } else if (tool.kind === "agent") {
      agents.push(tool);
    } else {
      actions.push(tool);
    }
  }
  const data = [...operationsByEntity].map(([entity, operations]) => ({
    entity,
    operations: Object.keys(OPERATION_VERBS).filter((op) =>
operations.includes(op)),
  }));
```

```
  return { data, agents, actions };
}

const humanize = (name) =>
  name.split(/[_-]+/).filter(Boolean).map((word) => word[0].toUpperCase() +
word.slice(1)).join(" ") || name;

// "View, create and delete records" — delete is emphasized so it doesn't
hide
// among harmless verbs.
function OperationSummary({ operations }) {
  return (
    <>
      {operations.map((op, i) => {
        const verb = OPERATION_VERBS[op];
        const separator = i === 0 ? "" : i === operations.length - 1 ? " and
" : ", ";
        return (
          <React.Fragment key={op}>
            {separator}
            <span className={op === "delete" ? "font-medium text-destructive"
: undefined}>
              {i === 0 ? verb[0].toUpperCase() + verb.slice(1) : verb}
            </span>
          </React.Fragment>
        );
      })}{" "}
      records
    </>
  );
}

function ToolSection({ title, children }) {
  return (
    <section>
      <h3 className="text-xs font-medium uppercase tracking-wide
text-muted-foreground mb-2">{title}</h3>
      <ul className="divide-y divide-border rounded-lg border
border-border">{children}</ul>
    </section>
  );
}

function ToolRow({ title, children }) {
  return (
```

```
    <li className="flex flex-col gap-0.5 px-3 py-2">
      <span className="font-medium text-foreground">{title}</span>
      {children && <span className="text-muted-foreground">{children}</span>}
    </li>
  );
}

// App-side OAuth consent page for the app's MCP server. The platform
redirects
// AI clients here (see base44/mcp/config.json `consent_path`) with an opaque
// `ctx` handle — the authorization request itself lives on the server. This
page
// gates on the app-user session, fetches the display info for that handle,
shows
// the categories of access being granted, and posts the approve/deny
decision.
// Do not change the fetch calls, headers, or the `ctx` handle handling —
styling
// and copy are safe to edit.
export default function OAuthConsent() {
  const ctx = new URLSearchParams(window.location.search).get("ctx");
  const [info, setInfo] = useState(null);
  const [checking, setChecking] = useState(true);
  const [submitting, setSubmitting] = useState(false);
  const [decided, setDecided] = useState("");
  const [error, setError] = useState("");
  const [reconnect, setReconnect] = useState("");
  const [showAllData, setShowAllData] = useState(false);

  useEffect(() => {
    (async () => {
      let redirecting = false;
      try {
        if (!ctx) {
          setError("This authorization link is invalid or has expired.");
          return;
        }
        // Resolve the handle first: a dead handle must never render
        // approve/deny, and the response carries the app's configured login
        // route for the signed-out redirect below. Send the session (cookie
+
        // bearer token) so the server can list the granted tools for a
        // signed-in user — the same auth the approve/deny call sends;
without
        // it the display request is anonymous and shows no tools.
```

```
        const infoHeaders = {};
        if (appParams.token) infoHeaders.Authorization = "Bearer " +
appParams.token;
        const res = await fetch(

`/api/apps/${appParams.appId}/mcp/consent-info?handle=${encodeURIComponent(ct
x)}`,
          { credentials: "include", headers: infoHeaders },
        );
        if (!res.ok) {
          setError("This authorization link is invalid or has expired.");
          return;
        }
        const data = await res.json();
        // Gate on the server's auth result, NOT
base44.auth.isAuthenticated():
        // the SDK check runs the bearer path, so a cookie-only session
(platform
        // login/SSO, or a private app with a stale localStorage token) would
read
        // as signed-out and redirect — even though /consent-info just
        // authenticated this same request via its cookie fallback.
data.authenticated
        // keeps the redirect decision in agreement with what the server
returned.
        if (!data.authenticated) {
          // The short handle rides back in returnTo; login_path is
          // owner-configured and validated server-side as a same-origin
path.
          // Send from_url too: a custom-auth app coerced to platform auth
(e.g.
          // public_without_login under workspace SSO) serves the platform
login,
          // which honors from_url rather than returnTo. Rebuild the query
from
          // `ctx` alone — never forward window.location.search raw: the
platform
          // resume returns from_url verbatim, so crafted extras on the
consent
          // link (app_base_url, access_token, …) would ride through the
login
          // round-trip and app-params.js would persist them into the freshly
          // authenticated session.
          const returnTo =
            window.location.pathname + "?ctx=" + encodeURIComponent(ctx);
```

```
          const encoded = encodeURIComponent(returnTo);
          redirecting = true; // keep the spinner while the browser navigates
          window.location.href =
            (data.login_path || "/login") + "?returnTo=" + encoded +
"&from_url=" + encoded;
          return;
        }
        setInfo(data);
      } catch (e) {
        setError("Could not load this authorization request. Please try
again.");
      } finally {
        if (!redirecting) setChecking(false);
      }
    })();
  }, [ctx]);

  const respond = async (action) => {
    setSubmitting(true);
    setError("");
    try {
      const headers = { "Content-Type": "application/json" };
      // Cookie-backed sessions carry no token; sending "Bearer null" would
      // shadow the valid cookie, so add the header only when a token exists.
      if (appParams.token) headers.Authorization = "Bearer " +
appParams.token;
      const res = await
fetch(`/api/apps/${appParams.appId}/mcp/authorize-grant`, {
        method: "POST",
        credentials: "include",
        headers,
        body: JSON.stringify({ ctx, action }),
      });
      if (!res.ok) {
        // 401 = the session expired before the (single-use,
still-unconsumed)
        // handle was spent; retrying the same controls re-sends the dead
session
        // forever. Send the user back through login preserving `ctx` — the
same
        // redirect the initial signed-out path uses — so they can return and
        // approve the still-valid handle.
        if (res.status === 401) {
          const returnTo = window.location.pathname + "?ctx=" +
encodeURIComponent(ctx);
```

```
          const encoded = encodeURIComponent(returnTo);
          window.location.href =
            ((info && info.login_path) || "/login") + "?returnTo=" + encoded
+ "&from_url=" + encoded;
          return;
        }
        // These all come AFTER the single-use handle is atomically consumed
        // (409 tool set changed; 403 host/resource/app mismatch; 404 access
        // gone; 400 malformed/handle already used), so retrying can only
404.
        // Show a terminal reconnect state, not an impossible "try again".
        if ([400, 403, 404, 409].includes(res.status)) {
          let detail = "";
          try { detail = (await res.json()).detail; } catch (_) { /* keep
default */ }
          setReconnect(detail || "This authorization can no longer be
completed. Reconnect from your AI client to try again.");
          setSubmitting(false);
          return;
        }
        throw new Error("Could not complete authorization. Please try
again.");
      }
      const data = await res.json();
      window.location.href = data.redirect_url;
      if (!/^https?:/i.test(data.redirect_url)) {
        // Custom-scheme redirect (native AI clients, e.g. cursor://):
browsers
        // may block or not visibly navigate, so show a terminal state
instead
        // of an eternal spinner.
        setDecided(action);
        setSubmitting(false);
      }
    } catch (e) {
      setError(e.message);
      setSubmitting(false);
    }
  };

  if (checking) {
    return (
      <AuthLayout icon={ShieldCheck} title="Authorize access">
        <div className="flex items-center justify-center py-6
text-muted-foreground">
```

```
          <Loader2 className="w-5 h-5 mr-2 animate-spin" aria-hidden="true"
/>
          Loading…
        </div>
      </AuthLayout>
    );
  }

  const client = (info && info.client_name) || "An AI client";
  const appName = (info && info.app_name) || "this app";

  if (decided) {
    return (
      <AuthLayout
        icon={ShieldCheck}
        title={decided === "approve" ? "Access granted" : "Access denied"}
        subtitle={`You can return to ${client} and close this window.`}
      />
    );
  }

  // Terminal: the authorization request is no longer valid (tool set changed
+
  // handle consumed). Retrying can't succeed, so show reconnect guidance
with
  // no approve/deny controls.
  if (reconnect) {
    return (
      <AuthLayout icon={ShieldCheck} title="Reconnect required">
        <div className="p-3 rounded-lg bg-destructive/10 text-destructive
text-sm">
          {reconnect}
        </div>
      </AuthLayout>
    );
  }

  // No consent details means nothing trustworthy to approve: a failed
  // consent-info load (expired handle, rate limit, transient error) renders
  // the error alone, never the approve/deny controls.
  if (error && !info) {
    return (
      <AuthLayout icon={ShieldCheck} title="Authorize access">
        <div className="p-3 rounded-lg bg-destructive/10 text-destructive
text-sm">
```

```
          {error}
        </div>
      </AuthLayout>
    );
  }

  const tools = Array.isArray(info.tools) ? info.tools : [];
  const { data, agents, actions } = groupTools(tools);

  return (
    <AuthLayout
      icon={ShieldCheck}
      title="Authorize access"
      subtitle={`${client} wants to access ${appName} on your behalf`}
    >
      {error && (
        <div className="mb-4 p-3 rounded-lg bg-destructive/10
text-destructive text-sm">
          {error}
        </div>
      )}

      <p className="text-sm font-medium text-foreground mb-3">
        {tools.length ? `${client} will be able to:` : "No tools requested"}
      </p>
      {tools.length > 0 && (
        <div className="space-y-4 text-sm mb-6">
          {data.length > 0 && (
            <ToolSection title={`Data (${data.length})`}>
              {(showAllData ? data : data.slice(0,
COLLAPSED_DATA_ROWS)).map(({ entity, operations }) => (
                <ToolRow key={entity} title={entity}>
                  <OperationSummary operations={operations} />
                </ToolRow>
              ))}
              {data.length > COLLAPSED_DATA_ROWS && (
                <li className="px-3 py-2">
                  <button
                    type="button"
                    className="text-sm font-medium text-primary
hover:underline"
                    onClick={() => setShowAllData((shown) => !shown)}
                  >
                    {showAllData ? "Show less" : `Show all ${data.length}`}
                  </button>
```

```
                </li>
              )}
            </ToolSection>
          )}
          {agents.length > 0 && (
            <ToolSection title="Agents">
              {agents.map((tool) => (
                <ToolRow key={tool.name} title={tool.agent ?
humanize(tool.agent) : tool.title || tool.name}>
                  {tool.description}
                </ToolRow>
              ))}
            </ToolSection>
          )}
          {actions.length > 0 && (
            <ToolSection title="Actions">
              {actions.map((tool) => (
                <ToolRow key={tool.name} title={tool.title || tool.name}>
                  {tool.description}
                </ToolRow>
              ))}
            </ToolSection>
          )}
        </div>
      )}

      <div className="flex gap-3">
        <Button
          variant="outline"
          className="flex-1 h-12 font-medium"
          disabled={submitting}
          onClick={() => respond("deny")}
        >
          Deny
        </Button>
        <Button
          className="flex-1 h-12 font-medium"
          disabled={submitting}
          onClick={() => respond("approve")}
        >
          {submitting ? <Loader2 className="w-4 h-4 mr-2 animate-spin" /> :
null}
          Approve
        </Button>
      </div>
```

```
    </AuthLayout>
  );
}
```

Register.jsx

```
import React, { useState } from "react";
import { Link } from "react-router-dom";
import { base44 } from "@/api/base44Client";
import { Button } from "@/components/ui/button";
import { Input } from "@/components/ui/input";
import { Label } from "@/components/ui/label";
import { UserPlus, Mail, Lock, Loader2 } from "lucide-react";
import { InputOTP, InputOTPGroup, InputOTPSlot } from
"@/components/ui/input-otp";
import AuthLayout from "@/components/AuthLayout";
import GoogleIcon from "@/components/GoogleIcon";
import { toast } from "@/components/ui/use-toast";
import { safeReturnTo } from "@/lib/authReturnTo";

export default function Register() {
  const [email, setEmail] = useState("");
  const [password, setPassword] = useState("");
  const [confirmPassword, setConfirmPassword] = useState("");
  const [error, setError] = useState("");
  const [loading, setLoading] = useState(false);
  const [showOtp, setShowOtp] = useState(false);
  const [otpCode, setOtpCode] = useState("");

  const handleSubmit = async (e) => {
    e.preventDefault();
    setError("");
    if (password !== confirmPassword) {
      setError("Passwords do not match");
      return;
    }
    setLoading(true);
    try {
      await base44.auth.register({ email, password });
      setShowOtp(true);
    } catch (err) {
      setError(err.message || "Registration failed");
    } finally {
      setLoading(false);
    }
  };
```

```
  const handleVerify = async () => {
    setError("");
    setLoading(true);
    try {
      const result = await base44.auth.verifyOtp({ email, otpCode });
      if (result?.access_token) {
        base44.auth.setToken(result.access_token);
      }
      window.location.href = safeReturnTo();
    } catch (err) {
      setError(err.message || "Invalid verification code");
    } finally {
      setLoading(false);
    }
  };

  const handleResend = async () => {
    setError("");
    try {
      await base44.auth.resendOtp(email);
      toast({
        title: "Code sent",
        description: "Check your email for the new code.",
      });
    } catch (err) {
      setError(err.message || "Failed to resend code");
    }
  };

  const handleGoogle = () => {
    base44.auth.loginWithProvider("google", safeReturnTo());
  };

  if (showOtp) {
    return (
      <AuthLayout
        icon={Mail}
        title="Verify your email"
        subtitle={`We sent a code to ${email}`}
      >
        {error && (
          <div className="mb-4 p-3 rounded-lg bg-destructive/10
text-destructive text-sm">
            {error}
```

```
          </div>
        )}
        <div className="flex justify-center mb-6">
          <InputOTP
            maxLength={6}
            value={otpCode}
            onChange={setOtpCode}
            autoFocus
            autoComplete="one-time-code"
          >
            <InputOTPGroup>
              <InputOTPSlot index={0} />
              <InputOTPSlot index={1} />
              <InputOTPSlot index={2} />
              <InputOTPSlot index={3} />
              <InputOTPSlot index={4} />
              <InputOTPSlot index={5} />
            </InputOTPGroup>
          </InputOTP>
        </div>
        <Button
          className="w-full h-12 font-medium"
          onClick={handleVerify}
          disabled={loading || otpCode.length < 6}
        >
          {loading ? (
            <>
              <Loader2 className="w-4 h-4 mr-2 animate-spin" />
              Verifying...
            </>
          ) : (
            "Verify"
          )}
        </Button>
        <p className="text-center text-sm text-muted-foreground mt-4">
          Didn't receive the code?{" "}
          <button onClick={handleResend} className="text-primary font-medium
hover:underline">
            Resend
          </button>
        </p>
      </AuthLayout>
    );
  }
```

```
  return (
    <AuthLayout
      icon={UserPlus}
      title="Create your account"
      subtitle="Sign up to get started"
      footer={
        <>
          Already have an account?{" "}
          <Link
            to={"/login" + (safeReturnTo() !== "/" ? "?returnTo=" +
encodeURIComponent(safeReturnTo()) : "")}
            className="text-primary font-medium hover:underline"
          >
            Log in
          </Link>
        </>
      }
    >
      <Button
        variant="outline"
        className="w-full h-12 text-sm font-medium mb-6"
        onClick={handleGoogle}
      >
        <GoogleIcon className="w-5 h-5 mr-2" />
        Continue with Google
      </Button>

      <div className="relative mb-6">
        <div className="absolute inset-0 flex items-center">
          <div className="w-full border-t border-border" />
        </div>
        <div className="relative flex justify-center text-xs uppercase">
          <span className="bg-card px-3 text-muted-foreground">or</span>
        </div>
      </div>

      {error && (
        <div className="mb-4 p-3 rounded-lg bg-destructive/10
text-destructive text-sm">
          {error}
        </div>
      )}

      <form onSubmit={handleSubmit} className="space-y-4">
        <div className="space-y-2">
```

```
          <Label htmlFor="email">Email</Label>
          <div className="relative">
            <Mail className="absolute left-3 top-1/2 -translate-y-1/2 w-4 h-4
text-muted-foreground" aria-hidden="true" />
            <Input
              id="email"
              type="email"
              autoComplete="email"
              autoFocus
              placeholder="you@example.com"
              value={email}
              onChange={(e) => setEmail(e.target.value)}
              className="pl-10 h-12"
              required
            />
          </div>
        </div>
        <div className="space-y-2">
          <Label htmlFor="password">Password</Label>
          <div className="relative">
            <Lock className="absolute left-3 top-1/2 -translate-y-1/2 w-4 h-4
text-muted-foreground" aria-hidden="true" />
            <Input
              id="password"
              type="password"
              autoComplete="new-password"
              placeholder="••••••••"
              value={password}
              onChange={(e) => setPassword(e.target.value)}
              className="pl-10 h-12"
              required
            />
          </div>
        </div>
        <div className="space-y-2">
          <Label htmlFor="confirm">Confirm Password</Label>
          <div className="relative">
            <Lock className="absolute left-3 top-1/2 -translate-y-1/2 w-4 h-4
text-muted-foreground" aria-hidden="true" />
            <Input
              id="confirm"
              type="password"
              autoComplete="new-password"
              placeholder="••••••••"
              value={confirmPassword}
```

```
              onChange={(e) => setConfirmPassword(e.target.value)}
              className="pl-10 h-12"
              required
            />
          </div>
        </div>
        <Button type="submit" className="w-full h-12 font-medium"
disabled={loading}>
          {loading ? (
            <>
              <Loader2 className="w-4 h-4 mr-2 animate-spin" />
              Creating account...
            </>
          ) : (
            "Create account"
          )}
        </Button>
      </form>
    </AuthLayout>
  );
}
```

ResetPassword.jsx

```
import React, { useState } from "react";
import { Link, useSearchParams } from "react-router-dom";
import { base44 } from "@/api/base44Client";
import { Button } from "@/components/ui/button";
import { Input } from "@/components/ui/input";
import { Label } from "@/components/ui/label";
import { Lock, Loader2, AlertTriangle } from "lucide-react";
import AuthLayout from "@/components/AuthLayout";

export default function ResetPassword() {
  const [searchParams] = useSearchParams();
  const resetToken = searchParams.get("token");

  const [newPassword, setNewPassword] = useState("");
  const [confirmPassword, setConfirmPassword] = useState("");
  const [error, setError] = useState("");
  const [loading, setLoading] = useState(false);

  const handleSubmit = async (e) => {
    e.preventDefault();
    setError("");
    if (newPassword !== confirmPassword) {
```

```
      setError("Passwords do not match");
      return;
    }
    setLoading(true);
    try {
      await base44.auth.resetPassword({ resetToken, newPassword });
      window.location.href = "/login";
    } catch (err) {
      setError(err.message || "Failed to reset password");
    } finally {
      setLoading(false);
    }
  };

  if (!resetToken) {
    return (
      <AuthLayout
        icon={AlertTriangle}
        title="Invalid reset link"
        subtitle="This password reset link is missing or invalid"
        footer={
          <Link to="/forgot-password" className="text-primary font-medium
hover:underline">
            Request a new link
          </Link>
        }
      >
        <p className="text-sm text-foreground text-center">
          The link you used appears to be incomplete. Please request a new
password reset email.
        </p>
      </AuthLayout>
    );
  }

  return (
    <AuthLayout
      icon={Lock}
      title="New password"
      subtitle="Enter your new password below"
    >
      {error && (
        <div className="mb-4 p-3 rounded-lg bg-destructive/10
text-destructive text-sm">
          {error}
```

```
        </div>
      )}
      <form onSubmit={handleSubmit} className="space-y-4">
        <div className="space-y-2">
          <Label htmlFor="password">New Password</Label>
          <div className="relative">
            <Lock className="absolute left-3 top-1/2 -translate-y-1/2 w-4 h-4
text-muted-foreground" aria-hidden="true" />
            <Input
              id="password"
              type="password"
              autoComplete="new-password"
              autoFocus
              placeholder="••••••••"
              value={newPassword}
              onChange={(e) => setNewPassword(e.target.value)}
              className="pl-10 h-12"
              required
            />
          </div>
        </div>
        <div className="space-y-2">
          <Label htmlFor="confirm">Confirm Password</Label>
          <div className="relative">
            <Lock className="absolute left-3 top-1/2 -translate-y-1/2 w-4 h-4
text-muted-foreground" aria-hidden="true" />
            <Input
              id="confirm"
              type="password"
              autoComplete="new-password"
              placeholder="••••••••"
              value={confirmPassword}
              onChange={(e) => setConfirmPassword(e.target.value)}
              className="pl-10 h-12"
              required
            />
          </div>
        </div>
        <Button type="submit" className="w-full h-12 font-medium"
disabled={loading}>
          {loading ? (
            <>
              <Loader2 className="w-4 h-4 mr-2 animate-spin" />
              Resetting...
            </>
```

```
          ) : (
            "Reset password"
          )}
        </Button>
      </form>
    </AuthLayout>
  );
}
```

/utils

index.ts

```
export function createPageUrl(pageName: string) {
    return '/' + pageName.replace(/ /g, '-');
}
```

App.jsx

```
import { Toaster } from "@/components/ui/toaster"
import { QueryClientProvider } from '@tanstack/react-query'
import { queryClientInstance } from '@/lib/query-client'
import { BrowserRouter as Router, Route, Routes } from 'react-router-dom';
import PageNotFound from './lib/PageNotFound';
import { AuthProvider, useAuth } from '@/lib/AuthContext';
import UserNotRegisteredError from '@/components/UserNotRegisteredError';
import ScrollToTop from './components/ScrollToTop';
import ProtectedRoute from '@/components/ProtectedRoute';
import AppLayout from '@/components/AppLayout';
import Home from '@/pages/Home';
import Analyze from '@/pages/Analyze';
import History from '@/pages/History';
import AnalysisDetail from '@/pages/AnalysisDetail';
import Login from '@/pages/Login';
import Register from '@/pages/Register';
import ForgotPassword from '@/pages/ForgotPassword';
import ResetPassword from '@/pages/ResetPassword';
import { Navigate } from 'react-router-dom';

const AuthenticatedApp = () => {
  const { isLoadingAuth, isLoadingPublicSettings, authError, navigateToLogin
} = useAuth();

  // Show loading spinner while checking app public settings or auth
  if (isLoadingPublicSettings || isLoadingAuth) {
    return (
      <div className="fixed inset-0 flex items-center justify-center">
```

```
        <div className="w-8 h-8 border-4 border-slate-200 border-t-slate-800
rounded-full animate-spin"></div>
      </div>
    );
  }

  // Handle authentication errors
  if (authError) {
    if (authError.type === 'user_not_registered') {
      return <UserNotRegisteredError />;
    } else if (authError.type === 'auth_required') {
      // Redirect to login automatically
      navigateToLogin();
      return null;
    }
  }

  // Render the main app
  return (
    <Routes>
      <Route path="/login" element={<Login />} />
      <Route path="/register" element={<Register />} />
      <Route path="/forgot-password" element={<ForgotPassword />} />
      <Route path="/reset-password" element={<ResetPassword />} />
      <Route element={<ProtectedRoute unauthenticatedElement={<Navigate
to="/login" replace />} />}>
        <Route element={<AppLayout />}>
          <Route path="/" element={<Home />} />
          <Route path="/analyze" element={<Analyze />} />
          <Route path="/history" element={<History />} />
          <Route path="/analysis/:id" element={<AnalysisDetail />} />
        </Route>
      </Route>
      <Route path="*" element={<PageNotFound />} />
    </Routes>
  );
};


function App() {

  return (
    <AuthProvider>
      <QueryClientProvider client={queryClientInstance}>
        <Router>
```

```
          <ScrollToTop />
          <AuthenticatedApp />
        </Router>
        <Toaster />
      </QueryClientProvider>
    </AuthProvider>
  )
}

export default App
```

index.css

```
@tailwind base;
@tailwind components;
@tailwind utilities;

@layer base {
  :root {
    --background: 44 34% 95%;
    --foreground: 152 14% 13%;
    --card: 40 46% 99%;
    --card-foreground: 152 14% 13%;
    --popover: 40 46% 99%;
    --popover-foreground: 152 14% 13%;
    --primary: 152 36% 25%;
    --primary-foreground: 44 40% 97%;
    --secondary: 44 24% 90%;
    --secondary-foreground: 152 14% 16%;
    --muted: 44 22% 91%;
    --muted-foreground: 152 8% 42%;
    --accent: 96 20% 44%;
    --accent-foreground: 44 40% 97%;
    --destructive: 8 64% 48%;
    --destructive-foreground: 44 40% 97%;
    --border: 44 18% 86%;
    --input: 44 18% 86%;
    --ring: 152 36% 25%;
    --chart-1: 152 34% 38%;
    --chart-2: 190 32% 42%;
    --chart-3: 42 34% 60%;
    --chart-4: 150 8% 32%;
    --chart-5: 96 20% 46%;
    --radius: 0.75rem;
    --font-heading: 'Fraunces', ui-serif, Georgia, serif;
    --font-body: 'Inter', ui-sans-serif, system-ui, sans-serif;
```

```
    --font-display: 'Fraunces', ui-serif, Georgia, serif;
    --font-mono: ui-monospace, SFMono-Regular, Menlo, Monaco, Consolas,
"Liberation Mono", "Courier New", monospace;
    --sidebar-background: 40 46% 99%;
    --sidebar-foreground: 152 14% 16%;
    --sidebar-primary: 152 36% 25%;
    --sidebar-primary-foreground: 44 40% 97%;
    --sidebar-accent: 44 24% 92%;
    --sidebar-accent-foreground: 152 14% 16%;
    --sidebar-border: 44 18% 86%;
    --sidebar-ring: 152 36% 25%;
  }

  .dark {
    --background: 160 18% 8%;
    --foreground: 44 30% 90%;
    --card: 158 16% 11%;
    --card-foreground: 44 30% 90%;
    --popover: 158 16% 11%;
    --popover-foreground: 44 30% 90%;
    --primary: 152 34% 52%;
    --primary-foreground: 160 20% 10%;
    --secondary: 158 12% 18%;
    --secondary-foreground: 44 30% 90%;
    --muted: 158 12% 18%;
    --muted-foreground: 44 12% 62%;
    --accent: 96 22% 52%;
    --accent-foreground: 160 20% 10%;
    --destructive: 8 58% 52%;
    --destructive-foreground: 44 40% 97%;
    --border: 158 12% 22%;
    --input: 158 12% 22%;
    --ring: 152 34% 52%;
    --chart-1: 152 34% 52%;
    --chart-2: 190 32% 52%;
    --chart-3: 42 40% 66%;
    --chart-4: 0 0% 70%;
    --chart-5: 96 22% 56%;
    --sidebar-background: 158 16% 11%;
    --sidebar-foreground: 44 30% 90%;
    --sidebar-primary: 152 34% 52%;
    --sidebar-primary-foreground: 160 20% 10%;
    --sidebar-accent: 158 12% 18%;
    --sidebar-accent-foreground: 44 30% 90%;
    --sidebar-border: 158 12% 22%;
```

```
    --sidebar-ring: 152 34% 52%;
  }
}

@layer base {
  * {
    @apply border-border outline-ring/50;
  }

  body {
    @apply bg-background text-foreground font-body antialiased;
    font-feature-settings: "ss01", "cv01", "cv11";
  }

  ::selection {
    background-color: hsl(var(--primary) / 0.22);
    color: hsl(var(--foreground));
  }
}

@layer utilities {
  .font-display {
    font-family: var(--font-display);
    font-optical-sizing: auto;
    letter-spacing: -0.02em;
  }
  .text-balance {
    text-wrap: balance;
  }
  .grain {
    background-image: radial-gradient(hsl(var(--foreground) / 0.03) 1px,
transparent 1px);
    background-size: 4px 4px;
  }
  .bg-grid {
    background-image:
      linear-gradient(hsl(var(--foreground) / 0.045) 1px, transparent 1px),
      linear-gradient(90deg, hsl(var(--foreground) / 0.045) 1px, transparent
1px);
    background-size: 28px 28px;
  }
  .bg-grid-fine {
    background-image:
      linear-gradient(hsl(var(--foreground) / 0.05) 1px, transparent 1px),
```

```
      linear-gradient(90deg, hsl(var(--foreground) / 0.05) 1px, transparent
1px);
    background-size: 14px 14px;
  }
  .glass {
    background-color: hsl(var(--card) / 0.72);
    backdrop-filter: blur(14px) saturate(140%);
  }
  .leaflet-container {
    font-family: var(--font-body);
    background: hsl(44 22% 88%);
  }
  .leaflet-control-attribution {
    font-size: 10px !important;
    background: hsl(var(--card) / 0.7) !important;
    backdrop-filter: blur(4px);
  }
}
```

main.jsx

```
import React from 'react'
import ReactDOM from 'react-dom/client'
import App from '@/App.jsx'
import '@/index.css'

ReactDOM.createRoot(document.getElementById('root')).render(
  <App />
)
```

.gitignore

```
#env
.env
.env.*

# Logs
/logs
*.log
npm-debug.log*
yarn-debug.log*
yarn-error.log*
pnpm-debug.log*
lerna-debug.log*

node_modules
```

```
dist
dist-ssr
*.local

# Nitro build output, once this app has a server. Both are regenerated by
# every build, and the deploy runs one before reading them.
.output
.wrangler

# Editor directories and files
.vscode/*
!.vscode/extensions.json
.idea
.DS_Store
*.suo
*.ntvs*
*.njsproj
*.sln
*.sw?

.env
.vite

# Local link to a Base44 app, written by `base44 link`
base44/.app.jsonc
```

AGENTS.md

```
# AGENTS.md

## Project Context

This is a Base44 app repository. Treat it as user-owned application code,
keep changes focused on the user's request, and preserve existing project
conventions.

Start with `README.md` for local setup, environment variables, and publish
workflow.

## Base44 References

- CLI overview:
https://docs.base44.com/developers/references/cli/get-started/overview.md
- Agent skills: https://docs.base44.com/developers/backend/overview/skills.md

If your agent supports Agent Skills, install or update Base44 skills before
Base44-specific work:
```

```bash
npx skills add base44/skills
```

Key Files

- `src/`: frontend application source.
- `src/api/base44Client.js`: frontend Base44 SDK client.
- `vite.config.js`: Vite config and Base44 Vite plugin setup.
- `.env.local`: local-only environment values; never commit secrets.

Working Notes

- Use `base44 dev` as the default local development command when you need the local Base44 backend. It can run the backend and frontend together.
- When docs or code mention the frontend being started automatically, that usually means the Base44 project config includes `site.serveCommand`, for example `"serveCommand": "npm run dev"` in `base44/config.jsonc`.
- Use `npm run dev` only for frontend-only work against the hosted Base44 backend.
- Prefer the existing Base44 CLI workflow over adding new npm scripts for Base44-specific tasks.
- Reuse the existing SDK client and Vite plugin patterns before adding new Base44 integration paths.
- Run the relevant checks from `package.json` before finishing code changes.

CLAUDE.md

```
# See AGENTS.md

Follow the instructions in `AGENTS.md`.
```

components.json

```
{
  "$schema": "https://ui.shadcn.com/schema.json",
  "style": "new-york",
  "rsc": false,
  "tsx": false,
  "tailwind": {
    "config": "tailwind.config.js",
    "css": "src/index.css",
    "baseColor": "neutral",
    "cssVariables": true,
    "prefix": ""
  },
```

```
  "aliases": {
    "components": "@/components",
    "utils": "@/lib/utils",
    "ui": "@/components/ui",
    "lib": "@/lib",
    "hooks": "@/hooks"
  },
  "iconLibrary": "lucide"
}
```

eslint.config.js

```
import globals from "globals";
import pluginJs from "@eslint/js";
import pluginReact from "eslint-plugin-react";
import pluginReactHooks from "eslint-plugin-react-hooks";
import pluginUnusedImports from "eslint-plugin-unused-imports";

export default [
  {
    files: [
      "src/components/**/*.{js,mjs,cjs,jsx}",
      "src/pages/**/*.{js,mjs,cjs,jsx}",
      "src/Layout.jsx",
    ],
    ignores: ["src/lib/**/*", "src/components/ui/**/*"],
    ...pluginJs.configs.recommended,
    ...pluginReact.configs.flat.recommended,
    languageOptions: {
      globals: globals.browser,
      parserOptions: {
        ecmaVersion: 2022,
        sourceType: "module",
        ecmaFeatures: {
          jsx: true,
        },
      },
    },
    settings: {
      react: {
        version: "detect",
      },
    },
    plugins: {
      react: pluginReact,
      "react-hooks": pluginReactHooks,
      "unused-imports": pluginUnusedImports,
```

```
    },
    rules: {
      "no-unused-vars": "off",
      "react/jsx-uses-vars": "error",
      "react/jsx-uses-react": "error",
      "unused-imports/no-unused-imports": "error",
      "unused-imports/no-unused-vars": [
        "warn",
        {
          vars: "all",
          varsIgnorePattern: "^_",
          args: "after-used",
          argsIgnorePattern: "^_",
        },
      ],
      "react/prop-types": "off",
      "react/react-in-jsx-scope": "off",
      "react/no-unknown-property": [
        "error",
        { ignore: ["cmdk-input-wrapper", "toast-close"] },
      ],
      "react-hooks/rules-of-hooks": "error",
    },
  },
];
```

index.html

```
<!doctype html>
<html lang="en">
  <head>
    <meta charset="UTF-8" />
    <link rel="icon" type="image/svg+xml"
href="https://base44.com/logo_v2.svg" />
    <meta name="viewport" content="width=device-width, initial-scale=1.0" />
    <meta name="description" content="GreenScope AI — AI-powered geospatial
analysis of green cover, land use, and carbon impact for any area on Earth."
/>
    <link rel="manifest" href="/manifest.json" />
    <link rel="preconnect" href="https://fonts.googleapis.com" />
    <link rel="preconnect" href="https://fonts.gstatic.com" crossorigin />
    <link
href="https://fonts.googleapis.com/css2?family=Fraunces:opsz,wght@9..144,400;
9..144,500;9..144,600;9..144,700&family=Inter:wght@300;400;500;600;700&displa
y=swap" rel="stylesheet" />
    <title>GreenScope AI — Geospatial Environmental Intelligence</title>
```

```
  </head>
  <body>
    <div id="root"></div>
    <script type="module" src="/src/main.jsx"></script>
  </body>
</html>
```

jsconfig.json

```
{
  "compilerOptions": {
    "baseUrl": ".",
    "paths": {
      "@/*": ["./src/*"]
    },
    "jsx": "react-jsx",
    "module": "esnext",
    "moduleResolution": "bundler",
    "lib": ["esnext", "dom"],
    "target": "esnext",
    "checkJs": true,
    "skipLibCheck": true,
    "allowSyntheticDefaultImports": true,
    "esModuleInterop": true,
    "resolveJsonModule": true,
    "types": []
  },
  "include": ["src/components/**/*.js", "src/pages/**/*.jsx",
"src/Layout.jsx"],
  "exclude": ["node_modules", "dist", "src/vite-plugins",
"src/components/ui", "src/api", "src/lib"]
}
```

package.json

```
{
  "name": "base44-app",
  "private": true,
  "version": "0.0.0",
  "type": "module",
  "scripts": {
    "dev": "vite",
    "build": "vite build",
    "lint": "eslint . --quiet",
    "lint:fix": "eslint . --fix",
    "typecheck": "tsc -p ./jsconfig.json",
    "preview": "vite preview"
```

```
  },
  "dependencies": {
    "@base44/sdk": "^0.8.57",
    "@base44/vite-plugin": "^1.0.47",
    "@hello-pangea/dnd": "^17.0.0",
    "@hookform/resolvers": "^4.1.2",
    "@radix-ui/react-accordion": "^1.2.3",
    "@radix-ui/react-alert-dialog": "^1.1.6",
    "@radix-ui/react-aspect-ratio": "^1.1.2",
    "@radix-ui/react-avatar": "^1.1.3",
    "@radix-ui/react-checkbox": "^1.1.4",
    "@radix-ui/react-collapsible": "^1.1.3",
    "@radix-ui/react-context-menu": "^2.2.6",
    "@radix-ui/react-dialog": "^1.1.6",
    "@radix-ui/react-dropdown-menu": "^2.1.6",
    "@radix-ui/react-hover-card": "^1.1.6",
    "@radix-ui/react-label": "^2.1.2",
    "@radix-ui/react-menubar": "^1.1.6",
    "@radix-ui/react-navigation-menu": "^1.2.5",
    "@radix-ui/react-popover": "^1.1.6",
    "@radix-ui/react-progress": "^1.1.2",
    "@radix-ui/react-radio-group": "^1.2.3",
    "@radix-ui/react-scroll-area": "^1.2.3",
    "@radix-ui/react-select": "^2.1.6",
    "@radix-ui/react-separator": "^1.1.2",
    "@radix-ui/react-slider": "^1.2.3",
    "@radix-ui/react-slot": "^1.1.2",
    "@radix-ui/react-switch": "^1.1.3",
    "@radix-ui/react-tabs": "^1.1.3",
    "@radix-ui/react-toast": "^1.2.2",
    "@radix-ui/react-toggle": "^1.1.2",
    "@radix-ui/react-toggle-group": "^1.1.2",
    "@radix-ui/react-tooltip": "^1.1.8",
    "@stripe/react-stripe-js": "^3.0.0",
    "@stripe/stripe-js": "^5.2.0",
    "@tanstack/react-query": "^5.84.1",
    "canvas-confetti": "^1.9.4",
    "class-variance-authority": "^0.7.1",
    "clsx": "^2.1.1",
    "cmdk": "^1.0.0",
    "date-fns": "^3.6.0",
    "embla-carousel-react": "^8.5.2",
    "framer-motion": "^11.16.4",
    "html2canvas": "^1.4.1",
    "input-otp": "^1.4.2",
```

```
    "jspdf": "^4.2.1",
    "lodash": "^4.17.21",
    "lucide-react": "^0.475.0",
    "moment": "^2.30.1",
    "next-themes": "^0.4.4",
    "react": "^18.2.0",
    "react-day-picker": "^8.10.1",
    "react-dom": "^18.2.0",
    "react-hook-form": "^7.54.2",
    "react-hot-toast": "^2.6.0",
    "react-leaflet": "^4.2.1",
    "react-markdown": "^9.0.1",
    "react-quill-new": "^3.8.3",
    "react-resizable-panels": "^2.1.7",
    "react-router-dom": "^6.26.0",
    "recharts": "^2.15.4",
    "sonner": "^2.0.1",
    "tailwind-merge": "^3.0.2",
    "tailwindcss-animate": "^1.0.7",
    "three": "^0.171.0",
    "vaul": "^1.1.2",
    "zod": "^3.24.2"
  },
  "devDependencies": {
    "@eslint/js": "^9.19.0",
    "@types/node": "^22.13.5",
    "@types/react": "^18.2.66",
    "@types/react-dom": "^18.2.22",
    "@vitejs/plugin-react": "^6.0.5",
    "autoprefixer": "^10.4.20",
    "baseline-browser-mapping": "^2.8.32",
    "eslint": "^9.19.0",
    "eslint-plugin-react": "^7.37.4",
    "eslint-plugin-react-hooks": "^5.0.0",
    "eslint-plugin-react-refresh": "^0.4.18",
    "eslint-plugin-unused-imports": "^4.3.0",
    "globals": "^15.14.0",
    "nitro": "^3.0.260610-beta",
    "postcss": "^8.5.3",
    "tailwindcss": "^3.4.17",
    "typescript": "^5.8.2",
    "vite": "^8.2.0"
  }
}
```

postcss.config.js

```
export default {
  plugins: {
    tailwindcss: {},
    autoprefixer: {},
  },
}
```

README.md

```
# Base44 Project

Use this repository to run and edit the app locally, then publish changes
back through Base44.

Any change pushed to the repo will also be reflected in the Base44 Builder.

## Prerequisites

1. Clone the repository using the project's Git URL.
2. Navigate to the project directory.
3. Install dependencies: `npm install`.
4. Install the Base44 CLI: `npm install -g base44@latest`.
5. Install
[Deno](https://docs.deno.com/runtime/getting_started/installation/) — the
local Base44 backend runs on it.

Run `base44 --help` (or see the [CLI
reference](https://docs.base44.com/developers/references/cli/commands/introdu
ction)) for the full command surface.

## Run Locally

Three commands, from the project root:

```bash
base44 login    # one-time per machine
base44 link     # one-time per clone
base44 dev      # local backend + frontend together
```

Open the frontend URL that `base44 dev` prints (typically
`http://localhost:5173`).

Notes:
```

- **Every fresh clone needs `base44 link`.** It writes `base44/.app.jsonc` (the app-id pointer), which is deliberately gitignored. Your app id is in the Builder URL (`app.base44.com/apps/<id>/...`); `base44 link --help` shows the non-interactive flags.
- **`base44 dev` runs the frontend for you** (via `site.serveCommand` in this repo's `base44/config.jsonc`) — never run `npm run dev` yourself: alone it serves a UI with no backend behind it (`[base44] Proxy not enabled`, every `/api` call fails), and alongside `base44 dev` the second Vite silently takes the next port and you end up looking at the wrong one.
- **The app must be published at least once for the UI to load under `base44 dev`.** The frontend boots by fetching app settings from the hosted app; before the first publish that fails and every page redirects to login. The local API works regardless.
- Entities, functions, and auth run locally — entity data is **in-memory only**, wiped when `base44 dev` restarts. Everything else (Core integrations, OAuth login) is forwarded to your deployed app. Full breakdown: [Local development overview](https://docs.base44.com/developers/backend/overview/local-dev/local-development-overview).

Frontend Only, Hosted Backend

To work on just the frontend against your app's live hosted backend:

```bash
base44 dev --remote
```

⚠️ In this mode writes go to your app's **production data** — plain `base44 dev` keeps everything local.

Publish Your Changes

After pushing your changes to git, open the Base44 dashboard and publish the app:

```bash
base44 dashboard open
```

This repo syncs to Base44 through git, so publish from the dashboard rather than `base44 deploy` — a CLI deploy ships your local tree directly, bypassing the sync, and the deployed state silently diverges from the repo.

```
## Docs & Support

GitHub integration:
[https://docs.base44.com/developers/app-code/local-development/github](https://docs.base44.com/developers/app-code/local-development/github)

Local development:
[https://docs.base44.com/developers/backend/overview/local-dev/local-development-overview](https://docs.base44.com/developers/backend/overview/local-dev/local-development-overview)

Support: [https://app.base44.com/support](https://app.base44.com/support)
```

tailwind.config.js

```
# Base44 Project

Use this repository to run and edit the app locally, then publish changes back through Base44.

Any change pushed to the repo will also be reflected in the Base44 Builder.

## Prerequisites

1. Clone the repository using the project's Git URL.
2. Navigate to the project directory.
3. Install dependencies: `npm install`.
4. Install the Base44 CLI: `npm install -g base44@latest`.
5. Install [Deno](https://docs.deno.com/runtime/getting_started/installation/) — the local Base44 backend runs on it.

Run `base44 --help` (or see the [CLI reference](https://docs.base44.com/developers/references/cli/commands/introduction)) for the full command surface.

## Run Locally

Three commands, from the project root:

```bash
base44 login   # one-time per machine
base44 link    # one-time per clone
base44 dev     # local backend + frontend together
```
```

Open the frontend URL that `base44 dev` prints (typically `http://localhost:5173`).

Notes:

- **Every fresh clone needs `base44 link`.** It writes `base44/.app.jsonc` (the app-id pointer), which is deliberately gitignored. Your app id is in the Builder URL (`app.base44.com/apps/<id>/...`); `base44 link --help` shows the non-interactive flags.
- **`base44 dev` runs the frontend for you** (via `site.serveCommand` in this repo's `base44/config.jsonc`) — never run `npm run dev` yourself: alone it serves a UI with no backend behind it (`[base44] Proxy not enabled`, every `/api` call fails), and alongside `base44 dev` the second Vite silently takes the next port and you end up looking at the wrong one.
- **The app must be published at least once for the UI to load under `base44 dev`.** The frontend boots by fetching app settings from the hosted app; before the first publish that fails and every page redirects to login. The local API works regardless.
- Entities, functions, and auth run locally — entity data is **in-memory only**, wiped when `base44 dev` restarts. Everything else (Core integrations, OAuth login) is forwarded to your deployed app. Full breakdown: [Local development overview](https://docs.base44.com/developers/backend/overview/local-dev/local-development-overview).

Frontend Only, Hosted Backend

To work on just the frontend against your app's live hosted backend:

```bash
base44 dev --remote
```

⚠️ In this mode writes go to your app's **production data** — plain `base44 dev` keeps everything local.

Publish Your Changes

After pushing your changes to git, open the Base44 dashboard and publish the app:

```bash
base44 dashboard open
```

```
This repo syncs to Base44 through git, so publish from the dashboard rather than `base44 deploy` — a CLI deploy ships your local tree directly, bypassing the sync, and the deployed state silently diverges from the repo.

## Docs & Support

GitHub integration:
[https://docs.base44.com/developers/app-code/local-development/github](https://docs.base44.com/developers/app-code/local-development/github)

Local development:
[https://docs.base44.com/developers/backend/overview/local-dev/local-development-overview](https://docs.base44.com/developers/backend/overview/local-dev/local-development-overview)

Support: [https://app.base44.com/support](https://app.base44.com/support)
```

tailwind.config.js

```
/** @type {import('tailwindcss').Config} */
module.exports = {
    darkMode: ["class"],
    content: ["./index.html", "./src/**/*.{ts,tsx,js,jsx}"],
  theme: {
    extend: {
        opacity: Object.fromEntries(Array.from({ length: 101 }, (_, i) => [i, `${i / 100}`])),
        borderRadius: {
            lg: 'var(--radius)',
            md: 'calc(var(--radius) - 2px)',
            sm: 'calc(var(--radius) - 4px)'
        },
        colors: {
            background: 'hsl(var(--background))',
            foreground: 'hsl(var(--foreground))',
            card: {
                DEFAULT: 'hsl(var(--card))',
                foreground: 'hsl(var(--card-foreground))'
            },
            popover: {
                DEFAULT: 'hsl(var(--popover))',
                foreground: 'hsl(var(--popover-foreground))'
            },
            primary: {
                DEFAULT: 'hsl(var(--primary))',
                foreground: 'hsl(var(--primary-foreground))'
```

```
            },
            secondary: {
                DEFAULT: 'hsl(var(--secondary))',
                foreground: 'hsl(var(--secondary-foreground))'
            },
            muted: {
                DEFAULT: 'hsl(var(--muted))',
                foreground: 'hsl(var(--muted-foreground))'
            },
            accent: {
                DEFAULT: 'hsl(var(--accent))',
                foreground: 'hsl(var(--accent-foreground))'
            },
            destructive: {
                DEFAULT: 'hsl(var(--destructive))',
                foreground: 'hsl(var(--destructive-foreground))'
            },
            border: 'hsl(var(--border))',
            input: 'hsl(var(--input))',
            ring: 'hsl(var(--ring))',
            chart: {
                '1': 'hsl(var(--chart-1))',
                '2': 'hsl(var(--chart-2))',
                '3': 'hsl(var(--chart-3))',
                '4': 'hsl(var(--chart-4))',
                '5': 'hsl(var(--chart-5))'
            },
            land: {
                vegetation: '#4f7a5a',
                built: '#5a6168',
                roads: '#2c2f33',
                open: '#c9b98e',
                water: '#3f7e8a'
            },
            sidebar: {
                DEFAULT: 'hsl(var(--sidebar-background))',
                foreground: 'hsl(var(--sidebar-foreground))',
                primary: 'hsl(var(--sidebar-primary))',
                'primary-foreground':
'hsl(var(--sidebar-primary-foreground))',
                accent: 'hsl(var(--sidebar-accent))',
                'accent-foreground': 'hsl(var(--sidebar-accent-foreground))',
                border: 'hsl(var(--sidebar-border))',
                ring: 'hsl(var(--sidebar-ring))'
            }
```

```
        },
        fontFamily: {
            heading: ['var(--font-heading)'],
            body: ['var(--font-body)'],
            display: ['var(--font-display)'],
            mono: ['var(--font-mono)']
        },
        keyframes: {
            'accordion-down': {
                from: { height: '0' },
                to: { height: 'var(--radix-accordion-content-height)' }
            },
            'accordion-up': {
                from: { height: 'var(--radix-accordion-content-height)' },
                to: { height: '0' }
            },
            'fade-up': {
                from: { opacity: '0', transform: 'translateY(12px)' },
                to: { opacity: '1', transform: 'translateY(0)' }
            },
            'fade-in': {
                from: { opacity: '0' },
                to: { opacity: '1' }
            }
        },
        animation: {
            'accordion-down': 'accordion-down 0.2s ease-out',
            'accordion-up': 'accordion-up 0.2s ease-out',
            'fade-up': 'fade-up 0.6s cubic-bezier(0.22, 1, 0.36, 1) both',
            'fade-in': 'fade-in 0.5s ease-out both'
        }
    }
  },
  plugins: [require("tailwindcss-animate")],
}
```

vite.config.js

```
import base44 from "@base44/vite-plugin"
import react from '@vitejs/plugin-react'
import { defineConfig } from 'vite'

// https://vite.dev/config/
export default defineConfig({
  plugins: [
    base44({
```

```
      // Support for legacy code that imports the base44 SDK with
@/integrations, @/entities, etc.
      // can be removed if the code has been updated to use the new SDK
imports from @base44/sdk
      legacySDKImports: process.env.BASE44_LEGACY_SDK_IMPORTS === 'true',
      hmrNotifier: true,
      navigationNotifier: true,
      analyticsTracker: true,
      visualEditAgent: true
    }),
    react(),
  ]
});
```